Jet Ranger Arduino and C# Program Summary

Master document for the Bell 206 Jet Ranger cockpit: Arduino sketches, Windows programs and PCBs

Regenerated 19 October 2026

Contents

1 Project overview....6
1.1 How it fits together....6
1.2 Arduino sketches....7
1.3 Windows programs....8
1.4 PCBs....8
1.5 Where things stand....9
2 Colour codes and cabling....11
2.1 Reading the diagrams....11
2.2 Colours in the programs....11
2.3 Cabling colour codes....12
2.3.1 Round power connectors....12
2.3.2 PCB connectors....12
2.4 OLED connectors....12
3 Network and communication....13
3.1 Devices and addresses....13
3.2 UDP ports....13
3.3 Packet format....14
3.4 Board life cycle....15
3.5 Power rules in the bridges....16
3.6 Backlight....17
3.7 Remote reset....18
3.8 Debug and monitoring....19
4 Radio controller (172.16.1.101)....20
4.1 Program flow....20
4.2 Pin usage....21
4.3 Data from the PC (UDP 13136)....22
4.4 Commands to the PC (UDP 27001)....22
4.5 Auto Pilot and Audio panel switches and knobs....22
4.6 Auto Pilot and Audio panel LEDs (MAX7219)....24
4.7 Tuning steps....24
4.8 Power and blanking....24
4.9 Diagnostics....24
4.10 PCB features the sketch does not use yet....25
4.11 Known issues....25
5 Servo controller (172.16.1.102)....26
5.1 Program flow....26
5.2 Servos and calibration....27
5.3 Warning lamps....28
5.4 Debounced switch inputs....28
5.5 On-demand commands....29
5.6 Backlight....29

5.7 Pin usage....29
5.8 Known issues....29
6 Upper controller (172.16.1.103)....30
6.1 Program flow....30
6.2 Pin usage....30
6.3 What it sends....31
6.4 Remote reset....31
6.5 Known issues....31
7 USB HID board (172.16.1.11)....32
7.1 Program flow....32
7.2 How a switch becomes a joystick button....33
7.3 Auto Pilot, Audio and ADF panel buttons....33
7.4 Switch mapping in use....33
7.5 Pin usage and network....34
7.6 Known issues....34
8 Stepper controller (172.16.1.105)....35
8.1 Gauges and calibration....35
8.2 Program flow....36
8.3 Packets....36
8.4 Backlight....36
8.5 On-demand swings....36
8.6 Pin usage....37
8.7 Network....37
8.8 Known issues....37
9 Dual stepper controller (172.16.1.106)....38
9.1 Gauges and calibration....38
9.2 OLED displays....39
9.3 Backlight....39
9.4 Program flow....40
9.5 Packets....40
9.6 On-demand commands....40
9.7 Pin usage....40
9.8 Network....40
9.9 Known issues....41
10 ADF controller (172.16.1.107)....42
10.1 Program flow....42
10.2 Pin usage....43
10.3 Panel brightness....43
10.4 Power monitoring....43
10.5 What it sends....44
10.6 Panel switches....44
10.7 What it receives....44
10.8 OLED....44

10.9 Notes....44
11 FSUIPC bridge (FSUIPCWinformsAutoCS)....46
11.1 Program flow....46
11.2 FSUIPC offsets....47
11.3 Packets sent....47
11.4 Aircraft information for the GPS interface....48
11.5 Commands from the Radio controller (UDP 27001)....48
11.6 Notes and quirks....48
12 MSFS bridge (MSFSWinformsAutoCS)....49
12.1 Files....49
12.2 Program flow....50
12.3 Packets....50
12.4 Aircraft information for the GPS interface....51
12.5 SimVars requested....51
12.6 Autopilot and audio panel LEDs....51
12.7 Commands from the panel (UDP 27001)....51
12.8 Aircraft profiles....52
12.8.1 FlyInside B206....52
12.8.2 CowaSim Jet Ranger....53
12.9 Warning lamps....53
12.10 Known gaps....54
13 Health Monitor (JetRangerHealthMonitor)....55
13.1 Program flow....55
14 Radio and Upper controller emulators....56
14.1 JetRangerRadioControllerEmulator....56
14.2 JetRangerUpperControllerEmulator....56
15 StepperVSITester....57
15.1 What it sends....57
16 SwingTester....58
16.1 How it works....58
16.2 Window....58
16.3 Brightness....59
16.4 Warning lamps....59
16.5 Keep-alive....59
17 Debug listener (UDP reflector)....60
18 Lowrance AirMap GPS interface (LowranceAirMapGps)....61
18.1 Behaviour....61
18.2 NMEA sentences....62
18.3 GPS date and time....62
18.4 Files....62
18.5 Status....62
19 GPS bench tester (LowranceAirMapGpsBenchTester)....63
19.1 Window....63

19.2 Testing....63
20 PCB notes....64
20.1 Boards in use....64
20.2 Backlight driver....66
20.3 Warning Mute, Fuel Valve and TOT Clear inputs....66
20.4 Pin and copper check....67
20.5 KiCad: changing the revision number....67
Appendix A: UDP token reference....68
A.1 Gauge tokens....68
A.2 Display and status tokens....68
A.3 Lamp tokens (Servo .102)....68
A.4 Control tokens (C,<COMMAND>:<value>)....69
A.5 Messages from the ADF controller to the PC (UDP 27003)....69
A.6 Aircraft information to the GPS interface (UDP 27002)....69
A.7 Messages from the Radio controller to the bridge (UDP 27001)....69
Appendix B: Building and flashing....70
B.1 Libraries....70
B.2 Compile and upload....70
B.3 COM port to board mapping....71
B.4 Sketch settings to know about....71
Appendix C: Calibration....72
Appendix D: How this document is built....73

1 Project overview

A home-built Bell 206 Jet Ranger cockpit: real instrument gauges, warning lamps, radio and overhead switch panels and small displays, driven from a flight simulator by Arduino Mega boards on a private Ethernet network. The simulator can be Microsoft Flight Simulator 2020 or 2024 (through SimConnect) or Prepar3D / FSX (through FSUIPC).

This document ties together the three parts of the repository: the **Arduino sketches**, the **Windows (C#) programs** and the **KiCad PCBs**. It is rebuilt from each project's own `PROGRAM_SUMMARY.md` and from the code, and it covers only the projects that are in use.

1.1 How it fits together

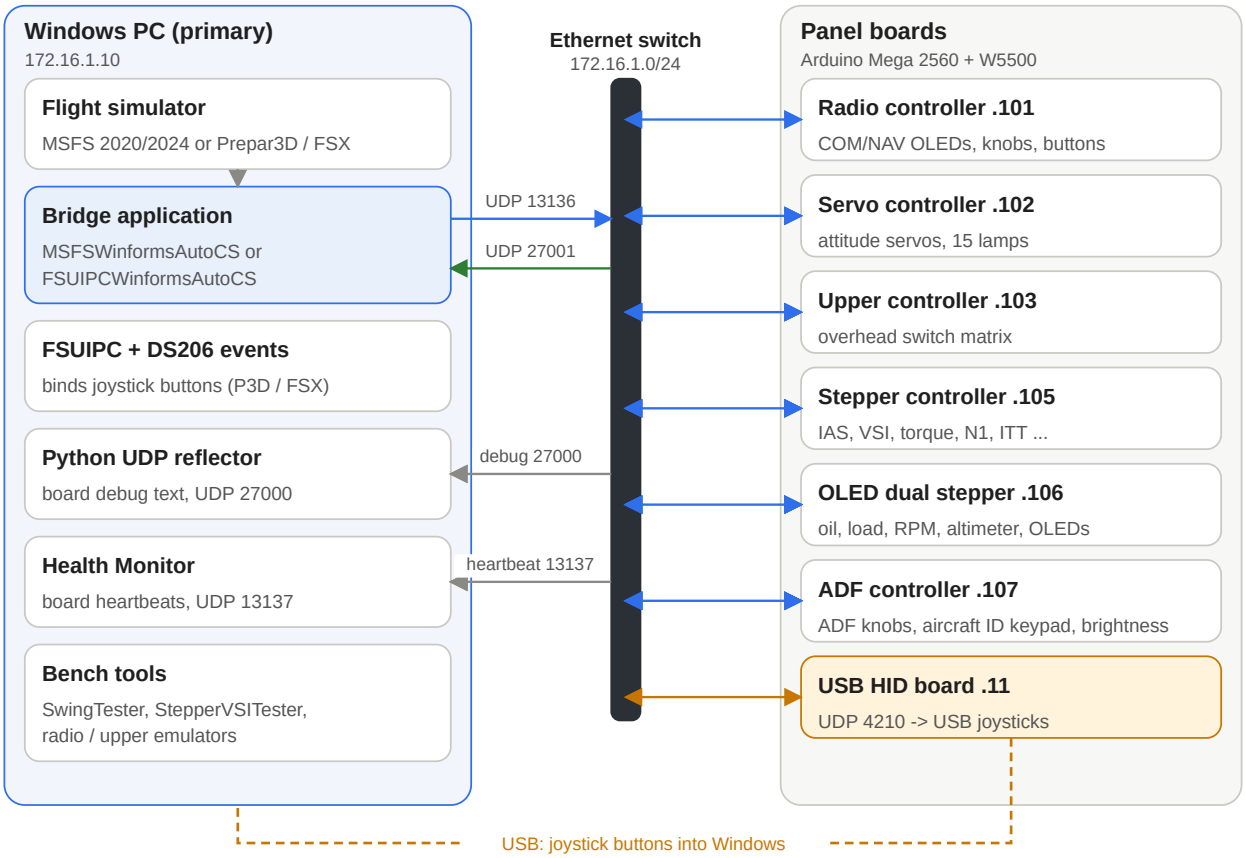


Figure 1 — The whole system: simulator, PC bridge, Ethernet and the panel boards

- The **simulator** supplies the flight data.
- A **bridge program** on the PC reads the data, converts it to real units and sends one short UDP text packet to each board about ten times a second.
- Each **Arduino board** turns the real units into stepper steps, servo positions, lamp states or display text with calibration tables in its own sketch.
- Switches and knobs go back the other way: the radio board sends commands to the bridge (UDP 27001), the ADF controller sends its frequency, radar follow ID and power status to the PC (UDP 27003, not read by the bridges yet), and the switch boards send joystick button changes to a USB HID board (UDP 4210) which Windows sees as game controllers.
- Every board sends a **heartbeat** every 10 seconds to the Health Monitor (UDP 13137) and its **debug text** to a listener on the PC (UDP 27000).

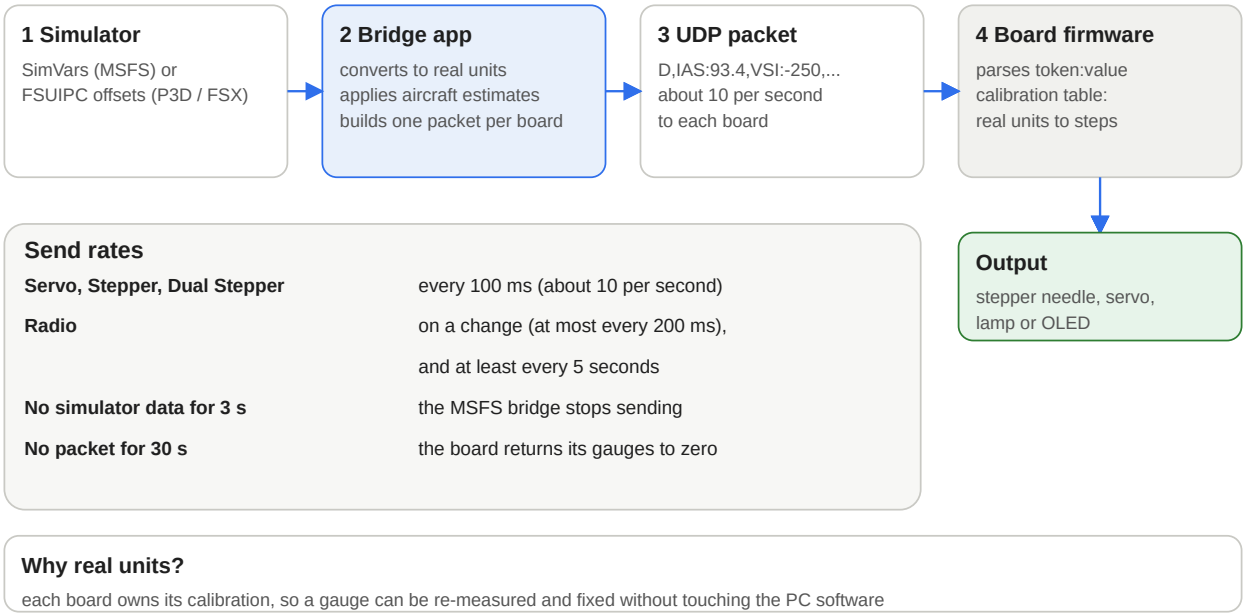


Figure 2 — From simulator value to moving needle

1.2 Arduino sketches

All sketches are in `Jet Ranger Arduino Sketches/`. Each is an Arduino Mega 2560 with a W5500 Ethernet module unless noted.

Sketch	IP	Role
`JET_RANGER_RADIO_CONTROLLER`	.101	COM and NAV frequency displays, eight tuning knobs, four frequency-swap switches; sends tuning and swap commands to the PC
`JET_RANGER_SERVO_CONTROLLER`	.102	Attitude indicator pitch and bank servos, 15 warning lamps, three front-panel switches (TOT clear, warning mute, fuel valve)
`JET_RANGER_UPPER_CONTROLLER`	.103	Overhead switch matrix (176 positions) and keypad; forwards changes to the USB HID board
`JET_RANGER_ADF_CONTROLLER`	.107	ADF major and minor knobs, aircraft ID keypad for radar following, panel brightness pot (sets the other panels' backlight), 12 V and 5 V rail monitoring with a status every 30 seconds; character OLED. Not flashed or bench-tested yet
`UDP_TO_HID`	.11	Leonardo-class board: UDP in, four 32-button joysticks and a keyboard out over USB
`JET_RANGER_STEPPER_CONTROLLER`	.105	Stepper gauges: airspeed, vertical speed, radar altitude, torque, gas producer, turbine out temperature, fuel quantity
`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER`	.106	Stepper gauges: engine and transmission oil, engine and rotor RPM, electrical load, fuel load, altimeter; four OLEDs (barometer, altimeter, clock, temperature)

Common to the three gauge boards (Servo, Stepper, Dual Stepper):

- bench-measured calibration tables with linear interpolation from real units to steps or servo positions, plus a raw bypass (`<CODE>RAW`) for calibration work;
- start-up self-test swings, also callable on demand (`C,SWINGTQ:1` and so on);
- a 30 second no-data watchdog that returns every gauge to zero;
- remote reset (`C,RESET:1`).

1.3 Windows programs

All programs are in `Jet Ranger/ C# Code/` (note the leading space in the folder name).

Program	Purpose
`FSUIPCWinformsAutoCS`	Bridge for Prepar3D / FSX using FSUIPC offsets; also sends position, altitude, airspeed, heading and the simulator date and time to the GPS interface
`MSFSWinformsAutoCS`	Bridge for MSFS 2020 / 2024 using SimConnect, with estimates for aircraft whose dials do not read the standard variables (FlyInside B206, CowaSim Jet Ranger); also feeds the GPS interface
`JetRangerHealthMonitor`	Traffic-light dashboard for the 10 second board heartbeats
`StepperVSITester`	Bench tester that sends real and raw values to the stepper boards and the servo board
`SwingTester`	Runs every gauge's swing individually or in a fixed order, lights the OLEDs and warning lamps, tests the Auto Pilot and Audio panel LEDs one by one, resets boards, and sets the backlight and LED brightness
`LowranceAirMapGps`	Makes the PC a GPS source for a Lowrance AirMap 2000C: NMEA over a serial port, fed by the bridges over UDP, defaulting to Brisbane Airport
`LowranceAirMapGpsBenchTester`	Bench sender for the GPS interface: typed values, 8-way cursor, country / city / airport selection, Fly mode and a date / time clock
`JetRangerRadioControllerEmulator`, `JetRangerUpperControllerEmulator`	Stand-ins for the radio and upper boards when they are disconnected

Python helpers in `Jet Ranger Python Code/`: a SimConnect exporter, a UDP test receiver and a wake-on-LAN script. The UDP debug listener (port 27000) is a small Python reflector that prints every board's debug text and can forward a copy to a second listener.

Run **only one bridge at a time**: `FSUIPCWinformsAutoCS` and `MSFSWinformsAutoCS` both listen on UDP 27001.

1.4 PCBs

KiCad projects are in `Jet Ranger/PCBs/`. The boards used by the sketches and programs above are:

PCB	Revision	Used by
Jet Ranger Radio Auto Pilot	1.3	Radio controller
Jet Ranger COM and NAV	no revision set	Radio panel (four swap switches, encoders)
Jet Ranger Front Panel Servo	1.4	Servo controller
Jet Ranger Upper Controller	1.4	Upper controller
Jet Ranger 13 Stepper	1.5	Stepper controller
Jet Ranger OLED Dual Stepper	1.5	Dual stepper controller
Jet Ranger X27 Small Stepper	not set	One per stepper gauge
Jet Ranger ADF Controller	1.4	ADF controller
Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel	1.0	Switch panels for the Radio Auto Pilot and ADF controllers
Annunciator, Altimeter, Large Guage, Simkits gauge boards		Other panels and gauges

On 18 October 2026 the six controller boards (Servo, Upper, 13 Stepper, Dual Stepper, Radio Auto Pilot and ADF Controller) were revised to fix the **backlight MOSFET**: the schematic symbol had source and drain swapped and the part is now an IRF540N. All six drive their panel backlights from Mega pin D5 through this transistor (see the PCB notes chapter).

Active PCBs, their revision and the sketches they belong to

Jet Ranger Radio Auto Pilot v1.3

plugs into: Radio controller (.101)
autopilot knobs, volume pots, MAX7219 display lines; backlight MOSFET on D5

Jet Ranger COM and NAV

plugs into: Radio controller (.101)
four swap switches in a diode matrix: column D26, rows D22-D25

Jet Ranger Front Panel Servo v1.4

plugs into: Servo controller (.102)
switches D43, D47, D48; backlight D5
MOSFET pin-out corrected in 1.4

Jet Ranger Upper Controller v1.4

plugs into: Upper controller (.103)
key matrix, brightness pot on A0,
backlight MOSFET on D5

Jet Ranger 13 Stepper v1.5

plugs into: Stepper controller (.105)
backlight D5, VSI coil B D8
IRF540N backlight MOSFET

Jet Ranger OLED Dual Stepper v1.5

plugs into: Dual stepper controller (.106)
D2 / D32 short removed in 1.4
IRF540N backlight MOSFET

Jet Ranger ADF Controller v1.4

plugs into: ADF controller (.107)
ADF knobs, keypad, brightness pot, 12 V monitor, gear lamps, I2C OLED, IRF540N backlight MOSFET

Audio, Auto Pilot and ADF panels v1.0

plugs into: switch panels on the controllers
diode-matrix switch boards, LEDs
and rotary encoders

COM and NAV (radio panel) has no revision set. Dual Stepper boards older than 1.4: fit only one of fuel quantity and fuel load.

Check every pin against the sketch pin table and the KiCad files before ordering or wiring a new board.

Figure 3 — The active PCBs, their revision and the sketches they belong to

1.5 Where things stand

Bench-verified with UDP debug logs from the boards and real swing timings: every swing command, the OLED lamp test, the temperature OLED, the keep-alive, the debounced servo inputs, the radio matrix, the remote reset on every board except the Upper controller, and the full SwingTester sequence. The FSUIPC bridge has been run against Prepar3D v6.1 and the MSFS bridge against MSFS 2024 with the FlyInside B206 and the CowaSim Jet Ranger.

Backlight control (Servo, Stepper and Dual Stepper boards) and the GPS interface were added on 18 October 2026: the backlight follows the main bus, the last commanded brightness and the 30 second no-data watchdog, and the GPS interface feeds a Lowrance AirMap 2000C from the simulator. Both are tested on the bench with packets; the GPS interface has not been tested against the real GPS, and the FSUIPC and MSFS bridges' GPS output has not been run against a simulator.

Open items:

- The three servo-board switches reach the HID board but are not sent to the PC itself (the bridges treat an unrecognised string on port 27001 as a radio frequency, so they would need updating first).
- The Upper controller PCB has a Warning Mute connector on D30, but the Upper sketch scans analog pins A9 to A15 for its keypad only, so it may not read it.
- The Upper controller's remote reset has not been bench-tested; the board was not connected.
- The estimates in the MSFS bridge for the FlyInside and CowaSim aircraft come from a few dial readings; check them at other power settings.

- The Radio controller and Upper controller PCBs have the backlight driver on D5 (and the Upper a brightness pot on A0), but their sketches do not use them yet.
- The Radio controller emulator still sends a bare frequency for every radio, so only COM1 tunes from it; the real board sends labelled COM2, NAV1 and NAV2 values, which both bridges understand.
- The Dual Stepper's RAM use is high (79 percent); be careful adding code to it.
- Calibration tables, swing times and waits come from bench measurements; recheck them after any mechanical change to a gauge.

2 Colour codes and cabling

This chapter explains the colours used in the diagrams and programs, and the colour codes used for cables and connectors.

2.1 Reading the diagrams

Each diagram box is shaded to say what kind of item it is. The same shades are used in every diagram.

Colour	What it means in the diagrams
White	An ordinary item: a board, program, step or detail
Blue	The subject of the figure or the main step: the bridge, the Health Monitor, the code that decides
Green	Running or healthy, power on, or an output: connected, the gauge or backlight being driven
Amber	Waiting, a warning or a special path: no data, the default position, the USB HID path
Red	Lost, off, failed or a reset
Gray	Supporting or inactive: notes, items that stay off
Purple	Tells boards or groups apart where the figure says so (for example the Dual Stepper in the Swing ALL sequence, the Upper controller in the PCB map)
Teal	The same, for the ADF controller in the PCB map
Dark	Column headers and the Ethernet switch

Arrows follow the same idea: blue is data going to a board over UDP, green is commands or events coming back, amber is the USB HID path, gray is an ordinary link, and a dashed line is an optional or recovery path.

2.2 Colours in the programs

Where	Colour	Meaning
Health Monitor light	Green	A heartbeat was heard within the last 15 seconds
Health Monitor light	Orange	15 seconds without a heartbeat (connection warning)
Health Monitor light	Red	30 seconds without a heartbeat (connection lost)
Bridge status bar	Red	Disconnected from the simulator
Bridge status bar	Orange	Connected but no data (the simulator is paused or loading)
Bridge status bar	Green	Connected, with the aircraft name
Board status LEDs (pins 14 green, 15 red)	Green and red	Alternate every 200 ms while the sketch runs; red flashes fast while Ethernet starts
SwingTester	Gold	The gauge row being swung, or a warning lamp that is on
GPS bench tester	Pink	A box whose value could not be read (it is not used)
Upper controller emulator	Green	A switch that is on
GPS interface	Green	The position comes from aircraft data over UDP
GPS interface	Orange	No data for 30 seconds: the Brisbane Airport default is being sent

2.3 Cabling colour codes

2.3.1 Round power connectors

Colour	Supply
Red	12 V
Gray	5 V

2.3.2 PCB connectors

Colour	Use	Example
Uncoloured	Digital input	
Blue	Analog	
Red	12 V	Magnetic coils
Green	12 V backlighting	
Yellow	OLED	
Orange	LEDs	

The two 12 V colours are different on purpose: red is 12 V for coils, green is 12 V for backlighting.

2.4 OLED connectors

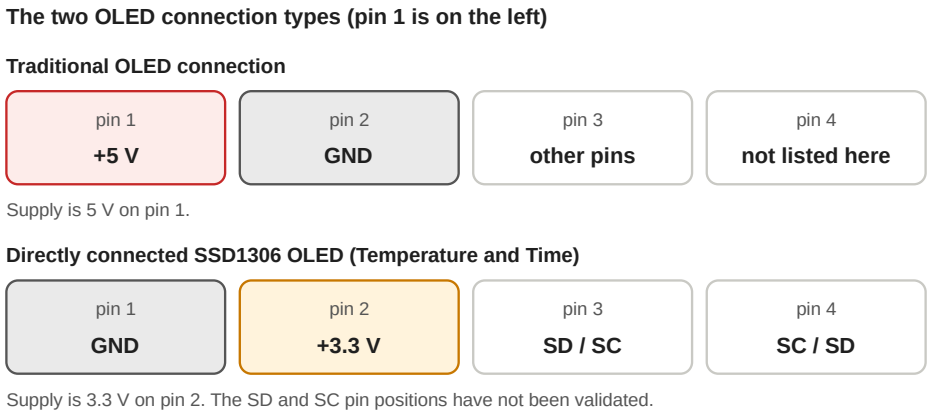


Figure 4 — The two OLED connection types

The OLED connectors are **not the same** on every display:

- **Traditional OLED connection:** +5 V on pin 1 and GND on pin 2.
- **Directly connected SSD1306 OLEDs** (the Temperature and Time displays): GND on pin 1 and +3.3 V on pin 2.

The SD and SC (data and clock) pin positions of the directly connected displays have not been validated. Do not plug one type of cable into the other type of connector: pins 1 and 2 are reversed and the supply voltages differ.

3 Network and communication

3.1 Devices and addresses

All devices sit on one wired `172.16.1.0/24` network with fixed addresses; there is no DHCP. The firmware sends its debug and heartbeat traffic to `172.16.1.10`, so the primary PC must hold that address (or `reflectorIP` must be changed in each sketch and the sketch uploaded again).

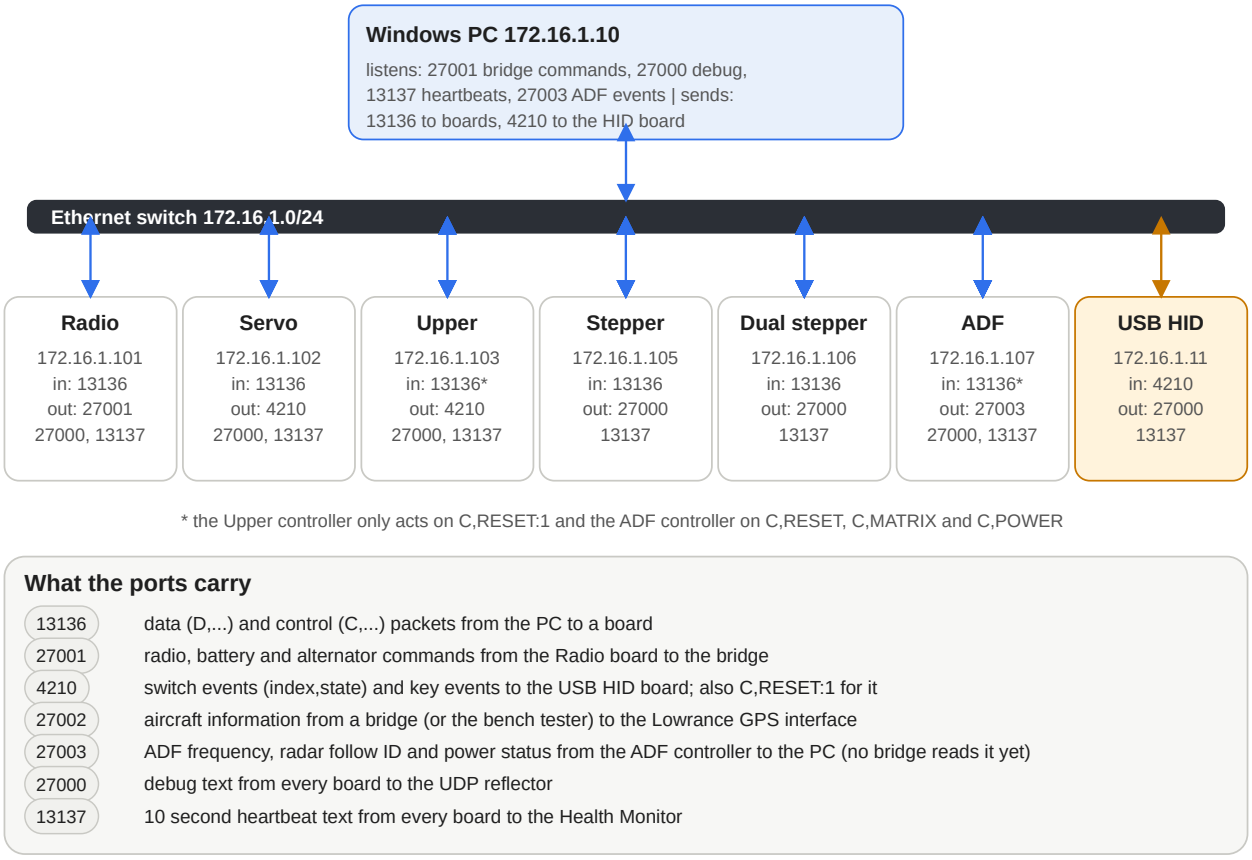


Figure 5 — Who talks to whom

Device	IP	MAC	Heartbeat text
Primary PC (simulator, bridge, listeners)	172.16.1.10		
USB HID board	172.16.1.11	DE:AD:BE:EF:FE:ED	`JOYSTICK`
Radio controller	172.16.1.101	A8:61:0A:9E:83:01	`COMM_NAV`
Servo controller	172.16.1.102	A8:61:0A:9E:83:02	`SERVO`
Upper controller	172.16.1.103	A8:61:0A:9E:83:03	`UPPER_INPUT`
Stepper controller	172.16.1.105	A8:61:0A:67:83:69	`STEPPER`
Dual stepper controller	172.16.1.106	A8:61:0A:67:83:6A	`DUAL_STEPPER`
ADF controller	172.16.1.107	A8:61:0A:67:83:6B	`ADF_PANEL` (the Health Monitor has no light for it)

Every live board has its own IP and MAC address.

3.2 UDP ports

Port	Direction	Purpose
13136	PC to boards	Data (`D,`) and control (`C,`) packets
27001	Radio and ADF controllers to PC bridge	Standby frequencies, radio swaps, and the Auto Pilot,

Port	Direction	Purpose
		Audio and ADF panel controls (`AP_MASTER`, `HEADING_BUG_INC:5`, `GEAR_DOWN`, ...)
27002	Bridge (or bench tester) to the GPS interface	Aircraft position, altitude, airspeed, heading and simulator UTC time, every 500 ms (both on the PC)
27003	ADF controller to the PC	ADF frequency, radar follow ID and power status every 30 seconds (no bridge reads it yet; not 27001, where an unknown string is taken as a COM1 standby frequency)
4210	Servo, Upper, Radio and ADF controllers to USB HID board	Button changes (`index,state`), keys (`KEY,key,state`), reset
13137	Boards to Health Monitor	Heartbeat text every 10 seconds
27000	Boards to debug listener	Debug text, each line prefixed with the board name
7788	Local on the boards	Source port opened at start-up for outgoing packets

3.3 Packet format

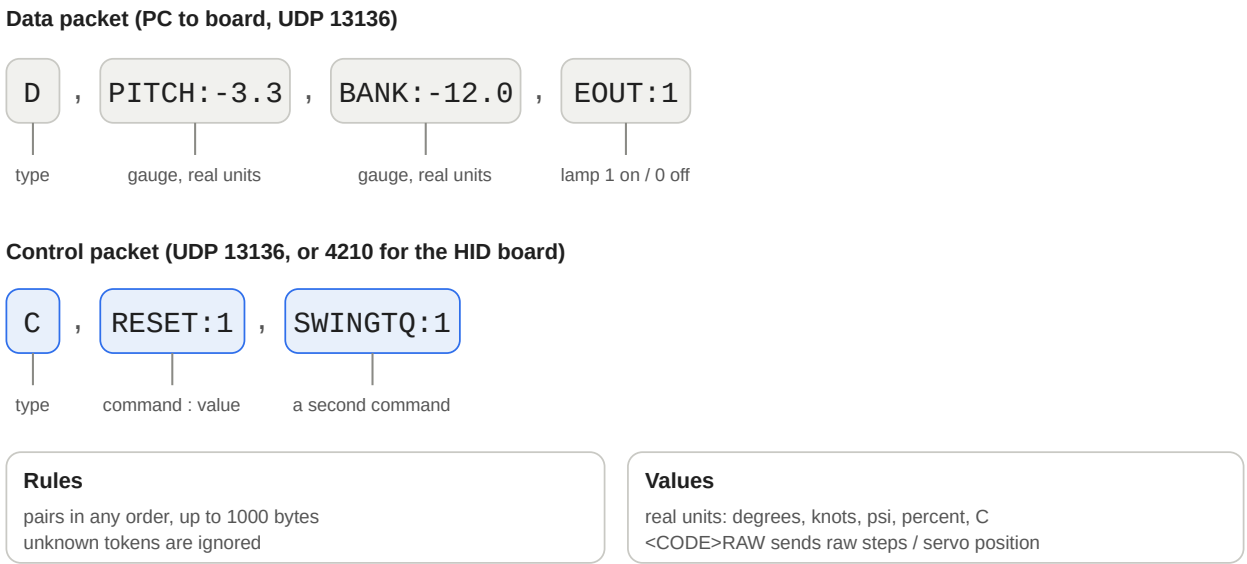


Figure 6 — Data and control packets

A packet is plain ASCII text, up to 1000 bytes.

- `D,CODE:value,CODE:value,...` is a **data** packet. It can carry any number of pairs, in any order. Unknown codes are ignored, so the PC can send only the values that changed.
- `C,COMMAND:value` is a **control** packet (swing tests, lamp test, keep-alive, reset). The value is usually ignored.
- Gauge codes take **real units**: degrees, knots, feet per minute, psi, percent, degrees C, US gallons. The board converts them with its own calibration table.
- A calibrated code usually has a raw twin with `RAW` on the end (for example `IASRAW:300`). It moves the needle to a raw step or servo position and skips the table, which is how new calibration points are measured.
- Lamp codes take `0` (off) or `1` (on).

Example sent to the servo board: `D,PITCH:5,BANK:-10,LOWF:0` sets 5 degrees pitch, minus 10 degrees bank and switches the Low Fuel lamp off.

To test without the simulator, send a packet from any PC on the subnet:

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes('D,PITCH:5,RLOW:1')
$u.Send($b, $b.Length, '172.16.1.102', 13136)
```

3.4 Board life cycle

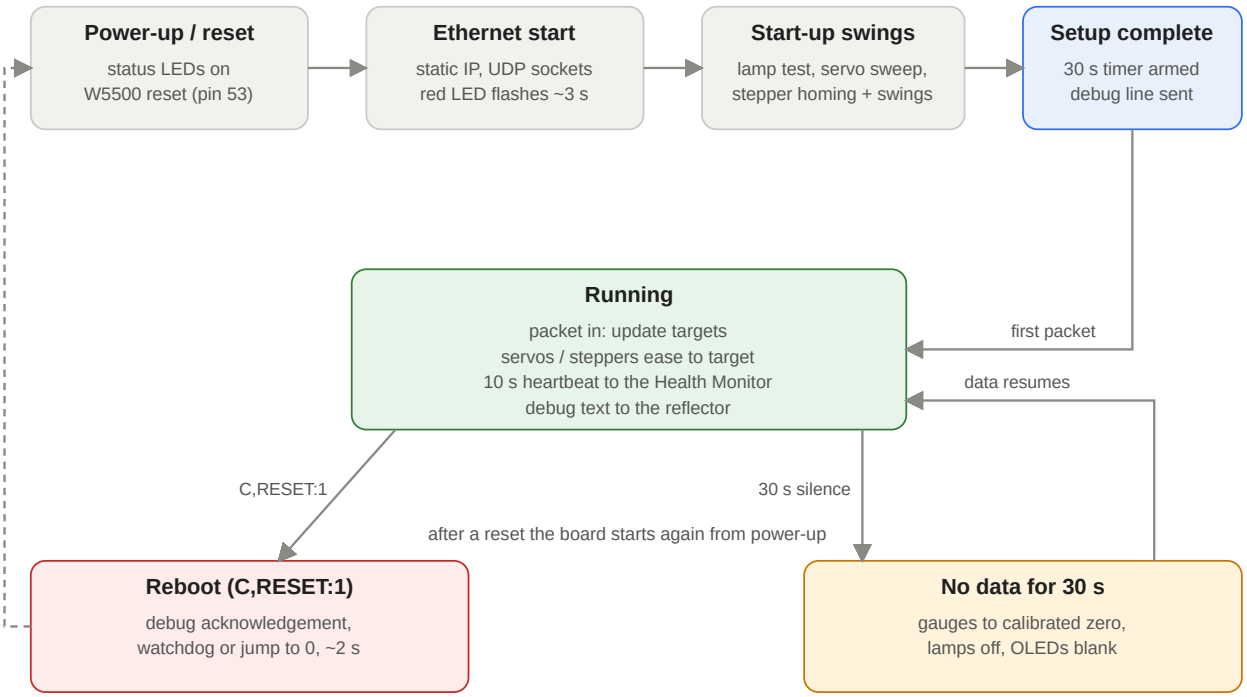


Figure 7 — What every gauge board does from power-up to shutdown

1. **Start-up:** status LEDs flash, the Ethernet module is reset, the board gets its static address and opens its sockets; the red LED flashes for a few seconds while the link settles.
2. **Self-test:** the gauge boards sweep each gauge (the Servo board also lights all lamps) and park at the calibrated zero.
3. **Running:** every 5 ms the board checks UDP 13136, stores each value as a new target and moves the needle towards it with smooth acceleration.
4. **Heartbeat:** every 10 seconds the board sends its heartbeat text to the Health Monitor.
5. **No-data watchdog:** if no packet (data or control) arrives for 30 seconds, every gauge returns to its calibrated zero. The Servo board also switches its lamps off, the Dual Stepper blanks its OLEDs and the Radio blanks its displays. The timer starts at the end of `setup()` so that a long self-test cannot use it up. Normal operation resumes with the next packet.

All sketches call `Ethernet.setRetransmissionTimeout(10)` after `Ethernet.begin()`. The W5500 default of 200 ms times 8 retries can block a UDP send for 1.6 seconds if the destination does not answer ARP (for example the PC is off); with 10 ms the worst case is about 80 ms, so one dead destination cannot stall gauge movement or button scanning.

3.5 Power rules in the bridges

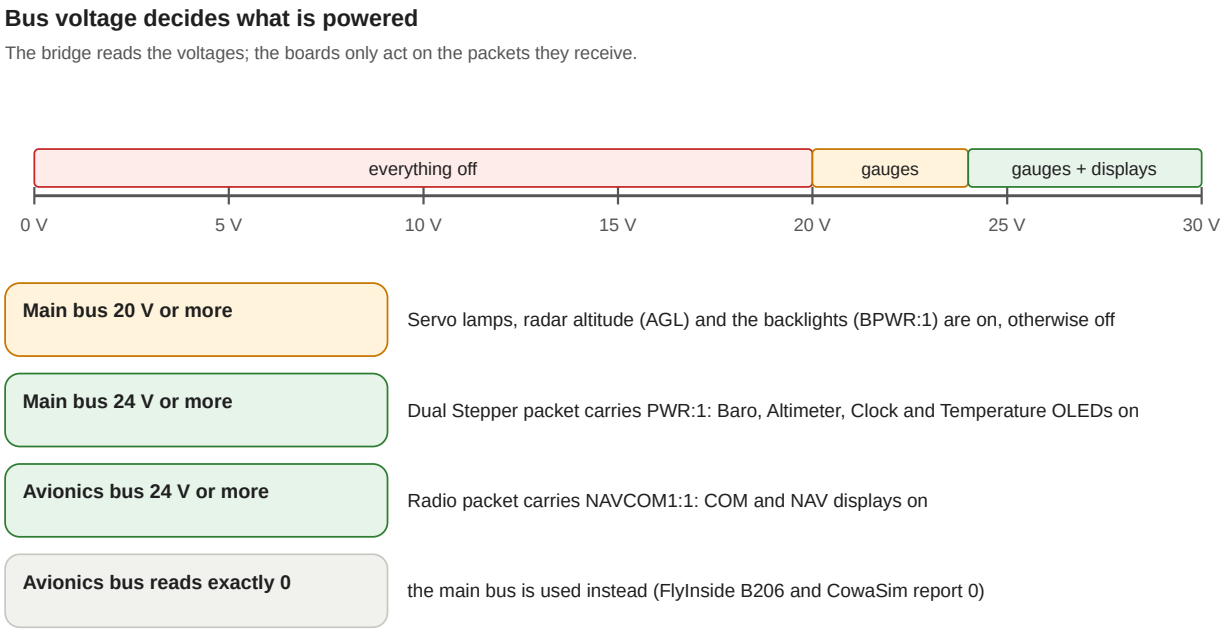


Figure 8 — How bus voltages switch the panel

- **Main bus below 20 V:** the Servo board's lamps are sent as off, the Stepper's radar altitude (`AGL`) is sent as zero and `BPWR:0` switches the Servo, Stepper and Dual Stepper backlights off.
- **Main bus below 24 V:** the bridge sends `PWR:0` to the Dual Stepper, which blanks its four OLEDs; `PWR:1` turns them back on.
- **Avionics bus below 24 V:** the bridge sends `NAVCOM1:0` to the Radio, which blanks its COM and NAV displays together. If the aircraft reports the avionics bus as exactly 0 (the FlyInside B206 does) the main bus is used instead.

There is no hysteresis on these thresholds. The 24 V figure is carried over from the FSUIPC bridge and has not been verified for every MSFS aircraft.

3.6 Backlight

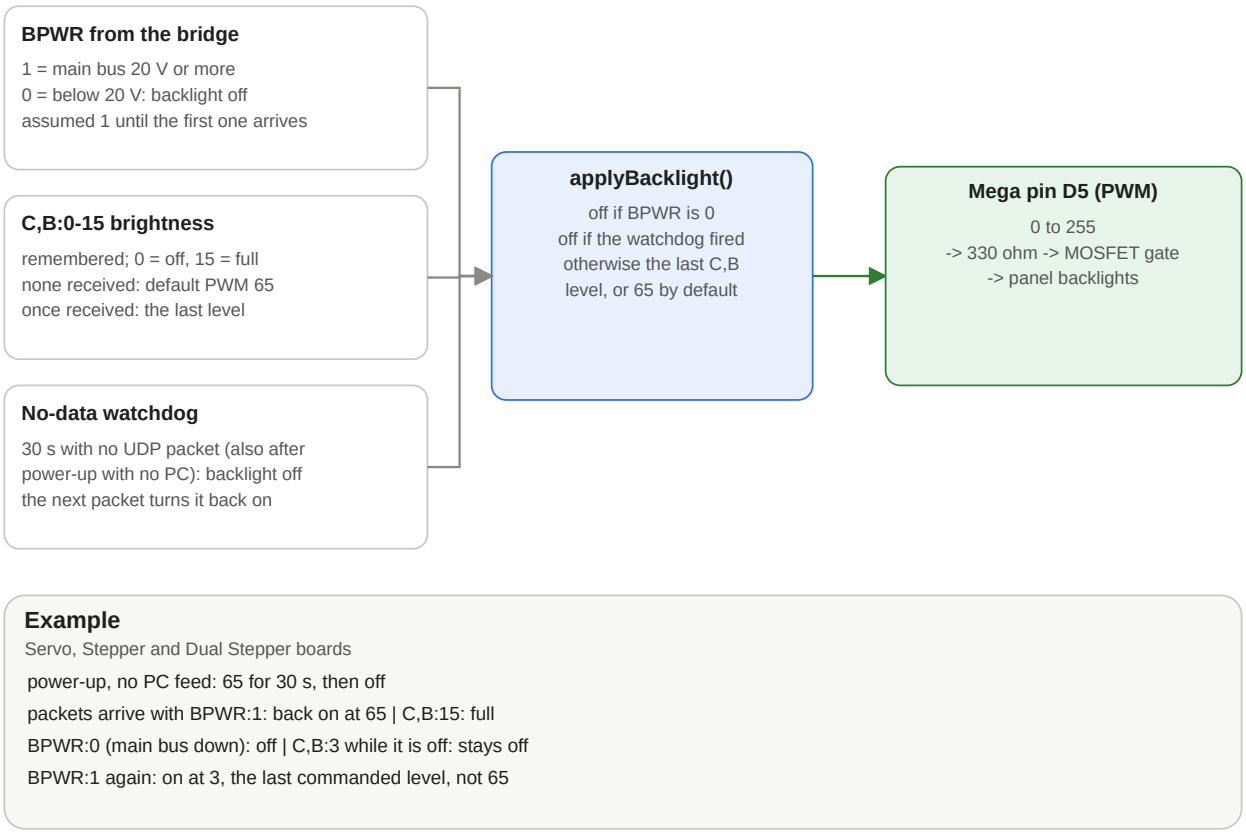


Figure 9 — How the backlight level is decided

The Servo, Stepper and Dual Stepper boards drive their panel backlights from Mega pin D5 (PWM). Three things decide the level:

- **`BPWR`** (from the bridges, first in the Servo and Stepper packets and after `PWR` in the Dual Stepper packet): 1 while the main bus is 20 V or more, 0 below that. At 0 the backlight is off. Until the first `BPWR` arrives the board assumes 1.
- **`C,B:<0-15>`** sets the brightness (0 off, 15 full, mapped to PWM 0 to 255; larger values are clamped). It is remembered. With no brightness command received the default level, PWM 65, is used; once one has been received every return of power restores the **last commanded level**. A command sent while the bus is down is applied when power returns.
- **The no-data watchdog:** after 30 seconds with no UDP packet, including after power-up with no PC feed, the backlight goes off along with the gauges. The next packet turns it back on at the commanded (or default) level, provided the main bus is up.

The SwingTester's brightness slider sends `C,B` to every board except the USB HID board, and its *Backlight power* buttons send `BPWR`. The ADF controller's brightness pot sends `C,B` to the Radio, Servo, Upper, Stepper and Dual Stepper boards whenever its level changes (see the ADF controller chapter). The Radio and Upper controllers accept `C,B` but do nothing with it yet.

3.7 Remote reset

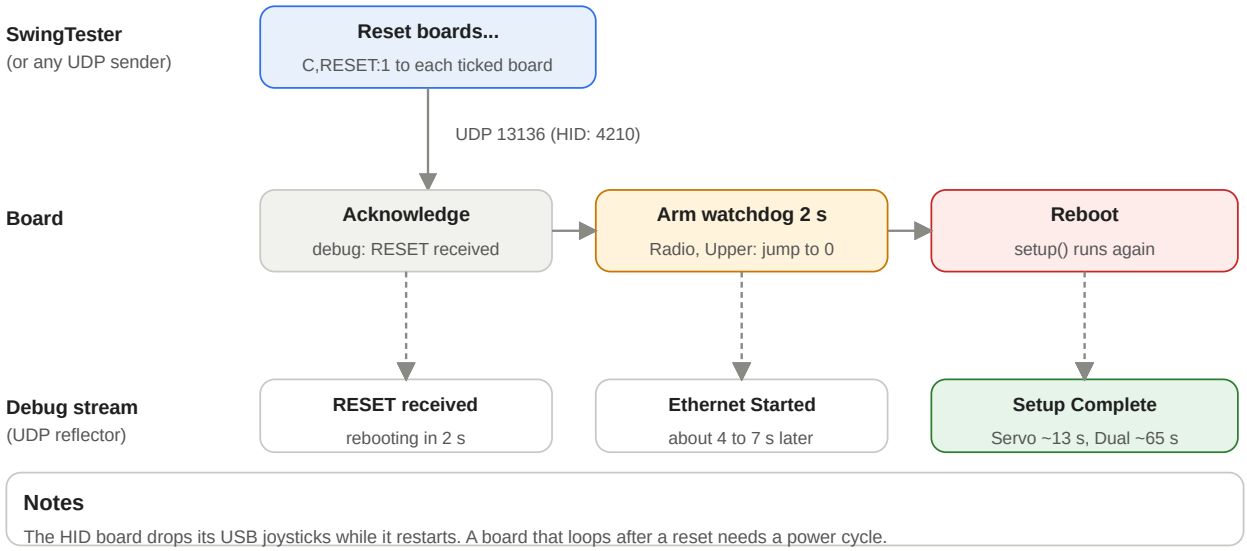


Figure 10 — C,RESET:1 on each board

The control packet `C,RESET:1` reboots a board about two seconds later.

- **Servo, Stepper, Dual Stepper and USB HID boards** arm the AVR watchdog for two seconds and spin until it fires. A short `.init3` function clears the watchdog first thing after every reset. The HID board listens on port 4210 (it also accepts the plain text `RESET`); its USB connection drops while it restarts, so Windows briefly loses the four joysticks and the keyboard.
- **Radio, Upper and ADF controllers** use a software reset (a jump to the reset vector after switching off interrupts and peripherals). A watchdog reset hung the Radio's bootloader, which restarts the watchdog at its shortest timeout and never reaches the sketch until it is reflashed.

If a board ever reboot-loops after a reset packet, power-cycle it and reflash. The SwingTester's *Reset boards...* button sends the command to the boards you tick.

> **Network note:** data that arrives in bursts with gaps of several seconds meant the PC's link to the board switch was dropping in and out. The cause on this cockpit was the 5 V rail feeding the network switch dipping below 4.75 V; the switch then dropped its ports. Check that supply first, then the cable and the switch port.

3.8 Debug and monitoring

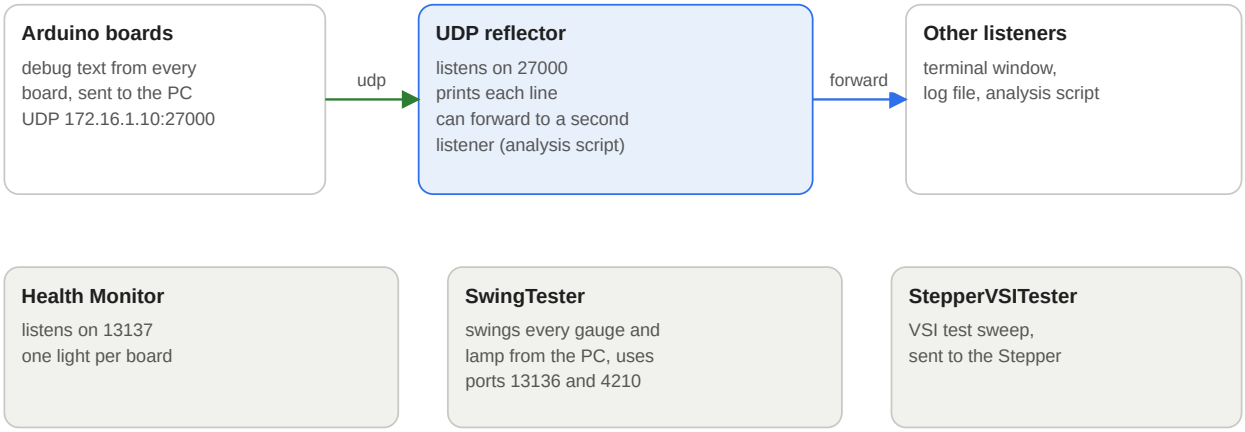


Figure 11 — Debug and test tools

Boards send debug text to `172.16.1.10:27000`. Only one program can own a UDP port, so if the reflector is running do not start a second listener on 27000; use the reflector's forwarding instead. The Health Monitor owns 13137 and each bridge owns 27001.

4 Radio controller (172.16.1.101)

`JET_RANGER_RADIO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. It shows the COM and NAV frequencies on two character OLEDs, reads eight rotary encoders (COM1, COM2, NAV1 and NAV2, each with a major and a minor knob) and a 4 by 5 button matrix, and keeps the radio panel in step with the simulator. It also serves the **Auto Pilot and Audio panels**: it reads their switches and the Auto Pilot's five knobs (sent as joystick buttons and as simulator controls) and drives their 19 LEDs through a MAX7219.

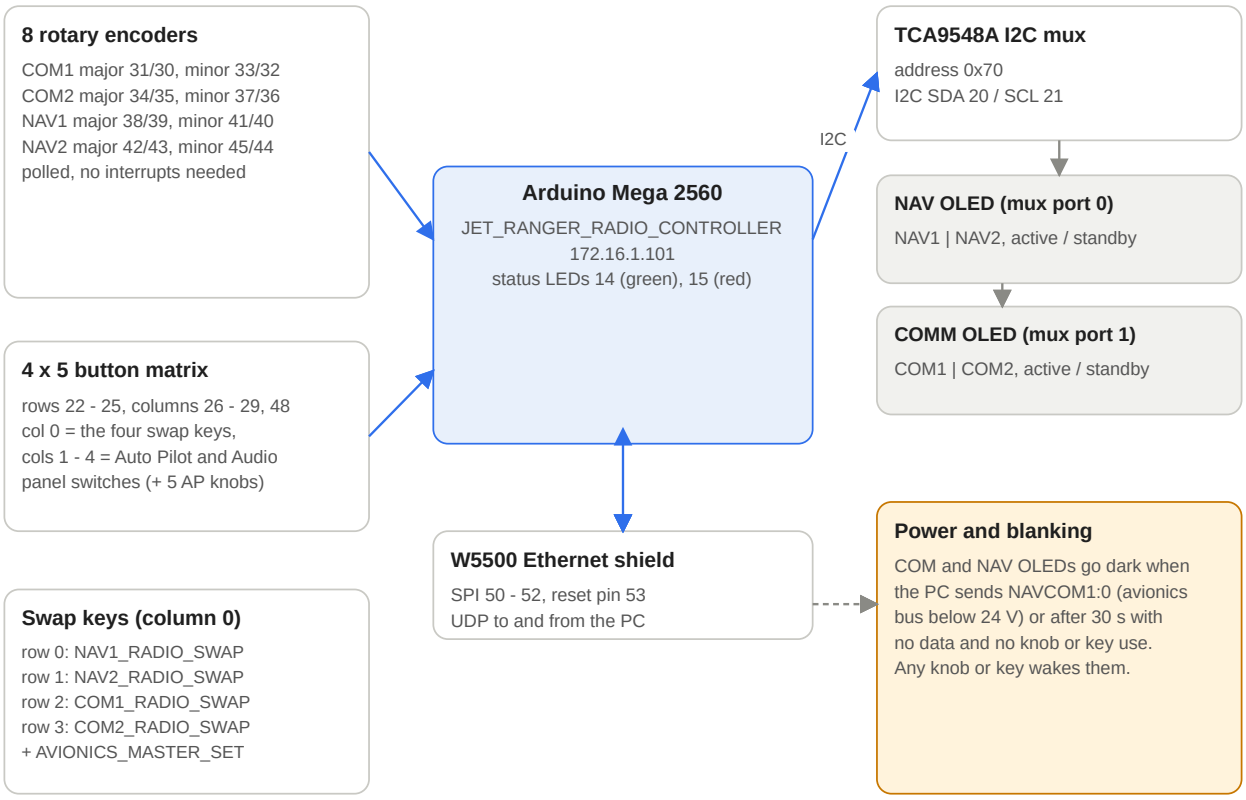


Figure 12 — Radio controller blocks

4.1 Program flow

1. **Setup:** reset the W5500 through pin 53, bring up Ethernet and open the UDP sockets, flash the red LED for 3 seconds while the link settles, send an "Ethernet Started" debug line, start the two OLEDs through the TCA9548A I2C multiplexer, start the eight encoders (with acceleration) and the matrix scanner.
2. **Main loop:**
    1. toggle the green and red status LEDs every 200 ms;
    2. every 10 seconds send the `COMM_NAV` heartbeat;
    3. poll the eight encoders; a change recalculates the standby frequency, sends it to the PC and redraws the OLEDs;
    4. scan the matrix and act on key presses (below);
    5. read any packet on port 13136 and update the active and standby frequencies, the main bus voltage and the power state.

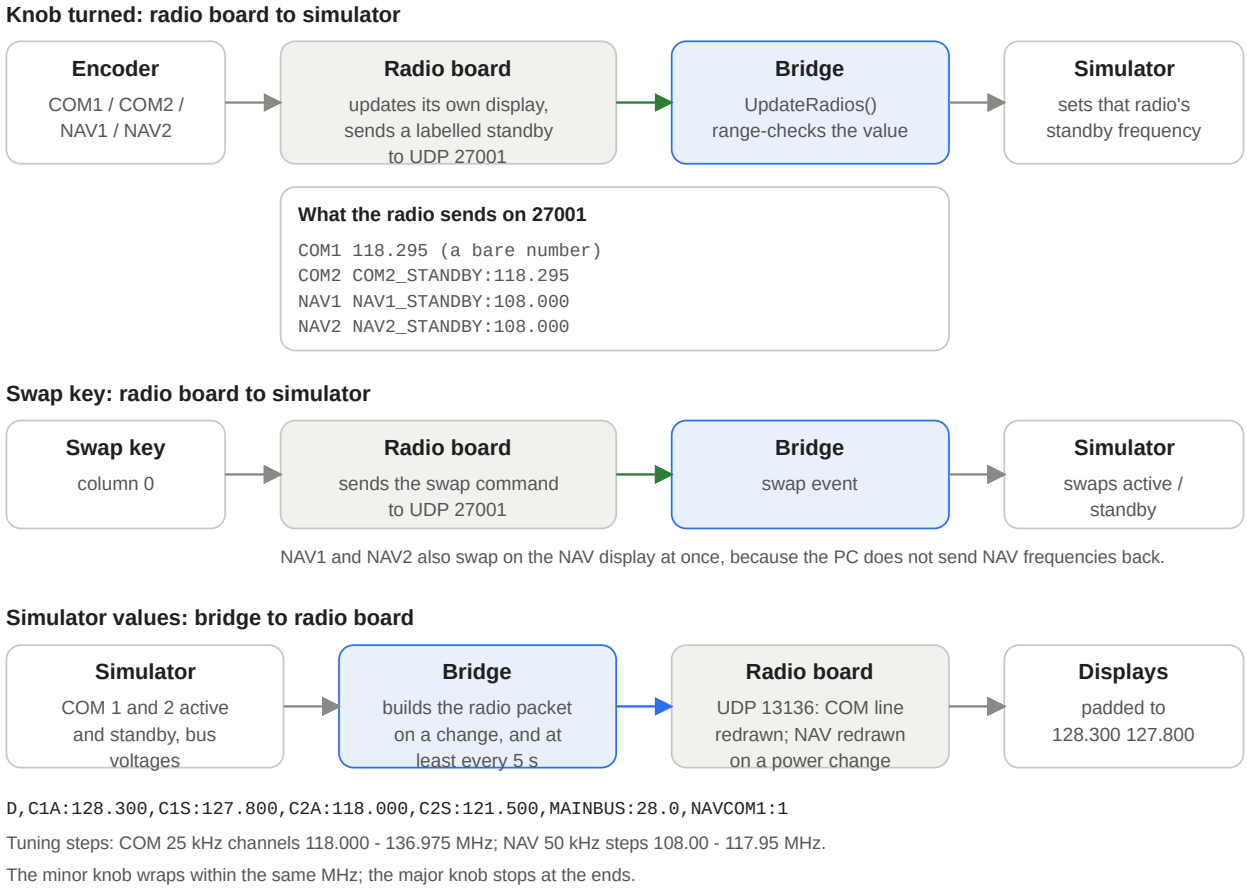


Figure 13 — Where each input and output goes

4.2 Pin usage

Pin	Function
14	Green status LED
15	Red status LED (flashes while the Ethernet link comes up)
53	W5500 manual reset
50 to 52 (and 10)	SPI to the W5500
SDA, SCL	TCA9548A mux (0x70): port 0 NAV OLED, port 1 COM OLED, both at 0x3C
31, 30	COM1 major encoder (A, B)
33, 32	COM1 minor encoder
34, 35	COM2 major encoder
37, 36	COM2 minor encoder
38, 39	NAV1 major encoder
41, 40	NAV1 minor encoder
42, 43	NAV2 major encoder
45, 44	NAV2 minor encoder
22 to 25	Matrix rows (`ROW0` to `ROW3`)
26, 27, 28, 29, 48	Matrix columns (`COL0` COM and NAV panel, `COL1` and `COL2` Auto Pilot panel, `COL3` and `COL4` Audio panel)
11, 12	Auto Pilot CRS knob (`CRS_A`, `CRS_B`)
A4, A5	Auto Pilot ALT knob (`ALT_A`, `ALT_B`)
46, 47	Auto Pilot VS knob (`VS_A`, `VS_B`)
7, 6	Auto Pilot IAS knob (`IAS_A`, `IAS_B`)
8, 9	Auto Pilot HDG knob (`HDG_A`, `HDG_B`)

Pin	Function
A15, A14, A13	MAX7219 `DIN`, `CLK` and `LOAD` (the panel LEDs)

4.3 Data from the PC (UDP 13136)

Token	Meaning	Example
`C1A`, `C1S`	COM1 active and standby frequency	`C1A:119.500`
`C2A`, `C2S`	COM2 active and standby frequency	`C2S:121.900`
`MAINBUS`	Main bus voltage	`MAINBUS:27.8`
`NAVCOM1`	Radio power: 1 = on, 0 = COM and NAV displays blank and the panel LEDs off	`NAVCOM1:1`
`LED_xxx`	One of the 19 Auto Pilot and Audio panel LEDs (see below): 1 = on, 0 = off	`LED_HDG:1`
`C,B:n`	Brightness 0 to 15 (control packet): the MAX7219 intensity of the panel LEDs, 0 = LEDs off	`C,B:5`
`C,LEDTEST:n`	Control packet: 1 lights all 19 panel LEDs, 0 returns them to the simulator's state	`C,LEDTEST:1`

The bridges send COM frequencies with two decimals; the radio pads every frequency to `xxx.xxx` (7 characters) so that the 16-column line (7 + 2 + 7) never slides COM2 or NAV2 behind the faceplate. The PC does not send NAV frequencies back.

4.4 Commands to the PC (UDP 27001)

Turning a knob changes the standby frequency on the OLED and sends it to the PC. COM1 is sent as the bare number (for example `118.295`); the others carry the radio's name: `COM2_STANDBY:118.295`, `NAV1_STANDBY:108.000`, `NAV2_STANDBY:108.000`. Nothing is sent during the first 1.5 seconds after power-up, so the knobs' start positions do not change the simulator.

The four frequency-swap switches are on the Jet Ranger COM and NAV PCB in a diode matrix that shares column 0 (D26); the rows are D22 to D25.

Switch	Matrix (row, col)	Action
NAV1 FREQ (SW6)	0, 0	Swaps NAV1 active and standby on the NAV OLED at once (the PC sends no NAV frequencies back) and sends `NAV1_RADIO_SWAP`
NAV2 FREQ (SW7)	1, 0	The same for NAV2, sends `NAV2_RADIO_SWAP`
COMM1 (SW4)	2, 0	Sends `COM1_RADIO_SWAP`; the display changes when the simulator returns the swapped frequencies
COMM2 (SW5)	3, 0	Sends `COM2_RADIO_SWAP`, then `AVIONICS_MASTER_SET`

Keys in columns 1 to 4 are the Auto Pilot and Audio panel switches (next section); any other key is only logged to the debug listener.

4.5 Auto Pilot and Audio panel switches and knobs

The Jet Ranger Auto Pilot Panel and Audio Panel PCBs plug into the Radio Auto Pilot controller. Each has eight switches in a diode matrix (row to the switch, diode to the column), and the Auto Pilot panel also has five rotary encoders wired straight to the Mega. Every switch and knob is sent **two ways**:

1. as a **joystick button** on the USB HID board (UDP `172.16.1.11:4210`, `index,state`), in the buttons nothing else uses: **Joystick 3** (indexes 64 to 95) for the Auto Pilot panel and **Joystick 4** (96 to 127) for the Audio and ADF panels. A switch holds its button down while it is pressed. A knob click is sent as a 40 ms press (the `HID_PULSE_MS` release goes out from the main loop), clockwise on one button and anticlockwise on the next;
2. as a **command to the PC bridge** (UDP 27001), the name of the simulator control, which the FSUIPC and MSFS bridges recognise (see their chapters). A switch sends its command when pressed; a knob sends one command for each click, with a `:N` repeat count when it is turned quickly (`HEADING_BUG_INC:5`).

Nothing is sent during the first 1.5 seconds after power-up (the quiet period). Windows numbers joystick buttons from 1, so index 64 is Joystick 3 button 1. These indexes correspond to Upper controller matrix positions that are wired but have no switch on them (the Upper's assigned switches are all below index 40).

Panel	Control	Matrix or pin	Joystick button (index)	Command to the bridge (UDP 27001)
Auto Pilot	AP ENGAGE	row 0, col 1	Joystick 3, button 1 (64)	`AP_MASTER`
Auto Pilot	HDG	row 1, col 1	Joystick 3, button 2 (65)	`AP_HDG_HOLD`
Auto Pilot	CRS	row 2, col 1	Joystick 3, button 3 (66)	`AP_NAV1_HOLD`
Auto Pilot	ALT	row 3, col 1	Joystick 3, button 4 (67)	`AP_ALT_HOLD`
Auto Pilot	APPR	row 0, col 2	Joystick 3, button 5 (68)	`AP_APR_HOLD`
Auto Pilot	FD	row 1, col 2	Joystick 3, button 6 (69)	`TOGGLE_FLIGHT_DIRECTOR`
Auto Pilot	BC	row 2, col 2	Joystick 3, button 7 (70)	`AP_BC_HOLD`
Auto Pilot	IAS	row 3, col 2	Joystick 3, button 8 (71)	`AP_PANEL_SPEED_HOLD`
Auto Pilot	CRS knob clockwise, anticlockwise	D11, D12	Joystick 3, buttons 9, 10 (72, 73)	`VOR1_OBI_INC`, `VOR1_OBI_DEC`
Auto Pilot	ALT knob	A4, A5	Joystick 3, buttons 11, 12 (74, 75)	`AP_ALT_VAR_INC`, `AP_ALT_VAR_DEC`
Auto Pilot	VS knob	D46, D47	Joystick 3, buttons 13, 14 (76, 77)	`AP_VS_VAR_INC`, `AP_VS_VAR_DEC`
Auto Pilot	IAS knob	D7, D6	Joystick 3, buttons 15, 16 (78, 79)	`AP_SPD_VAR_INC`, `AP_SPD_VAR_DEC`
Auto Pilot	HDG knob	D8, D9	Joystick 3, buttons 17, 18 (80, 81)	`HEADING_BUG_INC`, `HEADING_BUG_DEC`
Audio	COM1	row 0, col 3	Joystick 4, button 1 (96)	`COM1_TRANSMIT_SELECT`
Audio	BOTH	row 1, col 3	Joystick 4, button 2 (97)	`COM_RECEIVE_ALL_TOGGLE`
Audio	NAV2	row 2, col 3	Joystick 4, button 3 (98)	`RADIO_VOR2_IDENT_TOGGLE`
Audio	DME	row 3, col 3	Joystick 4, button 4 (99)	`RADIO_DME1_IDENT_TOGGLE`
Audio	COM2	row 0, col 4	Joystick 4, button 5 (100)	`COM2_TRANSMIT_SELECT`
Audio	NAV1	row 1, col 4	Joystick 4, button 6 (101)	`RADIO_VOR1_IDENT_TOGGLE`
Audio	MKR	row 2, col 4	Joystick 4, button 7 (102)	`MARKER_SOUND_TOGGLE`
Audio	ADF	row 3, col 4	Joystick 4, button 8 (103)	`RADIO_ADF_IDENT_TOGGLE`
ADF	Landing gear switch	ADF controller D29	Joystick 4, button 9 (104)	`GEAR_DOWN` when closed, `GEAR_UP` when open
ADF	Flaps switch	ADF controller D30	Joystick 4, button 10 (105)	`FLAPS_DOWN` when closed, `FLAPS_UP` when open
ADF	Speed brakes switch	ADF controller D31	Joystick 4, button 11 (106)	`SPOILERS_ON` when closed, `SPOILERS_OFF` when open
ADF	Brakes button	ADF controller D32	Joystick 4, button 12 (107)	`PARKING_BRAKES` when pressed
ADF	ADF major knob clockwise, anticlockwise	ADF controller D6, D7	Joystick 4, buttons 13, 14 (108, 109)	none (the knob tunes the ADF frequency, `ADF_FREQ`)
ADF	ADF minor knob clockwise, anticlockwise	ADF controller D8, D9	Joystick 4, buttons 15, 16 (110, 111)	none

The Radio controller sends the first 18 Auto Pilot rows and the 8 Audio rows; the ADF rows come from the ADF controller (see its chapter). If a knob turns the wrong way, swap its A and B pins in the `RotaryEncoder` lines for the five Auto Pilot knobs. Which row and column each switch is on comes from the panel schematics: the matrix columns are the Radio Auto Pilot PCB nets `COL1` to `COL4`, and `COL0` is the COM and NAV panel's four swap switches. The debounce is 50 ms and the knobs use the same polling encoder library, with acceleration, as the COM and NAV knobs.

4.6 Auto Pilot and Audio panel LEDs (MAX7219)

One MAX7219 on the Radio Auto Pilot PCB (U2, `DOUT` unconnected, ISET resistor 10 kohm) drives 19 LEDs: DIN on A15, CLK on A14, LOAD on A13 (LedControl library, scan limit 3, intensity 8 until a `C,B` arrives). Its segment pins A to G and DP are the PCB nets `ANNODE_00` to `ANNODE_07` and its digit pins 0 to 7 are `CATHODE_00` to `CATHODE_07`; each panel LED sits between one segment (anode) and one digit (cathode). The Auto Pilot panel uses digits 0 and 1 (connector `AUTOPILOT`) and the Audio panel digits 2 and 3 (connector `AUDIO`). LedControl's `setLed(0, digit, column)` uses column 0 for DP and 1 to 7 for A to G; in the chip's own digit registers (address = digit + 1) bit 7 is DP, bit 6 is A and bit 0 is G.

LED	Token	MAX7219 digit	Segment		LED	Token	MAX7219 digit	Segment
CRS	`LED_CRS`	0	A		COM1	`LED_COM1`	2	A
HDG	`LED_HDG`	0	B		BOTH	`LED_BOTH`	2	B
IAS	`LED_IAS`	0	C		NAV2	`LED_NAV2`	2	C
ALT	`LED_ALT`	0	D		DME	`LED_DME`	2	D
APPR	`LED_APPR`	1	A		OUTER	`LED_OUTER`	2	E
FD	`LED_FD`	1	B		INNER	`LED_INNER`	2	F
BC	`LED_BC`	1	C		COM2	`LED_COM2`	3	A
AP ENGAGE	`LED_APENG`	1	D		NAV1	`LED_NAV1`	3	B
					MKR	`LED_MKR`	3	C
					ADF	`LED_ADF`	3	D
					MIDDLE	`LED_MID`	3	E

The first two columns are the Auto Pilot panel and the others the Audio panel. The bridges send each LED as a `LED_xxx:0/1` token in the radio data packet. `HandleOutputValuePair()` handles these before its per-pair debug line (19 extra debug packets per radio packet would flood the debug stream), `applyPanelLeds()` writes only the LEDs that changed, and the LEDs follow the radio power: they are lit only while `NAVCOM1` is 1 and the no-data timeout has not blanked the displays. `C,LEDTEST:1` lights all 19 for checking the wiring, and `C,B:0-15` (for example from the ADF controller's pot or the SwingTester slider) sets the intensity: 0 shuts the MAX7219 down, 1 to 15 set the intensity. The SwingTester has a toggle for each LED, an All ON / OFF, a chase and the board test.

4.7 Tuning steps

The knobs follow standard channel spacing. COM runs 118.000 to 136.975 MHz in 25 kHz channels (major knob 118 to 136; minor knob 000, 025, 050, 075). NAV runs 108.00 to 117.95 MHz in 50 kHz steps (major 108 to 117). Each minor knob wraps round at its ends. The constants `COM_MINOR_STEP_KHZ` and `NAV_MINOR_STEP_KHZ` set the steps, and the wrap-around limits follow from them. 8.33 kHz channels are not offered. FSUIPC holds COM frequencies to 10 kHz, so in the simulator a channel such as 118.025 may appear as 118.02 or 118.03.

4.8 Power and blanking

- **Avionics power:** when the avionics bus is below 24 V the bridge sends `NAVCOM1:0` and the COM and NAV displays blank together; they light together again when the bus rises.
- **No data for 30 seconds:** if no packet arrives and no knob or key is used, both OLEDs blank once. A packet, a knob movement or a key press restores them, and they blank again after another 30 seconds of silence.
- **Quiet period:** for 1.5 seconds (`matrixQuietMs`) after the matrix scanner starts it records state but raises no events, so a floating input cannot send false button presses at boot.

4.9 Diagnostics

- The control packet `C,MATRIX:1` prints the raw matrix (H or L for each of the five columns, with all rows high and then each row driven low) to the debug stream, bypassing debounce and the quiet period. A

column that reads L with nothing pressed is a hardware fault on that Mega pin or board. A faulty or badly seated Mega did exactly this on the bench: all four swap switches share column 0, so none could register, and the board also dropped off USB and the network after flashing.

- Remote reset is a software reset (see the network chapter). Bench-tested: "Ethernet Started" about 4 seconds after the command and "Setup Complete" about 5 seconds.

4.10 PCB features the sketch does not use yet

The Jet Ranger Radio Auto Pilot PCB (v1.3) carries more than the sketch handles. What is still unused:

Mega pins	PCB net	Status in the sketch
D5	`BACKLIGHT_PWM`: 330 ohm to an IRF540N MOSFET that switches the AUDIO, AUTOPILOT and COMM_NAV panel backlights	Not driven. `C,B:n` sets the MAX7219 LED intensity, not this backlight
A0 to A3	`COMM1_VOL`, `COMM2_VOL`, `NAV1_VOL`, `NAV2_VOL` (volume pots)	Not read
D2 to D4	`I2C_SEL_A2`, `A1`, `A0` (mux address)	Not driven

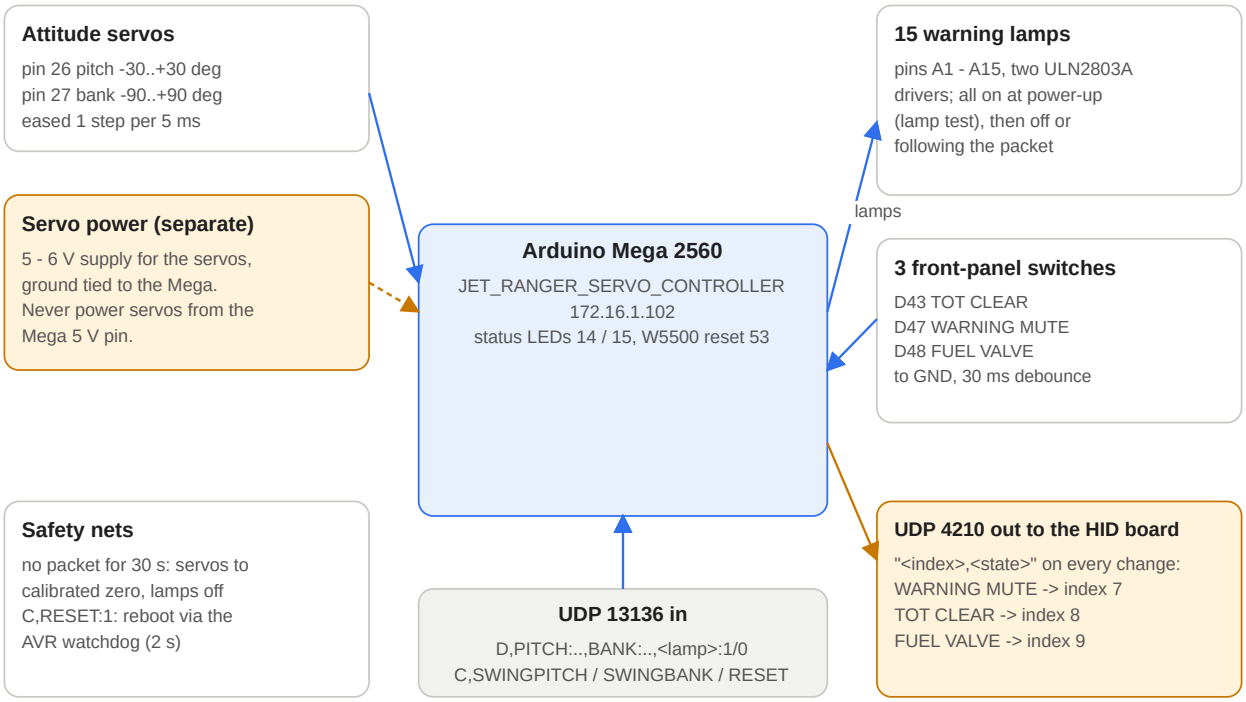
On the same PCB the COM1 major, COM1 minor, COM2 minor and NAV1 minor encoders have their A and B pins the other way round from the sketch (the sketch uses 31, 30 and 33, 32 for COM1, 37, 36 for COM2 minor and 41, 40 for NAV1 minor, where the PCB labels A as 30, 32, 36 and 40). It only matters if one of those knobs tunes backwards.

4.11 Known issues

- Only COM1 and COM2 frequencies arrive from the PC; there are no NAV frequency tokens yet.
- The Radio controller emulator sends a bare frequency for every radio (see the emulator chapter).
- The mux address pins (D2 to D4) are not driven by the sketch; the mux module's own pull-downs set address 0x70.

5 Servo controller (172.16.1.102)

`JET_RANGER_SERVO_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger Front Panel Servo PCB. It drives the attitude indicator's two servos (pitch and bank), switches the 15 caution and warning lamps and reads three front-panel switches. The other gauges that were once servo-driven (airspeed, vertical speed, torque, temperatures, pressures, fuel and RPM) now run on the two stepper boards.



Joystick 1 buttons 8, 9 and 10 in Windows (the firmware counts from 0). FSUIPC needs them bound to DS206 events.

Calibration: PITCH_DEG_TABLE -30 deg 179, 0 deg 113, +30 deg 70; BANK_DEG_TABLE -90 deg 5, 0 deg 93, +90 deg 179 (servo positions).

Figure 14 — Servo controller blocks

5.1 Program flow

1. **Setup:** set the status LED pins; light all 15 lamps as a lamp test; reset the W5500, start Ethernet and open the sockets (red LED flashes for 3 seconds); sweep pitch and then bank from minimum to maximum and back (about 2 seconds each) and park both at their calibrated zero; switch all lamps off; send the starting state of the three switches to the HID board; log "Setup Complete" and arm the 30 second watchdog.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `SERV0` heartbeat every 10 seconds;
    2. every 5 ms check UDP 13136 and store each value as a new **target** (servo targets, lamp states);
    3. every 5 ms nudge each servo one step towards its target, so the attitude indicator moves smoothly; a servo that has not moved for 1 second is detached to stop buzz and re-attached on the next movement;
    4. service the three debounced switch inputs.

3. **Watchdog:** after 30 seconds with no packet both servos are eased back to their calibrated zero and all lamps go out. The remembered lamp states are reset as well, so a stale `1` cannot keep a lamp dark when the simulator next sends `1`.

5.2 Servos and calibration

Instrument	Mega pin	Token	Range
Attitude pitch	26	`PITCH`	-30 to +30 degrees
Attitude bank (roll)	27	`BANK`	-90 to +90 degrees

Both tokens take attitude in degrees with one decimal place allowed (for example `D,PITCH:4.5,BANK:-12`). The board converts degrees to a servo position (0 to 180) with two tables, interpolating in a straight line between points and holding the end value beyond them.

Instrument	Attitude	Servo position
Pitch (`PITCH_DEG_TABLE`)	-30 degrees	179
Pitch	0 (level)	113
Pitch	+30 degrees	70
Bank (`BANK_DEG_TABLE`)	-90 degrees	5
Bank	0 (level)	93
Bank	+90 degrees	179

`PITCHRAW` and `BANKRAW` take a raw servo position and skip the tables. The set-up sweep uses `aServMinPosition[]` and `aServMaxPosition[]` (bank 5 to 179, pitch 166 to 70) and parks at `aServZeroPosition[]` (bank 93, pitch 113). The PCB still has 16 servo headers from the earlier design; only pins 26 and 27 are used.

Calibrating a servo:

1. Send raw positions (`D,PITCHRAW:113`) until the instrument shows a known attitude.
2. Note the position for at least both ends and level.
3. Enter them in the table and set `aServZeroPosition[]` to the level position.
4. Upload, then check with real values (`D,PITCH:0`, `D,BANK:30`).

Do not drive a servo hard against its mechanical stop: a stalled servo draws high current and gets hot. Never power servos from the Mega's 5 V pin; use a separate 5 to 6 V supply with a common ground.

5.3 Warning lamps

The 15 warning lamps on the Servo board

Green: the bridge drives it from the simulator. Grey: needs aircraft-specific variables, so it stays off.

ENGINE OUT pin A1 token EOUT engine speed < 60 %	**ROTOR RPM LOW** pin A2 token RLOW rotor <= 95 %	**XMSN OIL PRESS** pin A3 token TOPW pressure < 40 psi
XMSN OIL TEMP pin A4 token TOTW always off	**BATTERY HOT** pin A5 token BHOT always off	**BATTERY TEMP** pin A6 token BTMP always off
ENGINE CHIP pin A7 token EC always off	**TAIL ROTOR CHIP** pin A8 token TRC always off	**XMSN CHIP** pin A9 token TC always off
BAGGAGE DOOR pin A10 token BD always off	**AFT FUEL FILTER** pin A11 token FFLTR always off	**FUEL PUMP** pin A12 token FPMP always off
GENERATOR FAIL pin A13 token GENF engine speed < 10 %	**LOW FUEL** pin A14 token LOWF fuel <= 17 gal	**SC FAIL** pin A15 token SCF always off

All lamps go out when the main bus is below 20 V. Sent as <token>:1 / <token>:0 in every Servo packet.
The lamp test at power-up lights all 15; a no-data timeout turns them all off.

Figure 15 — The 15 lamps and their Mega pins

Token	Lamp	Pin
EOUT	Engine out	A1
RLOW	Rotor RPM low	A2
TOPW	Transmission oil pressure	A3
TOTW	Transmission oil temperature	A4
BHOT	Battery hot	A5
BTMP	Battery temperature	A6
EC	Engine chip	A7
TRC	Tail rotor chip	A8
TC	Transmission chip	A9
BD	Baggage door	A10
FFLTR	Aft fuel filter	A11
FPMP	Fuel pump	A12
GENF	Generator fail	A13
LOWF	Low fuel	A14
SCF	SC fail (standby compass)	A15

A lamp pin is HIGH when lit (through a driver for lamps above 20 mA). The power-up lamp test lights every lamp, so it cannot show a mis-wired pin: send one lamp at a time (for example `D,RLOW:1`) to check each one.

5.4 Debounced switch inputs

Three front-panel switches are wired between a Mega pin and GND on the Front Panel Servo PCB and read with the internal pull-up (closed = pin LOW). A new reading must stay unchanged for 30 ms (`InputDebounceMs`) before it counts.

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D43	TOT CLEAR (SW1)	/SW_TOT_CLEAR	8	9

Pin	Switch	PCB net	HID index	Joystick 1 button in Windows
D47	WARNING MUTE (SW3)	`/SW_WARNING_MUTE`	7	8
D48	FUEL VALVE (SW2)	`/SW_FUEL_VALVE`	9	10

Every accepted change is logged to the debug stream (for example `Jet Ranger Servo: WARNING MUTE CLOSED -> HID button 7`) and sent to the USB HID board at `172.16.1.11:4210` as `index,state` with 1 = closed. The HID board counts buttons from 0 and Windows from 1. The starting state of each switch is sent at the end of `setup()`; there is no periodic resend, so a HID board that boots later stays out of step until the switch next changes. Nothing is sent to the PC itself.

5.5 On-demand commands

- `C,SWINGPITCH:1` and `C,SWINGBANK:1` run one sweep (the value is ignored); the board pauses for about 2 seconds.
- `C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.9 seconds later, "Setup Complete" about 12.8 seconds after the command).

5.6 Backlight

Pin 5 (the PCB's `BACKLIGHT_PWM`) drives the panel backlights through the board's IRF540N MOSFET. The level follows `BPWR`, the last `C,B:0-15` brightness command and the 30 second no-data watchdog (see the network chapter): with no command the default PWM 65 is used, and once one has been received the last commanded level returns whenever power returns. After 30 seconds with no packet, including after power-up with no PC feed, the backlight goes off with the lamps and returns with the next packet. `setup()` sets pin 5 as an output and applies the default level; each change is logged (`Brightness 15 -> PWM 255`, `Backlight power off`, `Backlight restored - UDP data resumed`). Compiled at 24,468 bytes of flash (9%) and 30% RAM; flashed to the Servo board on COM16.

5.7 Pin usage

Pin	Function
14	Green status LED
15	Red status LED
26, 27	Pitch and bank servos
5	Backlight PWM (`BACK_LIGHTS`, to the MOSFET gate through 330 ohm)
43, 47, 48	TOT clear, warning mute and fuel valve switches
53	W5500 reset
A1 to A15	Warning lamps
50 to 52	SPI to the Ethernet module

5.8 Known issues

- `powerAvailable` is set from `NAVCOM1` but nothing uses it, so the attitude indicator and lamps stay live when the simulator's power is off (the bridge itself sends all lamps off and `BPWR:0` below 20 V).
- Several lamps log "Transmission Temperature changed" to the debug stream whichever lamp changed.
- The `#define` comments for pins 26 and 27 still describe the old servo layout.
- The PCB labels 10 of the 15 lamp outputs on different Mega pins from the sketch (A1/A2, A5/A6, A7 to A10, A11/A12). Test each lamp on its own and correct whichever side is wrong.

6 Upper controller (172.16.1.103)

`JET_RANGER_UPPER_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module. It scans the overhead switch matrix (up to 256 positions, 176 used) and a small numeric keypad, and sends every change over UDP to the USB HID board, which presents them to Windows as joystick buttons and keystrokes. The sketch also contains DCS-BIOS code for DCS World over the USB serial link, but `CreateDcsBiosMessage()` is still a stub, so only the joystick path works.

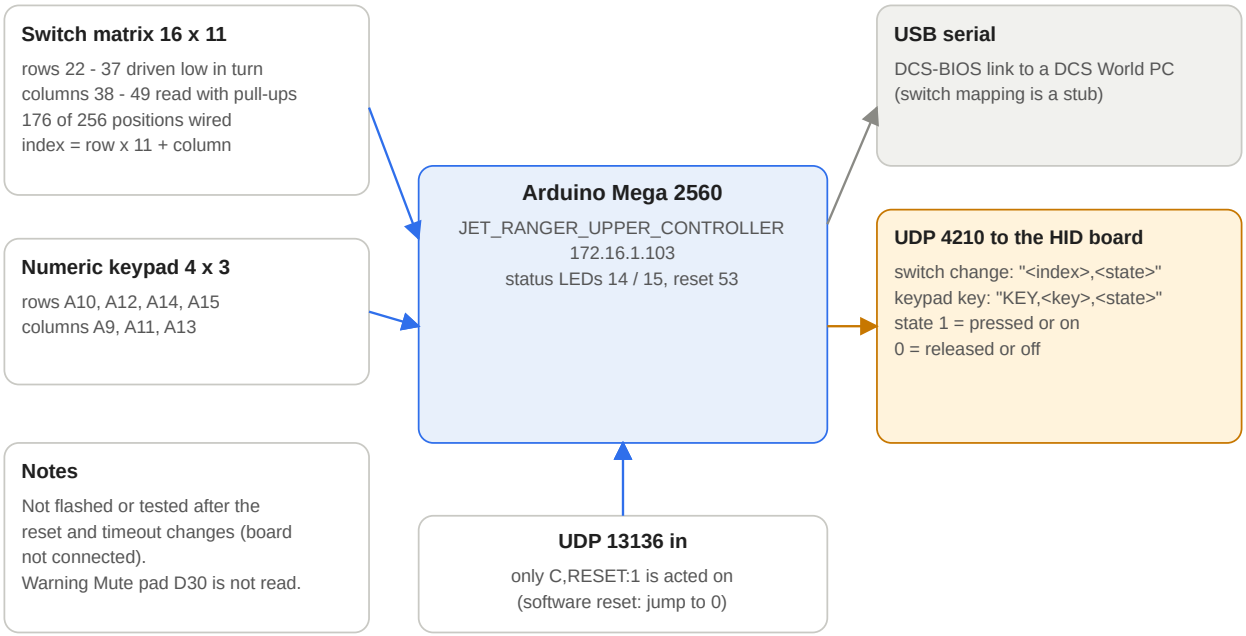


Figure 16 — Upper controller blocks

6.1 Program flow

1. **Setup:** set the status LEDs, reset the W5500, start Ethernet and open the sockets (red LED flashes for 3 seconds), configure pins 22 to 37 as matrix row outputs and 38 to 49 as column inputs with pull-ups, clear the button and debounce arrays and start the keypad.
2. **Main loop:**
    1. toggle the status LEDs every 200 ms and send the `UPPER_INPUT` heartbeat every 10 seconds;
    2. `ScanInputs()` drives each of the 16 rows low in turn with direct `PORTA` and `PORTC` writes and reads the columns with direct `PIND`, `PING` and `PINL` reads, building a 176-button state array;
    3. `FindInputChanges()` debounces and, for each change, calls `SendButtonToHid(index, state)`;
    4. `scanMatrix()` scans the 4 by 3 keypad and sends `SendKeyToHid(key, 1)` on press and `(key, 0)` on release;
    5. every 20 ms `ServiceControlPackets()` polls port 13136 and acts **only** on `C,RESET:1`; every other packet is ignored. The poll is slow on purpose so the extra W5500 access does not slow the matrix scan.

6.2 Pin usage

Pin	Function
14	Green status LED
15	Red status LED

Pin	Function
53	W5500 reset
22 to 37	16 matrix row drivers
38 to 49	Matrix column inputs; pins 50 to 53 are the Ethernet SPI bus, so the last four columns are fixed at 1 (unused)
A10, A12, A14, A15	Keypad rows
A9, A11, A13	Keypad columns
USB serial	DCS-BIOS link to a DCS World PC (separate from the Ethernet path)

6.3 What it sends

To	Port	Content
172.16.1.11 (USB HID board)	4210	`index,state` for matrix switches and `KEY,key,state` for keypad keys
172.16.1.10	27000	Debug text
172.16.1.10	13137	`UPPER_INPUT` heartbeat every 10 seconds

The Upper controller has its own address and MAC (`172.16.1.103`, `...83:03`); it never listens for simulator data, so no bridge sends to it.

6.4 Remote reset

`C,RESET:1` on port 13136 reboots the board with a software reset (see the network chapter). The code is the same as the Radio controller's and the sketch compiles, but it has **not been flashed or bench-tested** because the board was not connected.

6.5 Known issues

- The Upper controller PCB has a Warning Mute connector (J17) on `/ROW8`, Arduino D30, but the sketch's keypad uses analog pins A9 to A15, so J17 may not be read.
- `CreateDcsBiosMessage()` has no bodies, so none of these switches do anything in DCS.
- The PCB (v1.4) also has a backlight driver on D5 (`BACKLIGHT_PWM` to an IRF540N MOSFET that switches the BACKLIGHT connector) and a brightness input on A0 (`GUAGE_BACKLIGHT_BRIGHT`, connector J16), but the sketch uses neither, and it ignores `C,B` packets.

7 USB HID board (172.16.1.11)

`UDP_TO_HID` runs on an Arduino with native USB (Leonardo class, board option `arduino:avr:leonardo`) and a W5500 Ethernet module. It is a network-to-USB bridge: it receives button and key commands over UDP from the Upper controller, the Servo controller, the Radio controller (Auto Pilot and Audio panels) and the ADF controller and re-emits them to Windows as **four virtual 32-button joysticks (128 buttons) and a keyboard**. The joysticks have buttons only: no axes and no hat switches.

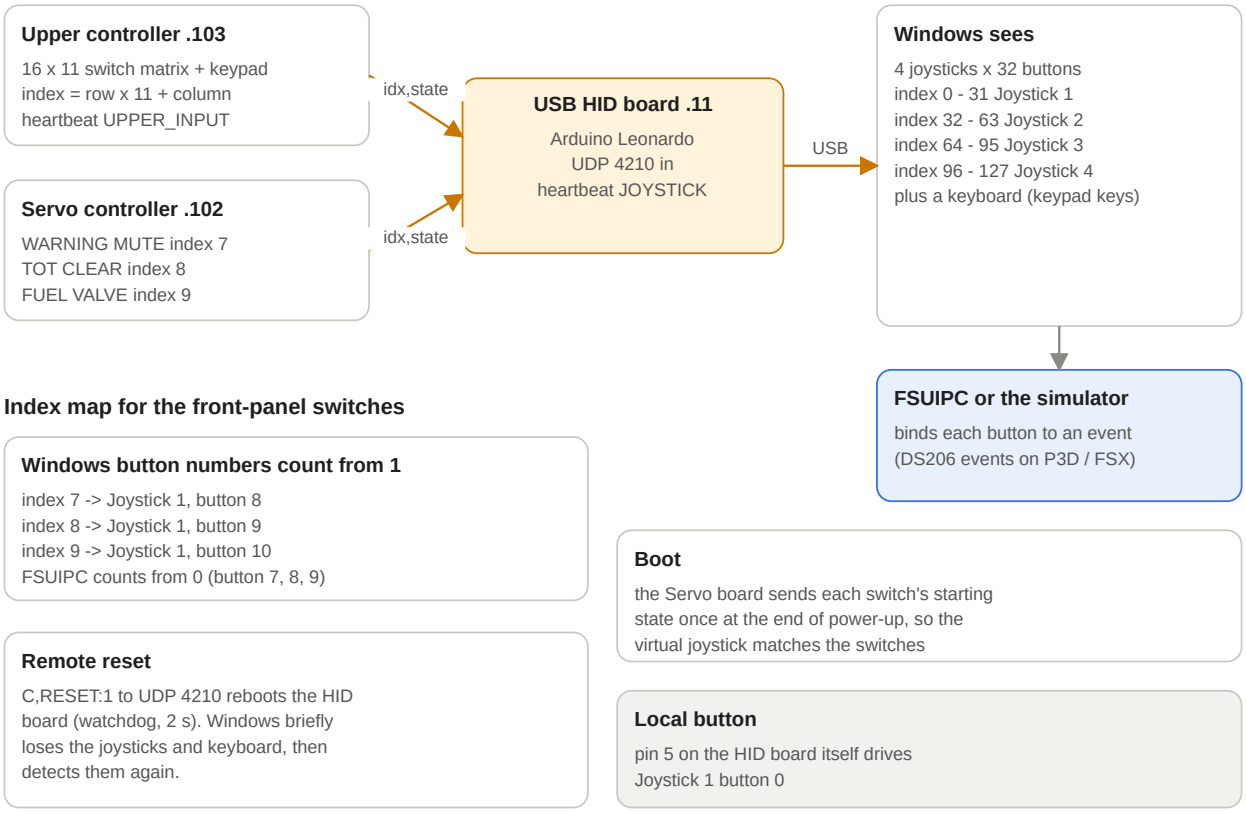


Figure 17 — From switch to joystick button

7.1 Program flow

1. **Setup:** start the four joysticks and the keyboard, bring up Ethernet on the static address, open the command, log and heartbeat sockets and log a boot message.
2. **Main loop:**
   1. send the `JOYSTICK` heartbeat every 10 seconds;
   2. read the one local push button on pin 5 and mirror it on Joystick 1, button 0;
   3. read any UDP command on port 4210, log the sender and payload and call `processCommand()`.

`processCommand()` understands three formats:

Payload	Effect
`index,state`	Sets button `index mod 32` of joystick `index div 32` (0 to 31 Joystick 1, 32 to 63 Joystick 2, 64 to 95 Joystick 3, 96 to 127 Joystick 4) pressed (1) or released (0)
`KEY,key,state`	Presses or releases a keyboard key. A single character is sent as its ASCII value; longer names (`ENTER`, `F5`, `LCTRL`) map to key codes
`C,RESET:1` (or `RESET`)	Reboots the board through the AVR watchdog; Windows briefly loses the joysticks and keyboard

7.2 How a switch becomes a joystick button

1. The Upper controller numbers each matrix position as `index = row x 11 + column` (0 to 175).
2. When a switch changes it sends `index,state` to `172.16.1.11:4210` with state 1 = switch on.
3. The HID board holds that button down while the switch is on.
4. Keypad keys are sent as `KEY,<key>,<state>` and typed as keyboard keys 0 to 9, `*` and `#`.
5. The Servo controller sends the same format for its three front-panel switches (indexes 7, 8 and 9).

Index range	Windows device
0 to 31	Joystick 1
32 to 63	Joystick 2
64 to 95	Joystick 3
96 to 127	Joystick 4
128 to 175	Not sent to Windows (out of range)

7.3 Auto Pilot, Audio and ADF panel buttons

The Radio controller (Auto Pilot and Audio panels) and the ADF controller send their switches and knob clicks in **Joystick 3** (indexes 64 to 95, Auto Pilot panel) and **Joystick 4** (96 to 127, Audio and ADF panels). Nothing else uses those buttons: the Upper controller's assigned switches are all below index 40, and the Servo's are 7 to 9. The Upper matrix positions that map onto indexes 64 to 127 (rows 5 to 11) are wired but have no switch on them, so if switches are ever fitted there they would clash with these buttons. Windows numbers buttons from 1 (button = index mod 32 + 1). A knob click is a 40 ms press, clockwise and anticlockwise on adjacent buttons.

Index	Windows button	Source
64 to 71	Joystick 3, buttons 1 to 8	Auto Pilot panel switches: ENGAGE, HDG, CRS, ALT, APPR, FD, BC, IAS
72 to 81	Joystick 3, buttons 9 to 18	Auto Pilot knobs, clockwise then anticlockwise: CRS, ALT, VS, IAS, HDG
96 to 103	Joystick 4, buttons 1 to 8	Audio panel switches: COM1, BOTH, NAV2, DME, COM2, NAV1, MKR, ADF
104 to 107	Joystick 4, buttons 9 to 12	ADF panel: landing gear, flaps, speed brakes, brakes
108 to 111	Joystick 4, buttons 13 to 16	ADF knobs, clockwise then anticlockwise: major, minor

Each of these is also sent to the bridge as a simulator control, so binding the joystick buttons in FSUIPC or MSFS is optional (see the Radio and ADF controller chapters for the full table).

7.4 Switch mapping in use

From `UpperSwitchMapping.ods`. Button numbers are as Windows and the simulators show them (1 to 32); the firmware counts from 0. A latching switch holds its button down while on, so bind both press and release. The suggested events are the closest match in `DODSIM206V2.EVT`; the bindings actually saved in FSUIPC or MSFS are not in the repository. All other matrix positions are wired but unassigned.

Index	Source	Switch	Joystick 1 button	Suggested DodoSim event
0 to 4	Upper 0, 0-4	Heater rotary HTR 0 to 4	1 to 5	
5	Upper 0, 5	Air vent	6	`HEAT_VENT_SWITCH_SET`
6	Upper 0, 6	Air heat	7	`HEAT_VENT_SWITCH_SET`
7	Servo D47	Warning mute	8	`DS206.WARNING_MUTE_BUTTON_SET`
8	Servo D43	TOT clear (TOT light test)	9	`DS206.TOT_TEST_BUTTON_SET`
9	Servo D48	Fuel valve	10	`DS206.FUEL_VALVE_SWITCH_SET`
11	Upper 1, 0	Forward fuel boost breaker	12	`FORE_FUEL_PUMP_CB_SET`
12	Upper 1, 1	Aft fuel boost breaker	13	`AFT_FUEL_PUMP_CB_SET`
13	Upper 1, 2	Generator field breaker	14	`GENERATOR_FIELD_CB_SET`
14	Upper 1, 3	Generator breaker	15	`GENERATOR_RESET_CB_SET`
15	Upper 1, 4	Directional gyro / attitude indicator	16	`DIR_GYRO_SWITCH_SET`

Index	Source	Switch	Joystick 1 button	Suggested DodoSim event
16	Upper 1, 5	Battery	17	`BATTERY_SWITCH_SET`
17	Upper 1, 6	Generator (on / start)	18	`GENERATOR_SWITCH_SET`
22	Upper 2, 0	Landing light	23	`LANDING_LIGHT_SWITCH_SET`
23	Upper 2, 1	Instrument lights	24	`PANEL_LIGHTS_SWITCH_SET`
24	Upper 2, 2	Caution light breaker	25	`CAUTION_LT_CB_SET`
25	Upper 2, 3	Anti-collision light	26	`COLLISION_LIGHTS_SET`
26	Upper 2, 4	Position lights	27	`POSITION_LIGHTS_SET`
27	Upper 2, 5	Pitot heat	28	`PITOT_HEAT_SWITCH_SET`
28	Upper 2, 6	Avionics	29	`AVIONICS_SWITCH_SET`
39	Upper 3, 6	Strobe light	Joystick 2, 8	none yet

7.5 Pin usage and network

Item	Value
Pin 5	Local push button, Joystick 1 button 0
Static IP, MAC	`172.16.1.11`, `DE:AD:BE:EF:FE:ED`
Listen port	4210 (button, key and reset commands)
Log to	172.16.1.10:27000 (boot, button and key events, malformed commands)
Heartbeat to	172.16.1.10:13137, `JOYSTICK` every 10 seconds

The Leonardo is flashed on COM8 with `--fqbn arduino:avr:leonardo`. Bench-tested remote reset: "Joystick controller booted" about 4 seconds after the command.

7.6 Known issues

- The board's own test button on pin 5 also drives Joystick 1 button 1, the same button as HTR 0.
- The Servo controller's three switches use indexes 7, 8 and 9, which Windows shows as buttons 8, 9 and 10. If Windows buttons 7, 8 and 9 were wanted, button 7 is already the Air heat switch (index 6), so the indexes would have to move.
- After a reset the USB connection drops; re-check any game that has the joysticks open.

8 Stepper controller (172.16.1.105)

`JET_RANGER_STEPPER_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the Jet Ranger 13 Stepper PCB. It drives X27-style stepper gauges directly from the Mega's pins (no driver chips) and takes **real units**, converting them to needle steps with calibration tables in the sketch. The oil, transmission, RPM, load and altimeter gauges are on the Dual Stepper board (next chapter).

The sketch started as an A-10C front instrument board and still carries some DCS-BIOS code and an unused SARI roll stepper claim on pins 30 and 32; the Jet Ranger path is the UDP protocol.

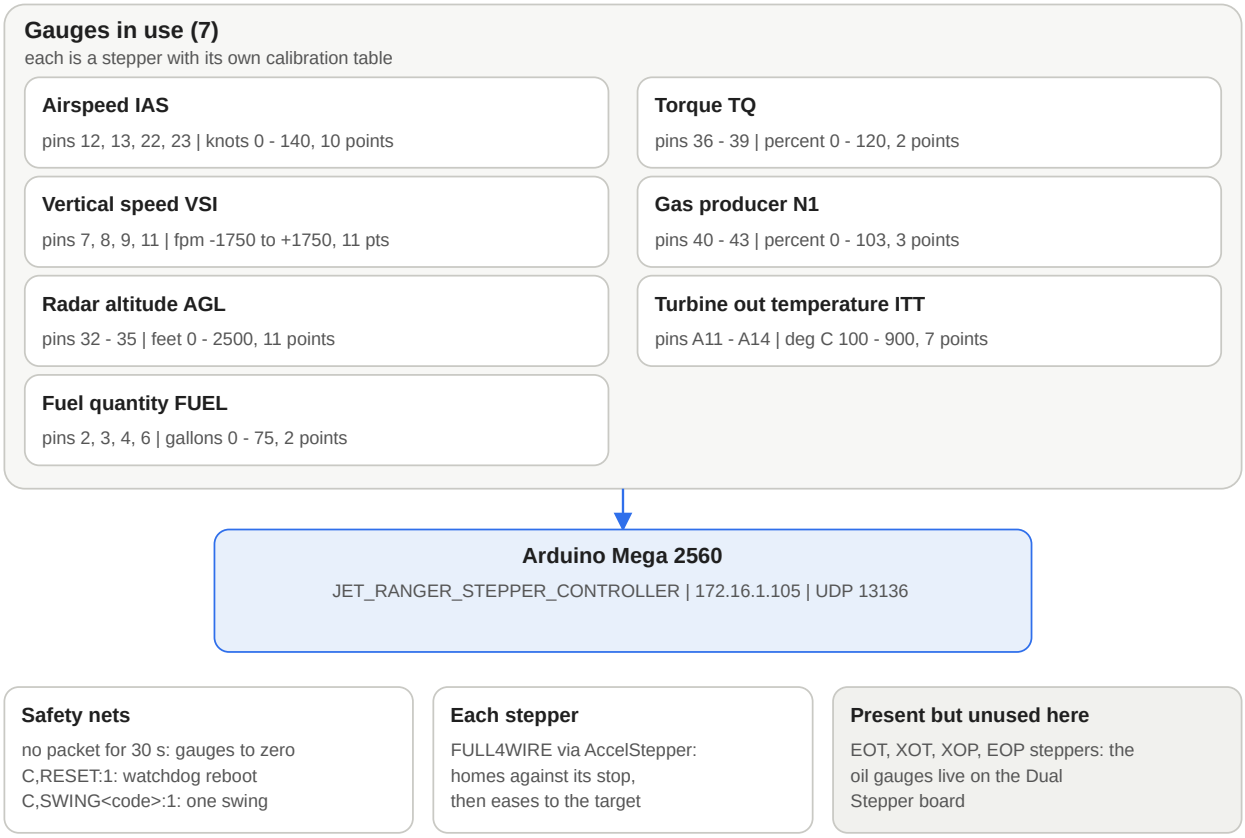


Figure 18 — Stepper controller blocks

8.1 Gauges and calibration

Gauge	Token	Mega pins	Unit and range	Calibration
Airspeed	`IAS`	12, 13, 22, 23	knots 0 to 140, one decimal allowed	`IAS_KT_TABLE`, 10 points (0 to 635 steps)
Vertical speed	`VSI`	7, 8, 9, 11	ft/min -1750 to +1750	`VSI_FPM_TABLE`, 11 points (-311 to +315 steps)
Radar altitude	`AGL`	32 to 35	feet 0 to 2500	`AGL_FT_TABLE`, 11 points (0 to 469 steps)
Torque	`TQ`	36 to 39	percent 0 to 120	`TQ_PCT_TABLE`, 2 points (0 to 600 steps)
Gas producer (N1)	`N1`	40 to 43	percent 0 to 103	`GP_PCT_TABLE`, 3 points (0 percent is step 15)
Turbine out temperature	`ITT`	A11 to A14	degrees C 100 to 900	`EGT_C_TABLE`, 7 points (100 C is step 0)
Fuel quantity	`FUEL`	2, 3, 4, 6	US gallons 0 to 75	`FA_GAL_TABLE`, 2 points (0 gal is step 53)

Notes:

- The first IAS row (0 kt = step 0) is an assumed zero. The VSI and IAS steppers are wired C, D, A, B.

- Every token has a raw twin (`IASRAW:300`) that moves the needle by raw steps and skips the table.
- Four more steppers are declared with placeholder linear scales and no start-up swing: engine oil temperature (`OILT`, pins 48, A0 to A2), transmission oil temperature (`XMSNT`, A3 to A6), transmission oil pressure (`XMSNP`, A7 to A10) and engine oil pressure (`OILP`, 44 to 47). The oil gauges are really driven by the Dual Stepper; wire each physical gauge to one board only.
- `RPME` and `RPMR` packets sent to this board are ignored.
- **Zero trims:** each gauge has a `<CODE>ZeroOffset` value (steps) to nudge its true zero without touching its table. Current values: `AGLZeroOffset` 10, `TQZeroOffset` 23, all others 0. The trim is applied in each gauge's real-value setter, so a `<CODE>:0` packet, the watchdog reset and the end of each swing all land on the same trimmed position. The `*RAW` codes do not apply it.

8.2 Program flow

1. **Setup:** flash the status LEDs, bring up Ethernet (10 ms retransmission timeout), ramp the backlight, then run each gauge's boot swing in this order, each behind its own `SwingXXX` gate: fuel quantity, torque, turbine out temperature, gas producer, VSI, airspeed, radar altitude. All seven gates are currently `true` (`SwingLoops` is 2). The 30 second watchdog is armed at the end of `setup()`.
2. **Main loop:** toggle the status LEDs, call `.run()` on every stepper, send the `STEPPER` heartbeat every 10 seconds, and every 5 ms check UDP 13136 and store new targets. Steppers move with smooth acceleration (maximum speed 9000, acceleration 1000).
3. **Watchdog:** after 30 seconds with no packet every gauge is driven to its calibrated zero through its own setter (trim included).

8.3 Packets

The bridges send this board its own packet:
`D,BPWR:1,IAS:93.4,VSI:500,AGL:850,FUEL:42,TQ:65,ITT:610,N1:92`. `AGL` is zero below 20 V on the main bus. The fractional part of `IAS` is interpolated between table rows, so the needle moves smoothly rather than in whole-knot jumps.

8.4 Backlight

Pin 5 (the PCB's `BACKLIGHT_PWM`) drives the panel backlights through the board's IRF540N MOSFET (`BACK_LIGHTS`). `applyBacklight()` sets the level from `BPWR`, the last `C,B:0-15` brightness command and the 30 second no-data watchdog (see the network chapter): the default is PWM 65; once a brightness command has been received the last commanded level returns whenever power returns; `BPWR:0` or 30 seconds with no packet (including after power-up with no PC feed) switches it off, and the next packet switches it back on. The backlight still starts at 128 during setup. The old DCS-BIOS brightness handlers still call `analogWrite()` directly but never run. Each change is logged (`Brightness 15 -> PWM 255`, `Backlight power off`, `Backlight restored - UDP data resumed`). Compiled at 31,544 bytes of flash (12%) and 50% RAM; flashed to the Stepper on COM4.

8.5 On-demand swings

Send `C,<COMMAND>:1` (the value is ignored) to run one gauge's swing without rebooting. The swing blocks the loop for about 10 seconds and re-homes the stepper, so the needle then rests at its calibrated zero until the next packet.

Command	Gauge
`SWINGFUEL`	Fuel quantity
`SWINGTQ`	Torque
`SWINGITT`	Turbine out temperature
`SWINGN1`	Gas producer
`SWINGVSI`	Vertical speed
`SWINGIAS`	Airspeed

Command	Gauge
`SWINGAGL`	Radar altitude

The airspeed and radar altitude swings return to zero after the full-scale wind-up instead of 635 steps past it.

`C,B:0-15` sets the backlight brightness (above). `C,RESET:1` reboots the board through the AVR watchdog (bench-tested: "Ethernet Started" 6.6 seconds later, a normal boot, no loop).

8.6 Pin usage

Pin	Function
14, 15	Green and red status LEDs
53	W5500 reset
5	Backlight PWM (`BACK_LIGHTS`; the PCB routes `BACKLIGHT_PWM` to D5 and VSI coil B to D8)
7, 8, 9, 11	VSI coils
12, 13, 22, 23	Airspeed coils
32 to 35	Radar altitude coils
36 to 39	Torque coils
40 to 43	Gas producer coils
A11 to A14	Turbine out temperature coils
2, 3, 4, 6	Fuel quantity coils
48, A0 to A2	Engine oil temperature coils
A3 to A6	Transmission oil temperature coils
A7 to A10	Transmission oil pressure coils
44 to 47	Engine oil pressure coils
24 to 31	Unused (Turbine and Rotor speed moved to the Dual Stepper)

8.7 Network

Static IP `172.16.1.105`, MAC `A8:61:0A:67:83:69`, data port 13136, heartbeat `STEPPER` to 172.16.1.10:13137, debug to 172.16.1.10:27000. The StepperVSITester can send every token and raw token to this board without a simulator.

8.8 Known issues

- Pin 32 is claimed by both the radar altitude coil and the unused SARI roll stepper; pin A2 (56) is claimed by the engine oil temperature coil and the shared stepper enable pin. Neither SARI nor the enable pin is used.
- `VSIstepper.runToNewPosition(-X27_FULLWIRE_HOMING_STEPS)` expands to -635 + 5 = -630, not -640, because the macro is not parenthesised (about 1.5 percent).
- The placeholder oil and transmission gauges use a shared linear scale, not bench-measured tables.

9 Dual stepper controller (172.16.1.106)

`JET_RANGER_OLED_DUAL_STEPPER_CONTROLLER` runs on a third Mega 2560, on the Jet Ranger OLED Dual Stepper PCB. It drives stepper gauges for engine and transmission oil, engine and rotor RPM, fuel load, electrical load and the altimeter, plus **four small OLED displays**. Like the Stepper board it takes real units. It started as a copy of the Stepper sketch, which is why the two share their structure, their zero-trim approach and most of their code.

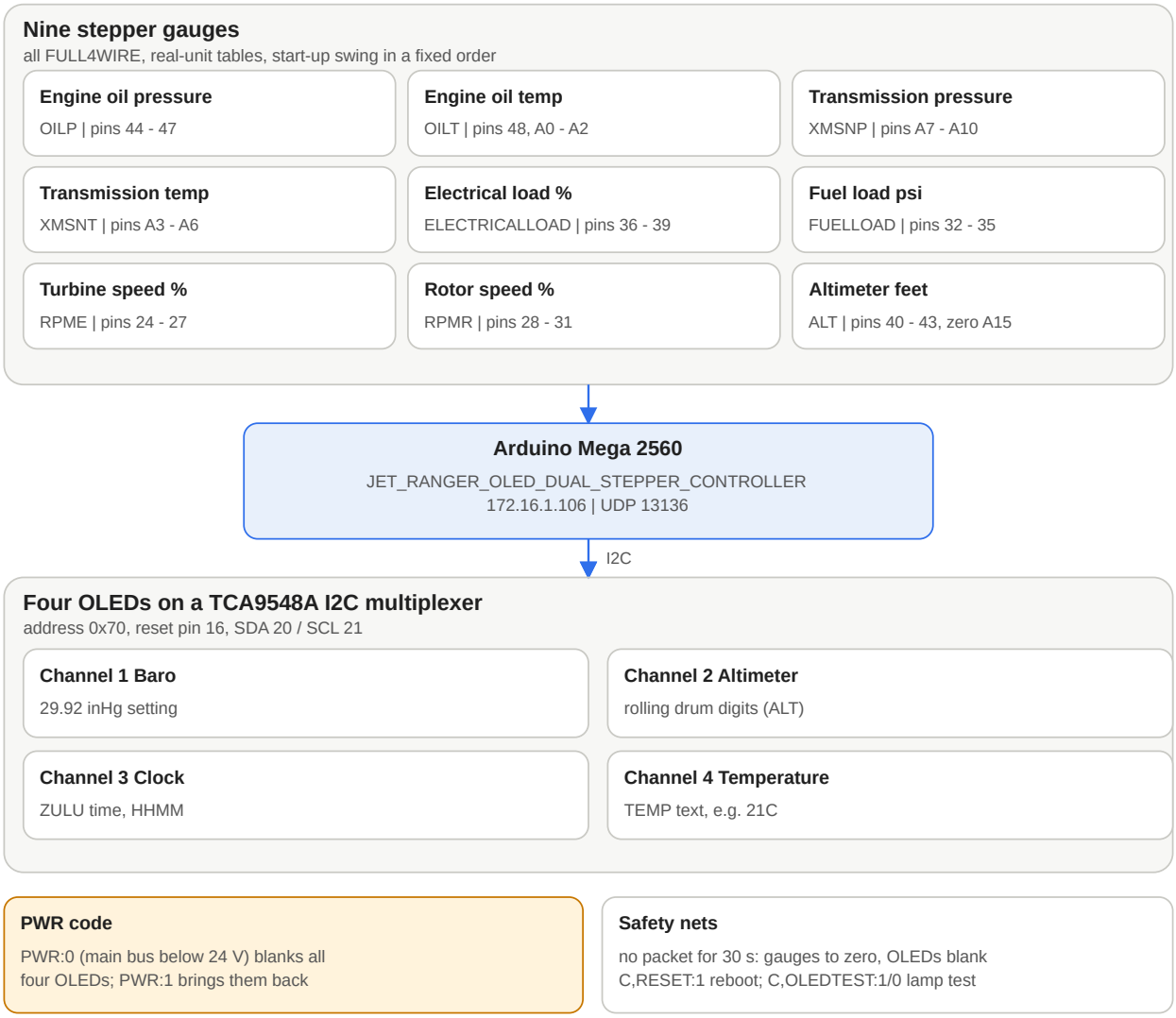


Figure 19 — Dual stepper controller blocks

9.1 Gauges and calibration

Gauge	Token	Mega pins	Unit	Calibration
Engine oil pressure	`OILP`	44 to 47	psi	`EOP_PSI_TABLE`: 0, 50, 100, 150 psi = 0, 74, 148, 220 steps
Engine oil temperature	`OILT`	48, A0 to A2	degrees C	`EOT_C_TABLE`: 0, 50, 100, 150 C = 0, 79, 156, 231 steps
Transmission oil pressure	`XMSNP`	A7 to A10	psi	`XOP_PSI_TABLE`: 0, 50, 70, 100, 150 psi = 0, 74, 104, 149, 220 steps
Transmission oil temperature	`XMSNT`	A3 to A6	degrees C	`XOT_C_TABLE`: 0, 50, 100, 150 C = 0, 75, 150, 225 steps
Electrical load	`ELECTRICALLOAD`	36 to 39	percent	`ELECTRICAL_LOAD_PCT_TABLE`: 0

Gauge	Token	Mega pins	Unit	Calibration
				percent = step 22, 100 percent = step 200
Fuel load	FUELLOAD	32 to 35	psi	FUEL_LOAD_PSI_TABLE: 0 psi = 0, 30 psi = 160 steps
Engine speed (N2)	RPME	24 to 27	percent, one decimal	TS_PCT_TABLE: 0, 55, 100, 110 percent = 0, 300, 535, 600 steps
Rotor speed	RPMR	28 to 31	percent, one decimal	RS_PCT_TABLE: the same values
Altimeter	ALT	40 to 43, zero sensor A15	feet	720 steps per 1000 ft

Turbine out temperature (ITT) and fuel quantity (FUEL) are also accepted, with the same tables as the Stepper board; wire each physical gauge to one board only. VSI, IAS, AGL, TQ and N1 are not handled here.

- Every token has a raw twin (FUELLOADRAW, RPMERAW, ALTRAW and so on).
- The 0 percent point of the RPM tables is assumed, and the 0 point of the electrical-load table is step 22, not zero.
- **Zero trims (steps):** TSoffset 10, RSoffset 0, EOTZeroOffset 24, EOPZeroOffset 0, XOTZeroOffset 16, XOPZeroOffset 5, FuelLoadZeroOffset 55, ElectricalLoadZeroOffset 24. The altimeter has none.
- The altimeter seeks its zero sensor on A15 by moving up to 10,080 steps (about 14 turns) before giving up, then sets position -25 and sweeps one 720 step revolution. The altimeter's own speed is 600 steps per second.

9.2 OLED displays

The diagram above shows the four displays on the multiplexer. Four 128 by 32 SSD1306 OLEDs share the Mega's I2C pins (20 SDA, 21 SCL) through a TCA9548A multiplexer at address 0x70. The firmware pulses the mux reset (pin 16) at start-up, calls Wire.begin() with a 25 ms timeout and probes the mux; if it does not answer the sketch logs "TCA mux NOT found - OLED init skipped" and starts the swings at once, then retries once after the swings.

Display	Mux channel	Token	Shows
Barometric setting	1	none yet	2992 at boot, not updated
Altimeter digits	2	ALT	Rolling-drum altitude, raw MSL (barometric correction ignored)
Clock	3	ZULU	Zulu time from HHMM, for example ZULU:1430 shows 14:30
Temperature	4	TEMP	The text as sent, for example TEMP:25C (up to 8 characters)

Updates are gated on change and throttled to every 300 ms. A boot-time I2C scan (OLED_I2C_SCAN) logs one line per channel to the debug stream, for example I2C scan ch1: 0x3c or I2C scan ch3: none; the latest scan found OLEDs on channels 1 to 4. The TCA9548A address pins (D17 to D19) are not driven by the sketch: the module's own pull-downs set address 0x70, so keep a module with pull-downs.

Power: PWR:0 (sent first in the bridge's packet while the main bus is below 24 V) blanks all four OLEDs and PWR:1 redraws them. The altimeter needle keeps following altitude. The lamp test ignores PWR.

Lamp test: C,OLEDTEST:1 fills all four OLEDs solid white and C,OLEDTEST:0 redraws their default content and clears the change gates so live data repaints them. It is ignored (with a debug line) if the mux was not found.

9.3 Backlight

Pin 5 (the PCB's BACKLIGHT_PWM) drives the panel backlights through the board's IRF540N MOSFET (BACK_LIGHTS). applyBacklight() sets the level from BPWR, the last C,B:0-15 brightness command and the 30 second no-data watchdog (see the network chapter): the default is PWM 65; once a brightness command has been received the last commanded level returns whenever power returns; BPWR:0 or 30 seconds with no packet (including after power-up with no PC feed) switches it off, and the next packet switches it back on. The backlight still starts at 128 during setup. The old DCS-BIOS brightness handlers still call analogWrite() directly but never run. Each change is logged (Brightness 15 -> PWM 255, Backlight power off, Backlight restored - UDP data resumed). Compiled at 58,312 bytes of flash (22%) and 79% RAM (1,720 bytes free); flashed to the Dual Stepper on COM5.

9.4 Program flow

1. **Setup:** initialise the OLEDs first so they are up while the needles swing, then run each gauge's swing in this order, each behind its own `SwingXXX` gate (all currently `true`, `SwingLoops` 1): engine oil pressure, engine oil temperature, transmission oil pressure, transmission oil temperature, electrical load, fuel load, engine speed, rotor speed, altimeter. Start-up takes about 65 seconds. The 30 second watchdog is armed at the end of `setup()`.
2. **Main loop:** run the steppers, send the `DUAL_STEPPER` heartbeat every 10 seconds and check UDP 13136 every 5 ms.
3. **Watchdog:** after 30 seconds without a packet every needle returns to its calibrated zero and all OLEDs blank.

9.5 Packets

Bridge packet:
`D,PWR:1,BPWR:1,ALT:1200,RPMR:100.0,RPME:100.0,ZULU:1430,OILT:60,OILP:55,XMSNT:50,XMSNP:35,FUELLOAD:17,ELECTRICALLOAD:50,TEMP:21C`. Rotor and engine speed carry one decimal so the fraction reaches the table interpolation.

9.6 On-demand commands

Command	Action
`SWINGOILP`, `SWINGOILT`, `SWINGXMSNP`, `SWINGXMSNT`	Oil and transmission gauges (about 5 to 6 seconds each)
`SWINGELECTRICALLOAD`, `SWINGFUELLOAD`, `SWINGRPME`, `SWINGRPMR`	Load and RPM gauges (about 6 to 7 seconds each)
`SWINGALT`	Altimeter (re-finds the zero sensor)
`OLEDTEST:1` / `OLEDTEST:0`	OLED lamp test on and off
`B:0` to `B:15`	Backlight brightness
`RESET:1`	Watchdog reboot (bench-tested: "Ethernet Started" 6.7 seconds later and a normal boot in about 56 seconds)

All are sent as `C,<COMMAND>:1`. The oil, transmission, load and RPM swings wind to the stop and return to the zero offset; the electrical-load swing returns to its 22 step zero.

9.7 Pin usage

Pin	Function
14, 15	Status LEDs
16	TCA9548A reset
20, 21	I2C SDA, SCL
53	W5500 reset
5	Backlight PWM (PCB `BACKLIGHT_PWM`)
24 to 27	Engine speed coils
28 to 31	Rotor speed coils
32 to 35	Fuel load coils
36 to 39	Electrical load coils
40 to 43, A15	Altimeter coils and zero sensor
44 to 47, 48, A0 to A2	Oil pressure and temperature coils
A3 to A10	Transmission temperature and pressure coils
2, 3, 4, 6, A11 to A14	Fuel quantity and turbine out temperature coils

9.8 Network

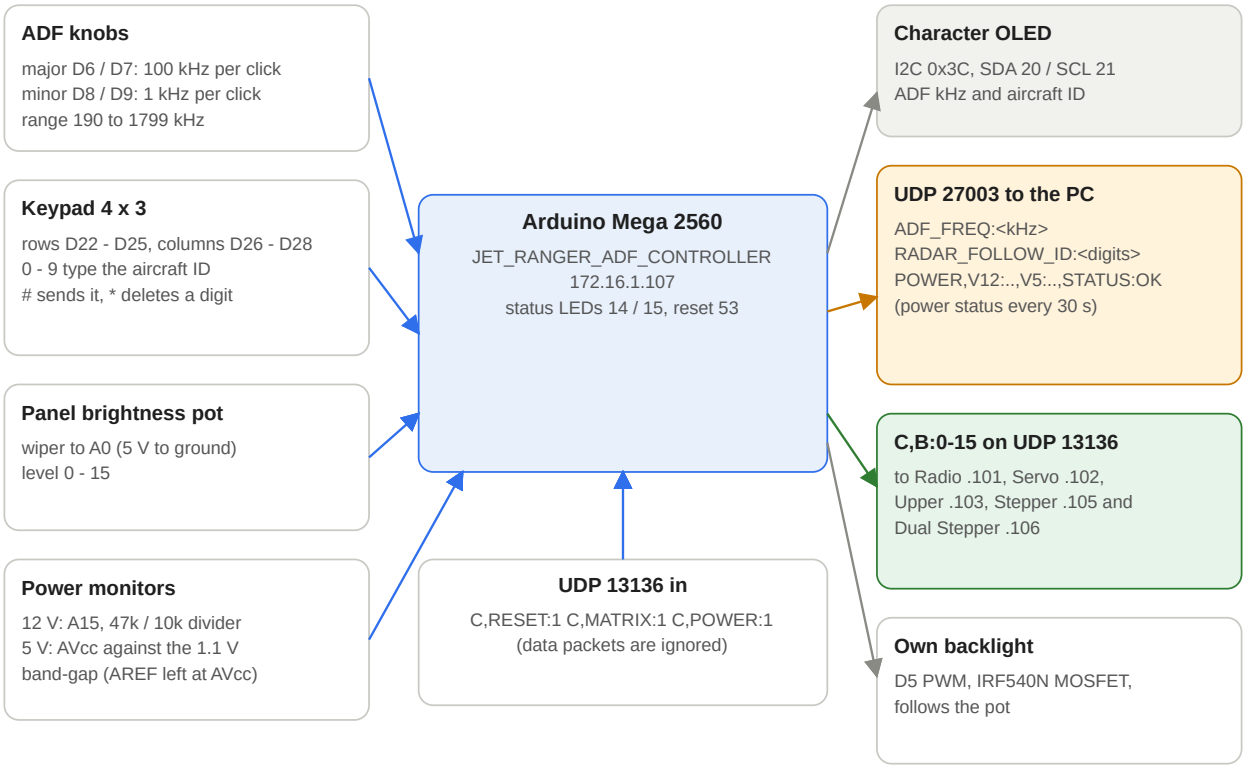
Static IP `172.16.1.106`, MAC `A8:61:0A:67:83:6A`, data port 13136, heartbeat `DUAL_STEPPER` (a prefix distinct from the Stepper's `STEPPER`, so the Health Monitor can tell them apart) to 172.16.1.10:13137, debug to 172.16.1.10:27000 including the I2C scan.

9.9 Known issues

- **RAM use is 79 percent** and the compiler warns "Low memory available". Avoid adding OLEDs or string-heavy code.
- The barometric OLED is initialised but never updated; nothing feeds it.
- The altimeter digits ignore barometric offset.
- Engine and rotor RPM tables assume 0 percent is 0 steps; `RSoffset` is not measured.
- The Clock OLED (channel 3) answered on the I2C scan but its display has not been confirmed by eye.
- **PCB version 1.4 (12 Oct 2026)** removes a short between Mega pins 2 and 32 (a copied net name joined fuel quantity coil A and fuel load coil A in copper). On a board older than 1.4 fit only one of those two gauges or cut the trace.

10 ADF controller (172.16.1.107)

`JET_RANGER_ADF_CONTROLLER` runs on an Arduino Mega 2560 with a W5500 Ethernet module, on the **Jet Ranger ADF Controller** PCB (revision 1.4). It is built from the Radio controller sketch (same Ethernet start-up, debug stream, heartbeat, polling encoder library, matrix scanner, character OLED start-up and software reset) and does five jobs: it tunes the ADF with two knobs, takes the ID of an aircraft to follow on the radar from a keypad, reads the **panel brightness pot** and sends the level to the other panels, and monitors the **12 V and 5 V** supply rails. The sketch compiles for `arduino:avr:mega` (25 KB of flash, 1.5 KB of RAM). It has **not been flashed or bench-tested** yet.



Landing gear lamps (D2 - D4) and the gear, flaps, speed brake and brake inputs (D29 - D32) are not used yet.

Figure 20 — ADF controller blocks

10.1 Program flow

1. **Setup:** set the status LEDs, read the brightness pot and set the board's own backlight, reset the W5500, start Ethernet with the 10 ms retransmission timeout and open the sockets (red LED flashes for 3 seconds), start the I2C character OLED, start the two encoders (centred at 1000) and the matrix scanner.
2. **Main loop:**
   1. toggle the status LEDs every 200 ms and send the `ADF_PANEL` heartbeat every 10 seconds;
   2. read both encoders. Each is kept near the middle of its range and recentred after every read, so the change since the last read is the distance from the middle; three counts make one click. The major knob changes the ADF frequency by 100 kHz per click and the minor knob by 1 kHz per click (the minor knob speeds up when it is turned quickly), clamped to 190 to 1799 kHz. 250 ms after the last movement the frequency is sent as `ADF_FREQ:<kHz>`;
   3. `scanMatrix()` scans the 4 by 3 keypad with a 50 ms debounce. For the first 1.5 seconds after start-up it only records the state, so a floating column is not reported as key presses;
   4. the keys build an aircraft ID: digits 0 to 9 add to it (up to 8), `#` sends `RADAR_FOLLOW_ID:<digits>` and makes it the followed ID, `*` deletes the last digit, and `*` on an

empty entry stops following (`RADAR_FOLLOW_ID:0`). An unfinished entry is dropped after 20 seconds;

5. read the brightness pot (below) and send any pending brightness packet;
6. redraw the OLED when something changed (at most every 50 ms);
7. send the power status (below);
8. poll port 13136 and act on the control packets.

10.2 Pin usage

Pin	Function
D2, D3, D4	Landing-gear lamps (nose, left, right): on the PCB, not driven yet
D5	`BACKLIGHT_PWM`: this board's backlight MOSFET (IRF540N)
D6, D7	ADF major encoder A, B
D8, D9	ADF minor encoder A, B
D14, D15	Green and red status LEDs
D20, D21	I2C SDA and SCL (character OLED at 0x3C)
D22 to D25	Keypad rows (`KEYPAD_7` to `KEYPAD_4`)
D26 to D28	Keypad columns (`KEYPAD_3` to `KEYPAD_1`)
D29 to D32	Landing gear, flaps, speed brakes and brakes switches (to ground when on)
D53	W5500 reset
A0	Panel brightness pot wiper (`GUAGE_BACKLIGHT_BRIGHT`)
A15	12 V monitor (`12V-Monitor`)

10.3 Panel brightness

The pot on A0 sits between 5 V and ground. The reading is smoothed (a running average over 8 samples every 20 ms) and converted to a **level 0 to 15**; a new level is accepted only once the pot is clearly past the boundary, so a reading on a boundary does not flicker. A new level:

- sets this board's own backlight (D5, PWM 0 to 255);
- is sent as `C,B:<level>` on UDP 13136 to the Radio .101, Servo .102, Upper .103, Stepper .105 and Dual Stepper .106 (not the USB HID board).

The level is also sent once 5 seconds after start-up, so the other panels take the pot's position without it being moved. The packets go out **one board per pass of the loop**, so a switched-off board costs at most one 80 ms send timeout and the knobs and keypad keep working. The Servo, Stepper and Dual Stepper boards apply the level; the Radio and Upper controllers accept it but do not drive a backlight yet. A brightness packet counts as received data for the receiving board's 30 second no-data watchdog, but it is sent only when the pot changes (and once at start-up), so it does not hold a board's watchdog off.

10.4 Power monitoring

- **12 V rail:** R3 47 kohm from the rail and R4 10 kohm to ground with C1 0.1 uF across R4, midpoint to A15. The sketch averages 16 readings and scales them by the measured 5 V: volts = pin volts x (47 + 10) / 10. At 12 V the pin sees about 2.1 V, and the circuit reads up to about 28 V.
- **5 V rail:** the ADC reference is AVcc, the 5 V rail the AREF pin follows (the AREF pin is only brought out to a header, and the sketch **leaves the reference at its default**: `analogReference()` is never called). The sketch measures it by converting the chip's internal 1.1 V band-gap against AVcc (ATmega2560 `ADMUX` selection `11110` with `MUX5` 0): AVcc in millivolts = 1125300 / reading. It waits 5 ms after switching the input and throws away the first conversion.
- **Status:** `OK`, or the rails outside their limits joined with `+`: `5V_LOW` (below 4.75 V), `5V_HIGH` (above 5.25 V), `12V_LOW` (below 10.8 V), `12V_HIGH` (above 15.0 V). `V12_CAL` and `V5_CAL` at the top of the sketch trim the readings against a meter.

A power status is sent 5 seconds after start-up, then **every 30 seconds**, and whenever `C,POWER:1` arrives.

10.5 What it sends

To	Port	Content
172.16.1.10 (PC)	27003	`ADF_FREQ:<kHz>`, `RADAR_FOLLOW_ID:<digits>` (`0` = stop following) and `POWER,V12:12.41,V5:5.02,STATUS:OK`
172.16.1.101, .102, .103, .105, .106	13136	`C,B:<0-15>` panel brightness
172.16.1.10 (PC bridge)	27001	The four panel switches as simulator controls: `GEAR_DOWN` / `GEAR_UP`, `FLAPS_DOWN` / `FLAPS_UP`, `SPOILERS_ON` / `SPOILERS_OFF` and `PARKING_BRAKES` (the bridges recognise these names)
172.16.1.11 (USB HID board)	4210	The four switches and the two ADF knobs as joystick buttons `index,state`: indexes 104 to 107 and 108 to 111 (Joystick 4, buttons 9 to 16)
172.16.1.10	27000	Debug text, each line prefixed `Jet Ranger ADF:`, including every power status
172.16.1.10	13137	`ADF_PANEL` heartbeat every 10 seconds

The events use **port 27003, not 27001**, because the FSUIPC and MSFS bridges treat any unrecognised string on 27001 as a COM1 standby frequency and would retune COM1. **Neither bridge reads port 27003 yet**, so the ADF frequency and the radar follow ID do not reach the simulator, and the Health Monitor has no light for `ADF_PANEL`.

10.6 Panel switches

The landing gear (D29), flaps (D30), speed brakes (D31) and brakes (D32) are read with pull-ups and a 30 ms debounce; a closed switch (to ground) is "on". For the first 1.5 seconds after power-up they are only read. Each change sends the joystick button (`index,1` closed or `index,0` open) and the bridge command: the gear switch sends `GEAR_DOWN` when it closes and `GEAR_UP` when it opens, the flaps `FLAPS_DOWN` and `FLAPS_UP`, the speed brakes `SPOILERS_ON` and `SPOILERS_OFF`, and the brakes button sends `PARKING_BRAKES` when it is pressed. When the quiet period ends the joystick buttons are brought into line with the switches as they are, but **no simulator command is sent**, so the aircraft is not moved at power-up. If a switch works the wrong way round, swap its two commands in `adfSwitches[]`. Each ADF knob click is also a 40 ms joystick press (major clockwise and anticlockwise 108 and 109, minor 110 and 111). The full joystick and command table is in the Radio controller chapter.

10.7 What it receives

Only control packets on UDP 13136 (`C,<COMMAND>:<value>`); data packets are ignored and there is **no no-data watchdog** and no power-down of the display.

Command	Action
`C,RESET:1`	Software reset (jump to the reset vector), as on the Radio controller
`C,MATRIX:1`	Print the raw keypad readings to the debug stream (idle line, then one line per row with that row driven low)
`C,POWER:1`	Send a power status now

10.8 OLED

The two-line, 16-character OLED is on the Mega's own I2C bus (no TCA9548A) and uses the Radio controller's start-up sequence. Line 1 is `ADF  340 kHz`. Line 2 is `ID: 1234_` while an ID is being typed, `FOLLOW 1234` for the followed ID, or `FOLLOW OFF`.

10.9 Notes

- The keypad assumes the PCB's connector order puts D22 to D25 on the four rows and D26 to D28 on the three columns, as on the Radio controller. This is **not verified**; use `C,MATRIX:1` and change `rowPins`, `colPins` and `keyMap` if it is wired the other way.

- The knob directions have not been checked. If a knob tunes the wrong way, swap its two pins in the `RotaryEncoder` lines.
- The display code is the Radio controller's character OLED code. If the ADF panel's display is a graphics SSD1306 it needs different code.
- The landing-gear lamps and the landing-gear, flaps, speed-brake and brake inputs are not used yet.
- The rail limits (5 V: 4.75 to 5.25, 12 V: 10.8 to 15.0) are first guesses; check them against the real supply.

11 FSUIPC bridge (FSUIPCWinformsAutoCS)

`FSUIPCWinformsAutoCS` is the bridge for Prepar3D and FSX. It reads flight data through **FSUIPC** (raw memory offsets), sends each panel board its own packet, and listens on UDP 27001 for radio, battery and alternator commands from the Radio controller, acting on them through the FSUIPC API. It is a Windows Forms application (internally named `FSUIPCTest`); its window lists the values being sent and the 21 annunciator bits.

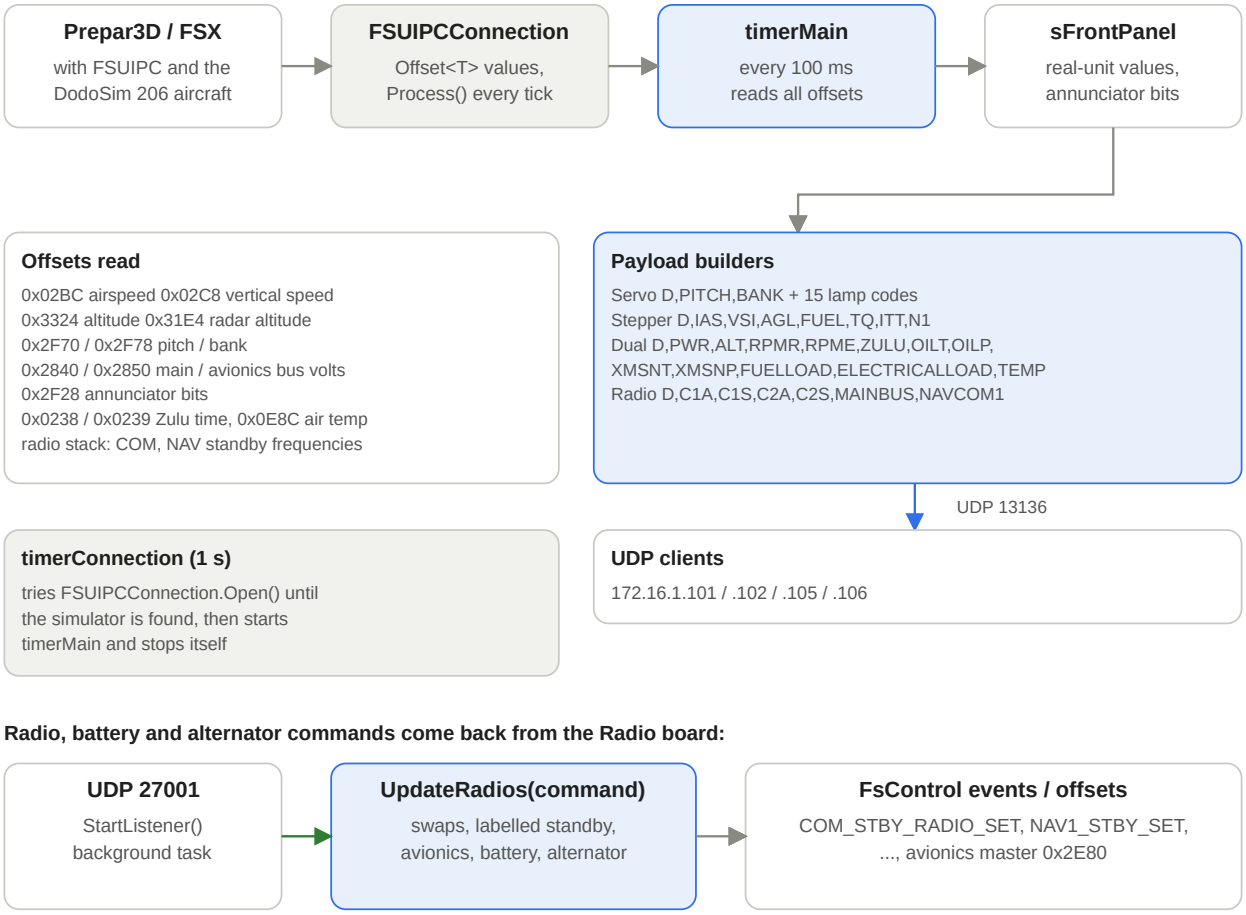


Figure 21 — FSUIPC bridge data flow

11.1 Program flow

1. **Constructor:** build the live annunciator list (one row per bit of the 0x2F28 field), connect the UDP clients and start `timerConnection`, which polls once a second for FSUIPC. Start the UDP listener on port 27001 in a background task. (The UDP client `.Connect` calls live in `Form1.Designer.cs`.)
2. **`timerConnection_Tick`:** keep trying `FSUIPCConnection.Open()`. Once it succeeds, stop and start `timerMain`.
3. **`timerMain_Tick`** (about 100 ms, so roughly 10 packets per second per board):
   1. `FSUIPCConnection.Process()` refreshes every declared offset and the radio-stack group;
   2. format the on-screen readout and update the annunciator list;
   3. build and send each board's packet (below);
   4. send the radio packet when a frequency or the bus voltage changed (at most every 200 ms) or every 5 seconds regardless.
4. **`UpdateRadios(command)`** handles port 27001 (below).

5. **chkAvionicsMaster_CheckedChanged** writes the avionics master offset when the operator toggles the checkbox, and closing the window stops both timers and the FSUIPC connection.

11.2 FSUIPC offsets

Offset	Meaning
0x02BC	Airspeed (knots x 128, so finer than 0.1 knot)
0x02C8	Vertical speed
0x0238, 0x0239	Zulu hours and minutes (combined into the HHMM ZULU)
0x0E8C	Outside air temperature, degrees C x 256 (sent as TEMP, for example 21C; a standard FSUIPC offset, not verified against DodoSim)
0x08F0	Turbine-out percent and temperature
0x08BA, 0x08B8	Engine oil pressure and temperature
0x0900, 0x0904	Transmission oil pressure and temperature
0x0B74, 0x0B78	Fuel percent quantity and fuel capacity
0x08F8	Fuel pressure
0x08F4	Torque percent
0x07BC, 0x07C4, 0x07C8, 0x07D0, 0x07DC, 0x0800, 0x0804	Autopilot master, NAV1 lock (CRS), heading lock, altitude lock, airspeed hold, approach hold and back course hold: the Auto Pilot panel LEDs
0x2EE0	Flight director active: the FD LED
0x3122	Radio audio switches (bit 7 COM1 transmit, 6 COM2 transmit, 5 COM receive both, 4 NAV1 sound, 3 NAV2 sound, 2 marker sound, 1 DME sound, 0 ADF1 sound): the Audio panel LEDs
0x0BAC, 0x0BAE, 0x0BB0	Inner, middle and outer marker: the INNER, MIDDLE and OUTER LEDs
0x0898	Gas producer
0x0908, 0x0896	Rotor RPM and engine N2
0x282C	Battery load amps
0x3324	Altitude
0x31E4	Radio altimeter
0x2F70, 0x2F78	Attitude pitch and bank
0x2F28	21-bit annunciator (warning lamp) bit field
0x2840, 0x2850	Main bus and avionics bus voltage
0x2E80	Avionics master (read and write)
0x0560, 0x0568	Latitude and longitude for the GPS interface (90 degrees = 10001750 x 65536 x 65536; 360 degrees = 65536^4)
0x0570	Altitude in metres for the GPS interface (64-bit with 32 bits of fraction, sent in feet)
0x0580	True heading for the GPS interface (360 degrees = 65536^2)
RadioStack group	COM1 and COM2, NAV1 and NAV2 active and standby, transponder, ADF

11.3 Packets sent

The Servo, Stepper and Dual Stepper boards each get their **own** packet in real units; the boards convert with their own calibration tables. Whole numbers are rounded.

Board	Packet contents
Servo .102	BPWR (1 while the main bus is 20 V or more), PITCH (degrees, one decimal, negated because FSUIPC pitch is nose-down positive), BANK (degrees, one decimal), the 15 lamp codes EOUT, TOPW, BHOT, EC, BD, GENF, TOTW, RLOW, FFLTR, BTMP, TC, TRC, FPMP, LOWF, SCF as :1 or :0
Stepper .105	BPWR, IAS (knots, one decimal), VSI (ft/min), AGL (feet, power-gated), FUEL (US gal), TQ (%), ITT (degrees C), N1 (%)
Dual Stepper .106	PWR (first: 1, or 0 while the main bus is below 24 V), BPWR, ALT, RPMR, RPME (one decimal), ZULU, OILT, OILP, XMSNT, XMSNP, FUELLOAD (psi, from fuel pressure), ELECTRICALLOAD

Board	Packet contents
	(%), TEMP
Radio .101	C1A, C1S, C2A, C2S, MAINBUS, NAVCOM1 (1 when the avionics bus, offset 0x2850, is 24 V or more) and the 19 LED_xxx tokens for the Auto Pilot and Audio panel LEDs (all 0 while NAVCOM1 is 0). The offsets above are standard FSUIPC offsets; the DodoSim aircraft may not drive all of them

11.4 Aircraft information for the GPS interface

Every 500 ms SendGpsPacket() sends D,LAT:..,LON:..,ALT:..,IAS:..,HDG:..,UTC:yyyyMMddHHmmss to the Lowrance GPS interface (127.0.0.1:27002; change GpsHost and GpsPort in Form1.cs if it runs on another PC). Latitude and longitude are in degrees, altitude in feet, airspeed in knots (the value already read from 0x02BC) and heading in degrees true. UTC is the simulator's UTC date and time from the FSUIPC client library's FSUIPCConnection.UTCDateTime; if that read fails it is left out and the GPS interface uses the PC clock. Send errors, for example the GPS interface not running, are ignored. Built clean; **not yet run against Prepar3D**, so the offset conversions are unchecked against the simulator.

11.5 Commands from the Radio controller (UDP 27001)

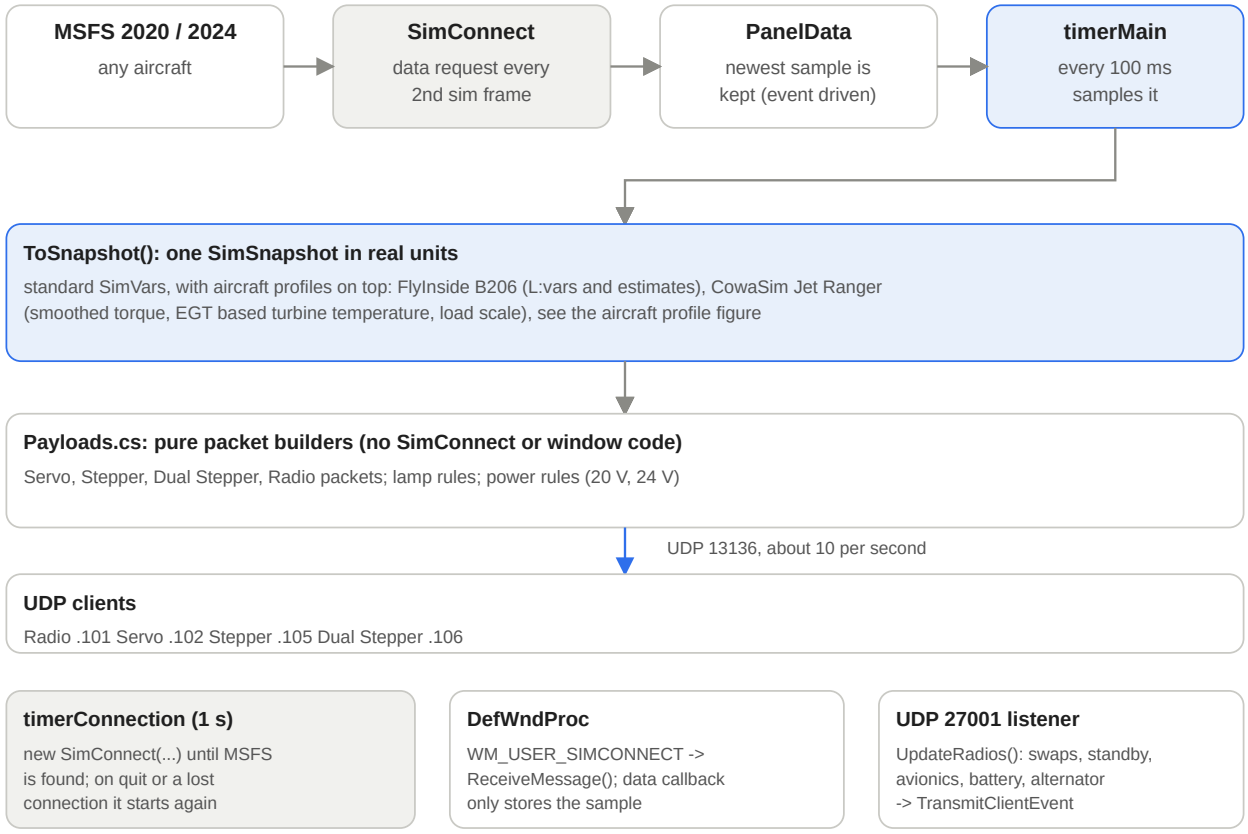
Command	Action
COM1_RADIO_SWAP, COM2_RADIO_SWAP	FSUIPC controls COM_STBY_RADIO_SWAP and COM2_RADIO_SWAP
NAV1_RADIO_SWAP, NAV2_RADIO_SWAP	FSUIPC controls NAV1_RADIO_SWAP and NAV2_RADIO_SWAP
AVIONICS_MASTER_SET	Writes the avionics master offset 0x2E80
MASTER_BATTERY_ON, MASTER_BATTERY_OFF	FSUIPC only has a toggle, so the bridge reads the main bus voltage and toggles only when that would move the switch the requested way
ALTERNATOR_ON, ALTERNATOR_OFF	TOGGLE_MASTER_ALTERNATOR unconditionally (no alternator state is read)
COM2_STANDBY:, NAV1_STANDBY:, NAV2_STANDBY: plus a frequency	Tunes that radio's standby frequency
Auto Pilot, Audio and ADF panel controls: AP_MASTER, AP_HDG_HOLD, AP_NAV1_HOLD, AP_ALT_HOLD, AP_APR_HOLD, TOGGLE_FLIGHT_DIRECTOR, AP_BC_HOLD, AP_PANEL_SPEED_HOLD, VOR1_OBI_INC / _DEC, AP_ALT_VAR_INC / _DEC, AP_VS_VAR_INC / _DEC, AP_SPD_VAR_INC / _DEC, HEADING_BUG_INC / _DEC, COM1_TRANSMIT_SELECT, COM2_TRANSMIT_SELECT, COM_RECEIVE_ALL_TOGGLE, RADIO_VOR1_IDENT_TOGGLE, RADIO_VOR2_IDENT_TOGGLE, RADIO_DME1_IDENT_TOGGLE, RADIO_ADF_IDENT_TOGGLE, MARKER_SOUND_TOGGLE, GEAR_UP, GEAR_DOWN, FLAPS_UP, FLAPS_DOWN, SPOILERS_ON, SPOILERS_OFF, PARKING_BRAKES	The FSUIPC control of the same name, sent with parameter 0 (TryPanelCommand(), checked first so none of these can fall through to the COM1 parser). A :N suffix repeats it N times (1 to 50) for a knob turned quickly
A bare number from 118.00 to 136.975	COM1 standby: BCD-encoded and sent with COM_STBY_RADIO_SET. Any other unrecognised string ends up in this parser too

11.6 Notes and quirks

- Several unused annunciator bits (SPARE_*, SIMCONNECT_FAIL, LOW_INLET_PRESSURE) all share the short code SCF, so the last one in the packet wins for that lamp.
- TURB_ENG_ITT_1 is assigned twice per tick (percent, then temperature), so the temperature wins.
- Run only one bridge at a time; the MSFS bridge also listens on UDP 27001.
- Built clean and run against Prepar3D v6.1 sending about 10 packets per second per board.

12 MSFS bridge (MSFSWinformsAutoCS)

`MSFSWinformsAutoCS` is the bridge for Microsoft Flight Simulator 2020 and 2024. It has the same window, the same timers, the same UDP endpoints and the same real-unit packet per board as the FSUIPC bridge, but reads the data through **SimConnect**. For aircraft whose cockpit dials do not read the standard simulator variables it applies estimates fitted to dial readings (a FlyInside B206 profile and a CowaSim Jet Ranger profile).



No new data for 3 s (paused or loading): the bridge stops sending and the boards' 30 s watchdogs zero the gauges.

Run only one bridge at a time: FSUIPCWinformsAutoCS and MSFSWinformsAutoCS both listen on UDP 27001.

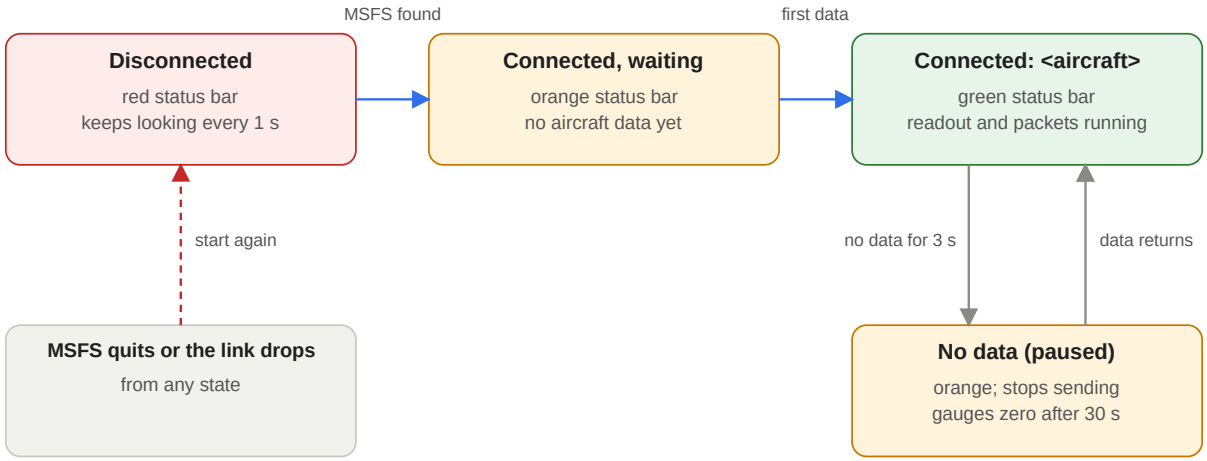
Figure 22 — MSFS bridge data flow

12.1 Files

File	Role
`Form1.cs`	SimConnect connection, 100 ms main timer, on-screen readout, lamp list, UDP listener and command handler
`Payloads.cs`	`SimSnapshot` (one sample of real-unit values) and `Payloads` (packet builders, lamp rules, power thresholds); no SimConnect or window code, so it can be tested on its own
`Form1.Designer.cs`, `Program.cs`	Window layout (identical to the FSUIPC bridge) and entry point
`lib\Microsoft.FlightSimulator.SimConnect.dll`, `lib\SimConnect.dll`	Managed wrapper and native library copied from the MSFS 2024 SDK so the project builds without the SDK path
`MSFSWinformsAutoCS.csproj`	`net10.0-windows10.0.19041.0`, WinForms

12.2 Program flow

1. **Constructor:** connect the UDP clients (port 13136), build the lamp list, start the 1 second `timerConnection` and the UDP listener on 27001.
2. **`timerConnection_Tick`:** try `new SimConnect("MSFS Winforms Bridge", Handle, WM_USER_SIMCONNECT, ...)`. While MSFS is not running this throws and the timer tries again. On success it registers the data definition, requests it every second sim frame, maps the client events, starts `timerMain` and stops itself.
3. **`DefWndProc`** passes `WM_USER_SIMCONNECT` to `ReceiveMessage()`; the data callback only stores the newest sample. If MSFS quits or the connection throws, the app closes SimConnect, stops `timerMain` and goes back to step 2.
4. **`timerMain_Tick`** (100 ms) converts the newest sample to a `SimSnapshot`, updates the readout, lamp list and Avionics Master box, then sends the packets: Radio when a frequency, the main bus voltage or the radio power flag changed (at most every 200 ms) or every 5 seconds; one packet each to the Servo, Stepper and Dual Stepper boards about every 100 ms.
5. **`UpdateRadios`** (UDP 27001) handles the Radio controller's commands as SimConnect client events.



A new aircraft is picked up while connected: the name in the status bar changes and the estimate models restart.

SimConnect errors are caught on every path, so a closed simulator never stops the bridge. FSUIPCWinformsAutoCS behaves the same way with FSUIPC.

Figure 23 — Connection states

If no new data has arrived for 3 seconds (paused or loading) the bridge stops sending, so the boards' 30 second watchdogs return the gauges to zero, and the status bar says so. The status bar also shows the aircraft title and follows it when a different aircraft is loaded.

12.3 Packets

Board	Packet
Servo .102	`BPWR` (1 while the main bus is 20 V or more), `PITCH` (degrees, one decimal, negated: the simulator reports nose-down positive), `BANK`, and the 15 lamp codes (all `0` while the main bus is below 20 V)
Stepper .105	`BPWR`, `IAS` (knots, one decimal), `VSI`, `AGL` (feet, 0 below 20 V), `FUEL` (US gal), `TQ`, `ITT` (degrees C), `N1`
Dual Stepper .106	`PWR` (1, or 0 while the main bus is below 24 V), `BPWR`, `ALT`, `RPMR`, `RPME`, `ZULU`, `OILT`, `OILP`, `XMSNT`, `XMSNP`, `FUELLOAD` (fuel pressure, psi), `ELECTRICALLOAD` (percent, below), `TEMP`
Radio .101	`C1A`, `C1S`, `C2A`, `C2S`, `MAINBUS`, `NAVCOM1` and the 19 `LED_xxx` tokens (see above) (`NAVCOM1` is 1 when the avionics bus is 24 V or more; if the aircraft reports the avionics bus as exactly 0, as the FlyInside B206 does, the main bus is used instead)

`ELECTRICALLOAD` is the magnitude of `ELECTRICAL TOTAL LOAD AMPS` divided by 300 A (the sign goes negative while the battery supplies the bus, for example with the engine stopped); the CowaSim profile uses 139 A.

12.4 Aircraft information for the GPS interface

Every 500 ms the main timer sends `D,LAT:..,LON:..,ALT:..,IAS:..,HDG:..,UTC:yyyyMMddHHmmss.fff` (`Payloads.Gps()`) to the Lowrance GPS interface (`127.0.0.1:27002`; change `GpsHost` and `GpsPort` in `Form1.cs` if it runs on another PC). Three SimVars were added at the end of the data definition and the `PanelData` struct, in the same order: `PLANE LATITUDE`, `PLANE LONGITUDE` (degrees) and `PLANE HEADING DEGREES TRUE`, plus `ZULU YEAR`, `ZULU MONTH OF YEAR` and `ZULU DAY OF MONTH`. Altitude (`PLANE ALTITUDE`, feet MSL) and airspeed (`AIRSPEED INDICATED`) were already read. The simulator's UTC time is built from the already-read `ZULU TIME` (seconds since midnight) and the three date SimVars, and is left out if the date is invalid. The readout shows the position and heading. Nothing is sent while the data is stale, so the GPS interface falls back to Brisbane Airport after 30 seconds. The packet builder was checked with sample values and read back by the GPS interface's parser; **not yet run against MSFS**.

12.5 SimVars requested

`PLANE ALTITUDE`, `AIRSPEED INDICATED`, `VERTICAL SPEED`, `PLANE ALT ABOVE GROUND`, `ATTITUDE INDICATOR PITCH` and `BANK DEGREES`, `ROTOR RPM PCT:1`, `GENERAL ENG PCT MAX RPM:1`, `ENG TORQUE PERCENT:1`, `TURB ENG MAX TORQUE PERCENT:1`, `ENG EXHAUST GAS TEMPERATURE:1`, `GENERAL ENG THROTTLE LEVER POSITION:1`, `ELECTRICAL TOTAL LOAD AMPS`, `TURB ENG ITT:1`, `ENG OIL TEMPERATURE:1`, `FUEL TOTAL QUANTITY`, `TURB ENG CORRECTED N1:1`, `ENG OIL PRESSURE:1`, `ENG TRANSMISSION PRESSURE:1`, `ENG TRANSMISSION TEMPERATURE:1`, `GENERAL ENG FUEL PRESSURE:1`, `ELECTRICAL MAIN BUS VOLTAGE`, `ELECTRICAL AVIONICS BUS VOLTAGE`, `AVIONICS MASTER SWITCH`, `AMBIENT TEMPERATURE`, `KOHLSMAN SETTING MB` (shown in the readout only), `ZULU TIME`, the COM 1, COM 2, NAV 1 and NAV 2 active and standby frequencies, the two FlyInside L:vars and the six GPS SimVars above. `PanelVars` and the `PanelData` struct must list them in the same order; `CheckDefinitionMatchesStruct()` checks the count at connect.

12.6 Autopilot and audio panel LEDs

The 19 LEDs on the Auto Pilot and Audio panels are read through **two extra SimConnect requests** (`PanelAp` and `PanelAudio`, every 6th frame), separate from the main gauge data, so a SimVar the simulator does not know can only take these LEDs down, not the gauges. `PanelAp`: `AUTOPILOT MASTER`, `AUTOPILOT NAV1 LOCK`, `AUTOPILOT HEADING LOCK`, `AUTOPILOT ALTITUDE LOCK`, `AUTOPILOT AIRSPEED HOLD`, `AUTOPILOT APPROACH HOLD`, `AUTOPILOT BACKCOURSE HOLD`, `AUTOPILOT FLIGHT DIRECTOR ACTIVE`. `PanelAudio`: `COM TRANSMIT:1` and `:2`, `COM RECEIVE ALL`, `NAV SOUND:1` and `:2`, `DME SOUND`, `ADF SOUND`, `MARKER SOUND`, `OUTER MARKER`, `MIDDLE MARKER`, `INNER MARKER`. They become the `LED_xxx` tokens of the radio packet (`Payloads.LedTokens()`, all 0 while the radio bus is below 24 V). `ADF SOUND` could not be confirmed to exist in MSFS; if it does not, the audio LEDs stay off and the autopilot LEDs are unaffected.

12.7 Commands from the panel (UDP 27001)

The Auto Pilot, Audio and ADF panel controls (`AP_MASTER`, `AP_HDG_HOLD`, `HEADING_BUG_INC`, `COM1_TRANSMIT_SELECT`, `GEAR_DOWN`, `PARKING_BRAKES` and the others listed in the FSUIPC bridge chapter) are mapped to client events of the same name when SimConnect opens (`PanelEventNames`, event numbers from 1000) and sent by `TryPanelCommand()`, which is checked first; a `:N` suffix repeats the event N times.

`COM1_RADIO_SWAP`, `COM2_RADIO_SWAP`, `NAV1_RADIO_SWAP`, `NAV2_RADIO_SWAP`; `COM2_STANDBY:`, `NAV1_STANDBY:`, `NAV2_STANDBY:` plus a range-checked frequency (sent as `COM2_STBY_RADIO_SET_HZ`, `NAV1_STBY_SET_HZ`, `NAV2_STBY_SET_HZ`); a bare number is the COM1 standby frequency (an out-of-range or unreadable value becomes 121.50); `AVIONICS_MASTER_SET`; `MASTER_BATTERY_ON` and `_OFF` (`MASTER_BATTERY_SET` 1 or 0); `ALTERNATOR_ON` and `_OFF` (`ALTERNATOR_SET`). Unlike FSUIPC, SimConnect has direct set events, so there is no voltage guard or blind toggle.

12.8 Aircraft profiles

Value	Standard SimVars	FlyInside B206	CowaSim Jet Ranger
Rotor, engine speed	**ROTOR RPM PCT,** GENERAL ENG PCT MAX RPM	**L:Aircraft.Rotor.RPM.Percent,** L:...Turbine.N2.Smooth	**standard**
Gas producer (N1)	**TURB ENG CORRECTED N1**	**0.92 x turbine N2**	**standard**
Torque (TQ)	**ENG TORQUE PERCENT**	**collective lever curve,** up to 120 %	**0.42 x SimVar,** smoothed 2.5 s
Turbine out temp (ITT)	**TURB ENG ITT**	**6 x N2 + 1.05 x** (torque - 21)	**EGT - 200 C**
Fuel load (psi)	**ENG FUEL PRESSURE**	**10 x SimVar**	**standard**
Electrical load (%)	**|total amps| / 300 A**	**|total amps| / 300 A**	**|total amps| / 139 A**
Transmission pressure	**SimVar (usually 0)**	**35 psi x rotor fraction**	**48 psi x rotor fraction**
Transmission temp	**SimVar (usually 0)**	**slow warm-up from air temp**	**0.92 x engine oil temp**
Recognised by	**otherwise**	**turbine L:var, or title** contains FlyInside	**title contains 206B3**

Green = the standard SimVar is used unchanged. Estimates come from a few dial readings and may be a few percent off.

Figure 24 — Which values come from where

Other aircraft use the standard SimVars unchanged. Estimates were fitted to a few dial readings taken from screenshots and may be a few percent off at power settings that were not checked.

12.8.1 FlyInside B206

The FlyInside B206 does not feed the standard rotor and engine speed SimVars (they read 0 and a slowly decaying idle while the cockpit shows 100 percent). Found with Developer Mode, Tools, Behaviors, Variables (Global tab, filter `Aircraft.`): the cockpit uses `L:Aircraft.Rotor.RPM.Percent` and `L:Aircraft.Engine.1.Turbine.N2.Smooth`. The aircraft is recognised by the turbine L:var being above zero or by `FlyInside` in the aircraft title. The bridge uses the L:vars when above zero, otherwise the standard SimVars. They feed `RPMR`, `RPME` and the `RLOW` and `EOUT` lamps.

Value	Estimate
Gas producer (`N1`)	0.92 x turbine N2 (no variable found; the standard N1 reads about 52 while the cockpit shows 92)
Torque (`TQ`)	`ENG TORQUE PERCENT:1` reads 0. With the turbine at 95 or more: 21 + 0.72 x collective lever up to lever 73.6 (dial 74), then about 1.74 per lever percent to 120 at lever 100, clamped 0 to 120. Below that speed it is estimated from `TURB ENG MAX TORQUE PERCENT:1` through two readings
Turbine out temperature (`ITT`)	6 x turbine N2 + 1.05 per torque percent above the idle torque of 21 (the standard value reads about 540 while the dial shows 600)
Fuel load (`FUELLOAD`)	10 x `GENERAL ENG FUEL PRESSURE:1` (the cockpit dial is a 0 to 3 scale marked x10)

Value	Estimate
Transmission pressure (XMSNP)	35 psi x the rotor speed fraction
Transmission temperature (XMSNT)	Starts at outside air temperature and moves slowly (5 minute warm-up, 10 minute cool-down) toward ambient + 0.8 per torque percent while the rotor is above 50 percent; restarts from ambient when the bridge restarts



Both curves were fitted to dial readings from screenshots; check them against the cockpit at other power settings.

FlyInside above lever 73.6 climbs at about 1.74 points per percent; the engine over-torques if full collective is held for 2 s.

Figure 25 — Torque curves

Holding full collective for more than about 2 seconds over-torques and breaks the engine in this aircraft (the dial swings to about 115 percent). The lever mapping depends on the controller; only two steady readings back the 0.72 slope.

12.8.2 CowaSim Jet Ranger

The CowaSim Jet Ranger (aircraft title `206B3 ...`, for example `206B3 C-GTOK`) reports the standard SimVars and most dials match them. Gas producer, engine oil, fuel pressure and rotor and engine speed need no change. Three dials do not read the standard values, and it has no transmission oil values:

- **Torque:** the dial is a smoothed copy of `ENG TORQUE PERCENT:1`, about 0.42 x the SimVar (dial about 38 at the steady idle value of 83, about 120 at a peak of about 260) with a smoothing time constant of about 2.5 seconds. Error is about 18 points on a signal swinging between 0 and 120.
- **Turbine out temperature:** the dial follows `ENG EXHAUST GAS TEMPERATURE:1`, not ITT (which reads over 2,000): EGT minus 200 C, within about 30 C over nine readings.
- **Electrical load:** the LOAD dial sat at about 50 percent while the total load was 69.5 A, so 139 A is 100 percent (one reading).
- **Transmission oil:** pressure is 48 psi x the rotor speed fraction (the dial read about 45 to 50 psi); temperature is 0.92 x the engine oil temperature (one reading, which did not follow a slow warm-up).

12.9 Warning lamps

Five lamps are driven; the other ten are aircraft specific (DodoSim supplies them through an FSUIPC offset, which MSFS aircraft do not have) and stay off. Add rules to `Payloads.LampState()` once an aircraft's variables are known.

Lamp	Rule
EOUT engine out	Turbine speed below 60 percent
RLOW rotor low	Rotor at or below 95 percent
LOWF low fuel	17 US gal or less
TOPW transmission oil pressure	Below 40 psi, only when a pressure is available (the FlyInside and CowaSim

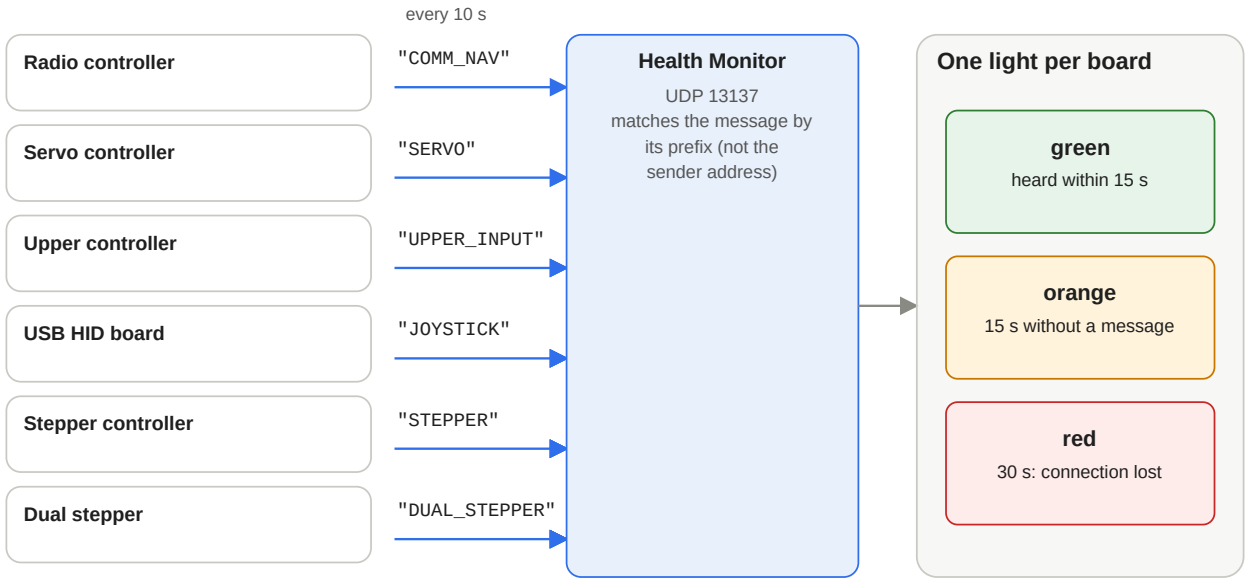
Lamp	Rule
	estimates, or an aircraft whose SimVar reads above 0)
GENF generator fail	Engine speed below 10 percent

12.10 Known gaps

- The FlyInside and CowaSim estimates come from a few readings and have not been checked at other power settings.
- Some MSFS Jet Ranger add-ons do not report every SimVar (transmission pressure and temperature read 0 on two of them; one reports a fixed rotor speed). A missing avionics bus voltage (exactly 0) falls back to the main bus for the Radio power rule.
- Pitch sign and the 24 V thresholds are carried over from the FSUIPC bridge and are not verified for every MSFS aircraft.
- There is no hysteresis on the 20 V and 24 V thresholds.
- The FlyInside electrical load dial stays at about 50 percent whatever the load.
- The FlyInside transmission pressure estimate is 35 psi at normal rotor speed, which is below the 40 psi lamp threshold, so the TOPW lamp stays on with that aircraft.

13 Health Monitor (JetRangerHealthMonitor)

A small Windows Forms dashboard (namespace `MegaHealthMonitor`) that watches for the 10 second heartbeat strings sent by the networked Arduino boards and shows one traffic light per board: Comm/Nav, Servo, Joystick, Upper Input, Stepper and Dual Stepper.



The monitor only listens; it never sends anything. It does not care which bridge is running.

Figure 26 — Heartbeats in, one light per board out

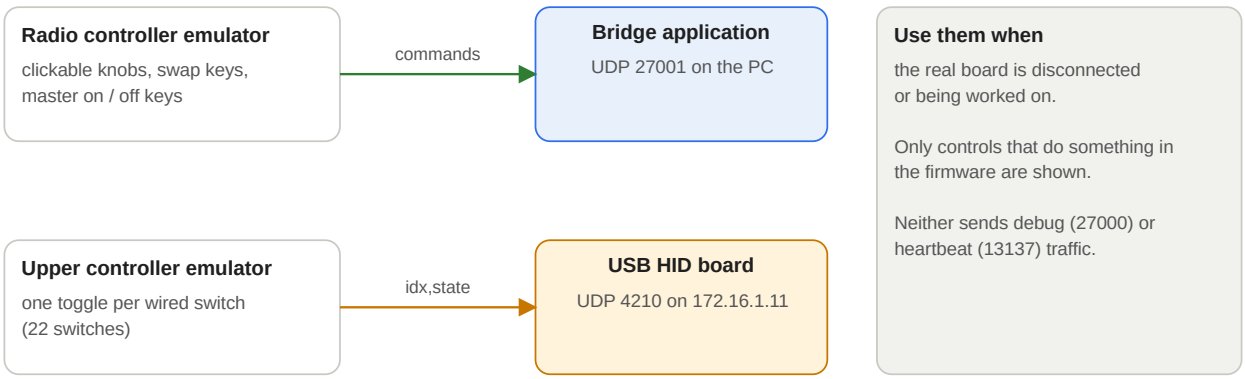
13.1 Program flow

1. `Form1_Load` starts a background task that binds a `UdpClient` to port **13137** and loops on `ReceiveAsync()`.
2. Each packet is matched by **prefix** (not by sender address): `COMM_NAV`, `SERVO`, `JOYSTICK`, `UPPER_INPUT`, `STEPPER` or `DUAL_STEPPER`. A match updates that board's last-received time and turns its label green; each match can also be logged (sender and timestamp) to a list box.
3. A timer calls `CheckHealth()` about once a second: 30 seconds or more since the last packet turns the indicator **red** (connection lost), 15 seconds or more **orange** (connection warning), otherwise it stays green.
4. *Toggle logs* shows or hides the log list box and resizes the form.

The monitor is a passive listener: it never sends anything and does not care which bridge is running. The Dual Stepper's prefix is deliberately distinct from `STEPPER` (the monitor uses `StartsWith`, so overlapping prefixes would light the wrong indicator). The ADF controller's `ADF_PANEL` heartbeat has no light yet.

14 Radio and Upper controller emulators

Two small Windows Forms panels stand in for the physical boards when they are not connected. Each shows only the controls that actually do something in the firmware.



The Radio emulator still sends a bare frequency for every radio, so only COM1 tunes from it.

(The real board sends labelled COM2_STANDBY:, NAV1_STANDBY: and NAV2_STANDBY: values, which both bridges understand.)

Figure 27 — The emulators and where they send

14.1 JetRangerRadioControllerEmulator

Shows the two character-OLED displays (COM and NAV), a clickable knob for each of the eight encoders (click the right half to turn one detent clockwise, the left half for counter-clockwise), the COM1 and COM2 *XFER* swap keys and the *MASTER ON* and *MASTER OFF* keys. It sends the same payloads the board would send to the bridge on UDP 27001 (default host `172.16.1.10`):

- each knob sends the new standby frequency as a bare ASCII number such as `121.010`;
- the XFER keys swap the local active and standby values and send `COM1_RADIO_SWAP`, or `COM2_RADIO_SWAP` followed by `AVIONICS_MASTER_SET`;
- the MASTER keys send `MASTER_BATTERY_ON` with `ALTERNATOR_ON`, or the two `_OFF` commands.

Limits: the emulator still sends a bare frequency for every radio, so only COM1 tunes correctly from it (the real board sends `COM2_STANDBY:`, `NAV1_STANDBY:` and `NAV2_STANDBY:`). It has no NAV swap keys and sends no debug (27000) or heartbeat (13137) traffic. Its knob model is 5 kHz per minor detent, not the 25 kHz and 50 kHz channel spacing the real board now uses.

14.2 JetRangerUpperControllerEmulator

Shows one toggle button per switch the Upper firmware acts on (22 switches in groups: Cabin Heat, Air, Circuit Breakers, Instruments, Electrical, Lights; green means on) and sends `index,state` packets to the USB HID board (default `172.16.1.11:4210`) exactly as the real board would. Indexes 7 to 10, 18 to 21, 29 to 38 and 40 to 175 are wired in the matrix but drive nothing and are not shown.

15 StepperVSITester

A Windows Forms bench tool that sends real-unit values or raw step counts straight to the boards' UDP handlers without a simulator. It started as a vertical speed tester (hence the name) and now covers the whole Stepper and Dual Stepper boards plus the servo pitch and bank.

15.1 What it sends

Target	Controls
Stepper .105	`VSI`, `AGL`, `ITT`, `IAS`, `TQ`, `N1`, `FUEL` (real values, with sliders for VSI, AGL, ITT, IAS); `OILT`, `OILP`, `XMSNT`, `XMSNP` (also broadcast to .106)
Dual Stepper .106	`ALT`, `RPME`, `RPMR` (broadcast to both stepper boards), `FUELLOAD`, `ELECTRICALLOAD`, `ZULU` (hour and minute boxes), `TEMP` (text)
Servo .102	`PITCH`, `BANK` (degrees, one decimal) and `PITCHRAW`, `BANKRAW`
Raw step panel	A drop-down of raw codes (`IASRAW`, `VSIRAW`, `ALTRAW`, `OILTRAW`, `OILPRAW`, `XMSNTRAW`, `XMSNPRAW`, `ITTRAW`, `RPMERAW`, `RPMRRAW`, `N1RAW`, `FUELRAW`, `TQRAW`, `FUELLOADRAW`, `ELECTRICALLOADRAW`) sent to both stepper boards, with -1 step and +1 step buttons

Each row has a slider or text box, *Send* (or Enter) and *Zero*. Values are sent as typed, with no conversion; `RPME` and `RPMR` carry one decimal. Raw codes that a board does not have are silent no-ops there (for example `FUELLOADRAW` on the Stepper board).

Notes: the raw panel still lists `FLAPS`, `AOA`, `GFORCE`, `SPDMAX` and `N1RAW`; the first four no longer exist on any board, and `N1RAW` does nothing on the Dual Stepper. `RPME` and `RPMR` are no longer handled by the Stepper board, so only the Dual Stepper acts on them. Run it **instead of** a bridge, since both would otherwise fight over what drives the gauges.

16 SwingTester

A Windows Forms (.NET 10) bench tool that triggers each gauge's start-up swing over UDP, one gauge at a time or all of them in a fixed order. It also lights the warning lamps and, in a **Panel LEDs** group, each of the 19 Auto Pilot and Audio panel LEDs on its own (a toggle per LED sending `D,NAVCOM1:1,LED_xxx:1/0` to the Radio controller), all on or off, in a chase (one at a time, with the on gap), or with the Radio's own `C,LEDTEST` board test. Build with `dotnet build -c Release` in its folder; the executable is `bin\Release\net10.0-windows\SwingTester.exe`.

16.1 How it works

It sends a control packet `C,<COMMAND>:1` to `<board IP>:13136`. The boards do not report completion, so after each command the app waits a per-row number of seconds (editable in the window, not saved between runs) before sending the next. A swing blocks the board's loop; too short a wait lets the next command queue in the board's UDP buffer and the order across different boards drifts.



Figure 28 — Swing ALL sequence

The default waits are the swing times measured on the real boards plus about one second of margin: Dual Stepper 5.0 to 5.2 s (oil and transmission gauges, altimeter) and 6.4 to 6.5 s (load and RPM gauges); Stepper 9.5 to 10.7 s; Servo about 2.1 s. A full pass takes about 220 seconds.

16.2 Window

- The three board IPs at the top (defaults `172.16.1.105`, `.106`, `.102`).
- **Swing ALL (in order)**, **Stop** (cancels the remaining waits; a swing already started still finishes), **OLEDs ON** and **OLEDs OFF**, and the check box *Light OLEDs during Swing ALL* (on by default).
- One row per gauge (18 rows on the left) with its board, a seconds box and a **Swing** button; the running row is highlighted gold. The 15 warning-lamp buttons are on the right.
- **OLEDs ON** sends `C,OLEDTEST:1` to the Dual Stepper and **OLEDs OFF** sends `C,OLEDTEST:0`. The **Temp OLED** box (default `21C`) with **Send TEMP** sends `D,TEMP:<text>`.

- **Reset boards...** opens a dialog with a tick box per board (Radio .101, Servo .102, Upper .103, Stepper .105, Dual Stepper .106, USB HID .11; all ticked by default) and sends `C,RESET:1` to each ticked board (the HID board on port 4210). The Dual Stepper takes about a minute to restart and the HID board briefly drops its USB devices. The button is disabled while a run is active.

16.3 Brightness

A **Brightness (0-15)** slider sends `C,B:<level>` to the Radio (.101), Servo (.102), Upper (.103), Stepper (.105) and Dual Stepper (.106), every board except the USB HID board, each time it is moved; **Send to all boards** resends the current level, and the status line lists the boards sent to. The Servo, Stepper and Dual Stepper set their backlight from it; the Radio parses it but does nothing and the Upper ignores it. **Backlight power ON** and **OFF** send `D,BPWR:1` and `D,BPWR:0` to the Servo, Stepper and Dual Stepper, the packet the bridges send from the main bus voltage, to test the main bus rule from the bench: OFF switches the backlight off and ON returns it to the last brightness commanded.

16.4 Warning lamps

The Servo board's lamps are driven by ordinary `D,<code>:1` and `D,<code>:0` packets to `172.16.1.102:13136`. There is a toggle per lamp (gold when on), **Lamps ON (in order)** (one lamp at a time, *on gap* default 1000 ms), **Lamps OFF (in order)** (*off gap* default 400 ms) and the check box *Light warning lamps during Swing ALL* (on by default). Turn-off order, also the button order: `SCF`, `GENF`, `FFLTR`, `BD`, `EC`, `BHOT`, `TOPW`, `EOUT`, `LOWF`, `FPMP`, `TRC`, `TC`, `BTMP`, `TOTW`, `RLOW`.

Swing ALL with the check box on first lights the 15 lamps one at a time (about 15 seconds), then runs the swings, then turns the lamps off one at a time (always at the end, also after Stop or an error). The Servo board only drives a lamp when the incoming value differs from the state it last remembered, so keep the bridge off while testing.

16.5 Keep-alive

Every sketch resets its gauges after 30 seconds with no packet. During a full run some boards get no command for 100 seconds or more, so while any run is active (Swing ALL, an individual swing or the lamp sequence) the app sends `C,KEEPALIVE:1` to all three boards every 10 seconds (and once at the start). The boards ignore the command name but count the packet as received data. The keep-alive stops when the run ends, so lamps or OLEDs left on manually still time out after 30 seconds.

17 Debug listener (UDP reflector)

Every board sends debug text to `172.16.1.10:27000`. A small Python reflector on the primary PC prints each line and can forward a copy to a second listener, so an analysis script can read the same stream. Only one program can own the port, so use the reflector's forwarding rather than starting a second listener. Keep the reflector running while boards are being flashed and restarted: the start-up lines (Ethernet Started, I2C scan, Setup Complete) are the quickest check that a flash worked.

Other helpers in `Jet Ranger Python Code/`: `Export_From_SimConnect.py` (a SimConnect exporter), `Test_Receiver.py` (a generic UDP JSON receiver) and `send_wol.py` (wake-on-LAN).

18 Lowrance AirMap GPS interface (LowranceAirMapGps)

`LowranceAirMapGps` makes the PC act as the GPS for a **Lowrance AirMap 2000C**. It is a Windows Forms program (.NET 10) that receives aircraft information over UDP and sends NMEA 0183 sentences with the position, altitude, airspeed, heading and time over a serial port, twice a second.

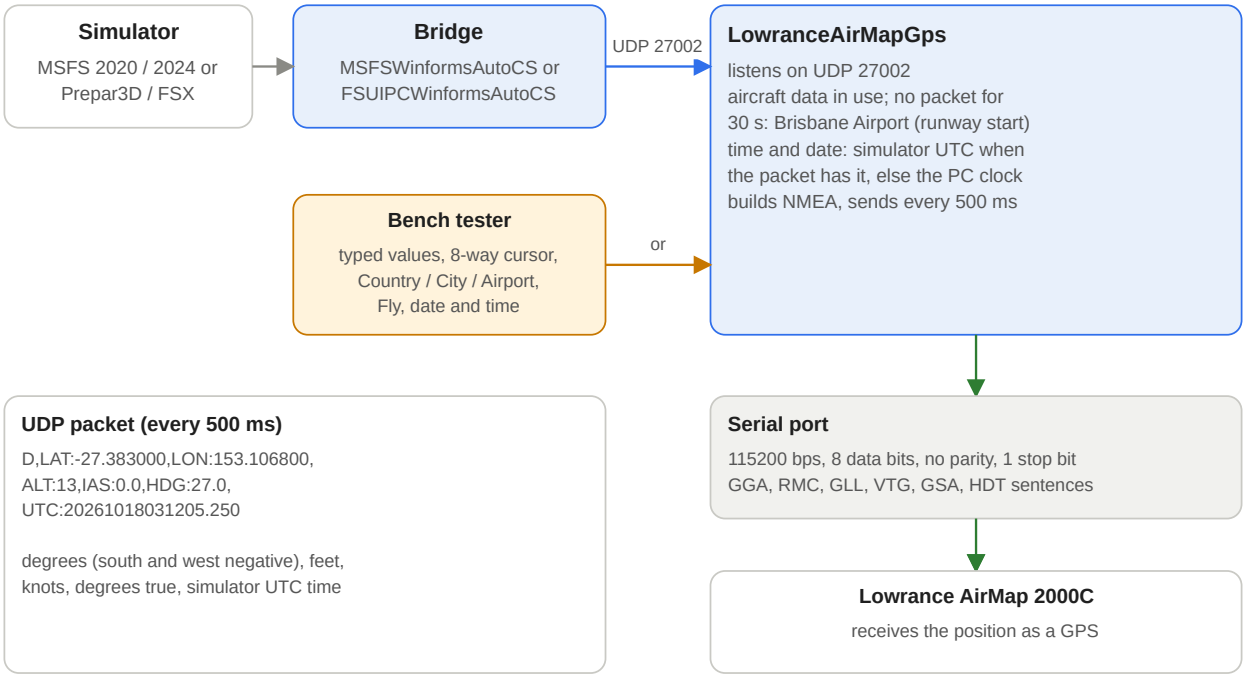


Figure 29 — From the simulator to the GPS

18.1 Behaviour

1. **Serial port:** 115200 bps, 8 data bits, no parity, 1 stop bit, no handshake. The chosen port is saved in `%AppData%\LowranceAirMapGps\settings.json`. If none is saved (or the saved port no longer exists) a dialog asks for one at start-up, and *Change serial port...* reopens it. If the port cannot be opened or a write fails the program retries every 3 seconds, so a USB serial adapter can be plugged in later.
2. **UDP input:** UDP port **27002** (`UdpPort` in the settings file). Two layouts are accepted: key/value `D,LAT:-33.9461,LON:151.1772,ALT:1500,IAS:110,HDG:90,UTC:20261018031205.250` (any subset of keys; missing keys keep the previous value) and CSV `lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc]`. Units are degrees (south and west negative), feet, knots and degrees true. Packets that are not aircraft data or are out of range are ignored and do not reset the 30 second timer.
3. **Default position:** with no accepted packet for 30 seconds, or since start-up, the program sends Brisbane Airport: -27.3830, 153.1068, 13 ft, speed 0, heading 027 degrees true, the threshold of runway 01L (016 magnetic). The values can be changed in the settings file (`DefaultLatitude`, `DefaultLongitude`, `DefaultAltitudeFt`, `DefaultHeadingDeg`). They come from SkyVector and were not checked against the AIP.
4. **Output rate:** one set of sentences every 500 ms.

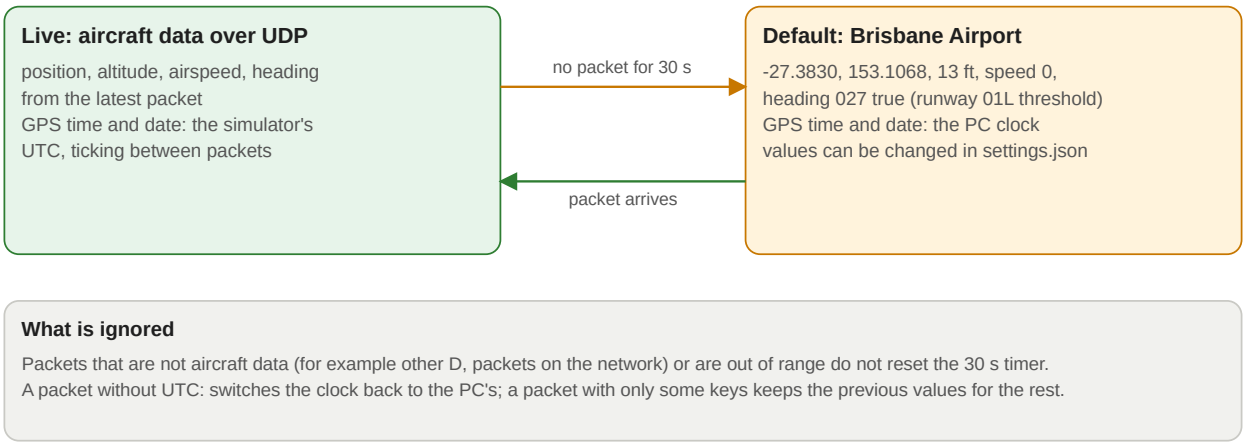


Figure 30 — Live data and the Brisbane default

18.2 NMEA sentences

Each update sends `$GPGGA` (position, altitude in metres), `$GPRMC` (position, speed in knots, track, date), `$GPGLL`, `$GPVTG` (track, speed in knots and km/h), `$GPGSA` (a fixed 3D fix with 8 satellites) and `$GPHDT` (heading), each with its `*CS` checksum and CR LF. NMEA has no airspeed sentence a GPS accepts, so the airspeed is sent as the GPS speed in RMC and VTG, and the heading as the track and as HDT.

18.3 GPS date and time

When a packet carries `UTC:` the program takes the offset between the simulator's UTC and the PC's UTC and adds it to the PC clock for every sentence, so the GPS time and date (the RMC date field and every sentence time) are the simulator's and tick smoothly between packets. The offset is only re-taken when the simulator time moves more than 1.5 seconds from where it predicts, for example when the user changes the time or the simulation rate is not 1x. The window shows *GPS time ... (simulator time)* or *(PC clock)*. The PC clock is used when no packet has arrived for 30 seconds or when a packet has no `UTC:`.

18.4 Files

File	Role
`MainForm.cs`	Window, serial port handling, UDP listener task, 500 ms timer and the 30 second fallback
`NmeaBuilder.cs`	Builds the sentences and checksums
`AircraftData.cs`	The `AircraftData` record and the UDP packet parser
`AppSettings.cs`	JSON settings: serial port, UDP port, default position
`SerialPortDialog.cs`	The serial port picker

18.5 Status

Built clean. The NMEA builder (checksums checked against two reference sentences) and the parser passed a scratch test program, and the running window was exercised with UDP packets from the bench tester: it switched to the packet position and time, and went back to Brisbane Airport 30 seconds after the last packet. **Not yet tested against a real GPS**, so the AirMap 2000C's acceptance of these sentences and of the 115200 bps rate is unverified. Run one sender at a time: a second sender's packets simply overwrite the position.

19 GPS bench tester (LowranceAirMapGpsBenchTester)

A Windows Forms bench tool that sends aircraft information to the GPS interface (default `127.0.0.1:27002`, twice a second) so the GPS can be exercised without a simulator.

19.1 Window

- **Send to UDP** host and port, and a *Send twice a second* check box.
- **Country, City and Airport** drop-down lists (starting on Australia, Brisbane, YBBN). Choosing an airport puts the aircraft on the ground there (the airport's reference point and field elevation, speed 0; the heading is left as it is). The lists hold 10,133 airports with an ICAO code in 236 countries, from an embedded list trimmed from the public-domain OurAirports `airports.csv`. The positions are reference points, not runway thresholds.
- **Latitude, longitude, altitude (ft), speed (kt) and heading (degrees true)** boxes: type a value and press Enter (an unreadable value turns the box pink and is not used). They follow the cursor while it moves.
- **8-way cursor:** a 3 by 3 pad of buttons (N, NE, E, SE, S, SW, W, NW) moves the position by the **Cursor step** (metres, default 100). A press moves one step; holding repeats every 100 ms after 400 ms. The arrow keys do the same, two together giving a diagonal, while the window is active and no box is being edited. With *Point heading in direction of travel* ticked (the default) the cursor also sets the heading.
- **Fly:** while ticked the position advances along the heading at the current speed with altitude, speed and heading constant. Distance is the real time since the last 100 ms tick times the speed (1 kt = 0.514444 m/s), multiplied by **Time x** (1 to 100) so a long leg can be flown quickly. Nothing moves while the speed is 0; change the speed, heading or altitude while flying to change course.
- **Date / time (UTC)** with a *PC clock* button and a *Send date/time* check box (on by default). Type `yyyy-MM-dd HH:mm:ss` and press Enter; the clock then runs in real time from that value and is sent as the simulator time.
- **Back to the selected airport** returns to the selected airport.

Movement uses a great-circle destination-point calculation, so a step or a leg is the same distance on any bearing and stays accurate over long flights.

19.2 Testing

Run together with the GPS interface on one PC. The GPS interface showed the tester's position; a North click moved latitude about 0.0009 degrees (100 m) and set heading 0, holding the Left arrow moved the position west with heading 270, choosing Japan and then Osaka moved it to Kansai (RJBB), Fly at 120 kt, heading 90 and Time x10 moved the longitude about 2.8 km in 4.5 seconds, and with the clock set to 2027-03-05 12:30:00 the GPS interface's RMC sentence carried time 123001 and date 050327 and advanced in real time.

20 PCB notes

Notes from reading the KiCad projects in `Jet Ranger/PCBs` (schematics, PCB layouts and netlists), brought up to date on 18 October 2026. Matching boards to sketches is by name. The checks read the files only; no board was measured.

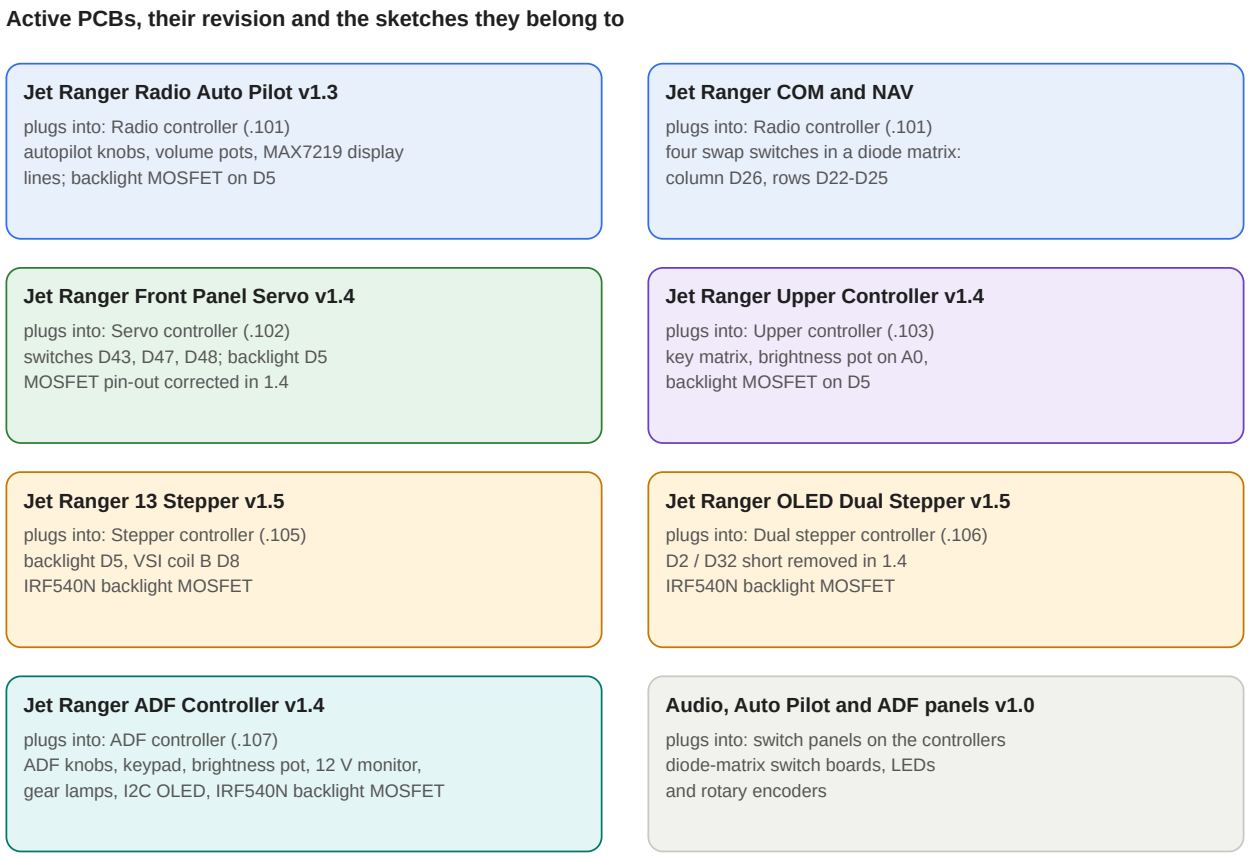


Figure 31 — Active PCBs, their revision and the sketches they belong to

20.1 Boards in use

PCB	Revision (title block)	Used by	Notes
Jet Ranger Front Panel Servo	1.4 (2026-10-18)	Servo controller	Mega shield, two ULN2803A lamp drivers, servo headers (16, 2 used), 15 lamp outputs, three switch inputs, backlight MOSFET. Change log: 1.3 had the MOSFET pinned wrongly (source and drain need swapping), 1.4 corrects it
Jet Ranger 13 Stepper	1.5 (2026-10-16)	Stepper controller	Mega shield, USR-ES1 Ethernet, 14 six-pin gauge sockets, 12 V lamp supply with polyfuses, backlight MOSFET. No change log text; the schematic now uses the IRF540N
Jet Ranger OLED Dual Stepper	1.5 (2026-10-18)	Dual stepper controller	Mega shield, USR-ES1, gauge sockets, TCA9548A sockets for the OLEDs, backlight MOSFET. No

PCB	Revision (title block)	Used by	Notes
			change log text; the schematic now uses the IRF540N
Jet Ranger X27 Small Stepper	not set	Each stepper gauge	One X27 or VID29 stepper, backlight LED, six-pin connector to the controller
Jet Ranger Radio Auto Pilot	1.3 (2026-10-18)	Radio controller	Radio and autopilot controller. Change log: 1.2 added the audio panel, 1.3 corrected the MOSFET mis-pinout
Jet Ranger COM and NAV	not set	Radio panel	Eight rotary encoders, push buttons and the four frequency-swap switches in a diode matrix
Jet Ranger Upper Controller	1.4 (2026-10-18)	Upper controller	Mega shield for the switch matrix, backlight MOSFET, brightness input on A0. Change log: 1.4 corrected the MOSFET mis-pinout
Jet Ranger ADF Controller	1.4 (2026-10-19)	ADF controller (`JET_RANGER_ADF_CONTROLLER`, .107)	Mega shield with the ADF major and minor knobs (D6 to D9), an I2C OLED, a 7-pin keypad (D22 to D28), landing-gear, flaps, speed-brake and brake inputs (D29 to D32), three landing-gear lamps (D2 to D4), the backlight MOSFET, a panel brightness pot (A0) and a 12 V monitor (A15, 47 k / 10 k divider). Change log: 1.2 added the audio panel, 1.3 corrected the MOSFET mis-pinout (IRF540 selected), 1.4 added the 12 V power rail monitors and the pot for panel brightness
Jet Ranger Audio Panel, Auto Pilot Panel, ADF Panel	1.0 (2026-10-17)	Switch panels	Diode-matrix switch boards: Audio (10 switches, 21 diodes), Auto Pilot (16 switch and encoder parts, 18 diodes, 2 LEDs), ADF (9 switch and encoder parts, 4 diodes)
Annunciator, Altimeter, Large Guage, Simkits gauge boards	not set	Other panels and gauges	Not covered by the sketches in this document

The X27 steppers run straight from the Mega's pins, so the stepper boards need no driver chips; the 12 V input feeds lamps and backlights.

20.2 Backlight driver

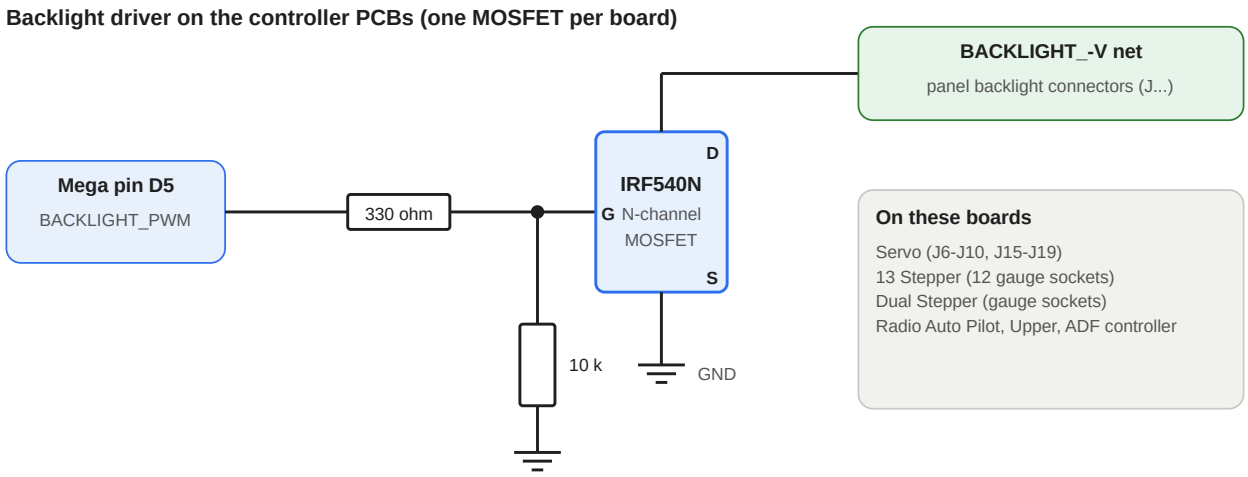


Figure 32 — The backlight MOSFET circuit

All six controller boards (Servo, 13 Stepper, Dual Stepper, Radio Auto Pilot, Upper and ADF Controller) have the same backlight circuit, traced pad to pad in the PCB files: Mega pin **D5** is the net `BACKLIGHT_PWM`, which goes through a 330 ohm resistor to the gate of an **IRF540N** in a TO-220 package, with a 10 k resistor from the gate to ground. The drain is the backlight net (`/BACKLIGHT_PANELS` or `BACKLIGHT_-V`) that the panel or gauge backlight connectors share, and the source is ground, so the MOSFET switches the backlight low side and D5 PWM sets the brightness. `BACKLIGHT_PWM` is also brought out on pin 6 of an 8-pin header on each board.

Board	Backlight connectors on the MOSFET drain net
Front Panel Servo	Ten `BK_LIGHT` connectors (J6 to J10, J15 to J19)
13 Stepper	Pin 6 of 13 gauge sockets (airspeed, radar altitude, gas producer, transmission oil pressure, VSI, engine torque, turbine speed, rotor speed, fuel level, engine oil pressure and temperature, transmission oil temperature, EGT)
Dual Stepper	Gauge sockets for airspeed, VSI, fuel level, EGT, altimeter, engine, transmission, rotor and engine speed, and electrical and fuel load
Radio Auto Pilot	The AUDIO, AUTOPILOT and COMM_NAV connectors
Upper Controller	The BACKLIGHT connector (J5)
ADF Controller	The ADF connector (J5)

The earlier revisions used a generic N-channel MOSFET symbol (`Q_NMOS_GSD`) whose pin-out had source and drain swapped; the revisions dated 18 October 2026 (1.4 Servo, 1.3 Radio Auto Pilot and ADF Controller, 1.4 Upper, 1.5 Stepper and Dual Stepper) use the IRF540N with the correct pin-out. A board made from an earlier revision should have the transistor checked.

The Servo, Stepper, Dual Stepper and ADF sketches drive D5 (see the network and ADF controller chapters). The Radio and Upper sketches do not. The Upper PCB has a brightness input on A0 (`GUAGE_BACKLIGHT_BRIGHT`, connector J16) that no sketch reads; the ADF PCB has the same pot input, and the ADF sketch reads it and sends the level to the other panels.

20.3 Warning Mute, Fuel Valve and TOT Clear inputs

All KiCad schematic, PCB and netlist files under `Jet Ranger/PCBs` were searched for a Warning Mute (horn mute) input. Two boards have one.

- **Jet Ranger Front Panel Servo:** SW3 (`WARNING_MUTE`, a 2-pin Molex KK-254 connector) connects net `/SW_WARNING_MUTE` to Arduino Mega pin D47 and GND. The same board has SW1 `TOT_CLEAR` on D43 and SW2 `FUEL_VALVE` on D48, each also wired to GND. The Servo sketch reads these three pins as

debounced inputs (internal pull-up, closed = LOW), logs every transition and sends it to the USB HID board.

- **Jet Ranger Upper Controller:** J17 (`WARNING MUTE`, a 2-pin Molex KK-254 connector) connects `/ROW8` (Arduino D30, shared with J18 TOT TEST) on pin 2 and, on pin 1, a 1N4148 diode D5 whose other side goes to `/COL0`, a switch in the board's diode-isolated key matrix. The Upper sketch scans a 4 by 3 matrix on analog pins A9 to A15, which does not include D30, so this input may not be read by the current firmware.

20.4 Pin and copper check

The KiCad boards were traced pad to pad through tracks and vias on 12 October 2026 and the Mega pin tables were compared again on 18 October 2026 against that check. This checks connections only; it is not a design rule check (clearances, pours and footprints are not checked).

- **No Mega pin assignment changed** between the two checks on the Servo, 13 Stepper, Dual Stepper and Upper boards, apart from the 13 Stepper D2 net being renamed (below). The Radio Auto Pilot and ADF Controller were first checked on 18 October.
- **Jet Ranger OLED Dual Stepper:** up to version 1.3, D2 and D32 were one net (`STEPPER_FL_A`), joining fuel quantity and fuel load coil A. Fixed in 1.4: D2 is `STEPPER_FA_A` and D32 `STEPPER_FL_A` in 1.5. Boards older than 1.4 still have the short: fit only one of those two gauges or cut the trace.
- **Jet Ranger 13 Stepper:** D2 carried the same copied name but nothing else used it; it is `STEPPER_FA_A` (label only). D32 is the radar altitude coil `STEPPER_RA_A`.
- **Both stepper PCBs:** `BACKLIGHT_PWM` is D5 and VSI coil B is D8. Both stepper sketches use D5 for `BACK_LIGHTS` (pin 8 used to be both the backlight and VSI coil B).
- **Jet Ranger Front Panel Servo:** the lamp outputs are still wired to different Mega pins from the sketch for 10 of the 15 lamps: the PCB has A1 Rotor RPM low, A2 Engine out, A5 Battery temp, A6 Battery hot, A7 Transmission chip, A8 Baggage door, A9 Engine chip, A10 Tail rotor chip, A11 Fuel pump and A12 Aft fuel filter; the sketch has them the other way round for A1/A2, A5/A6 and A11/A12 and in a different order for A7 to A10 (A3, A4 and A13 to A15 agree). The power-up lamp test cannot show this. Test each lamp with a single `D,<lamp>:1` and fix whichever side is wrong.
- **Jet Ranger Radio Auto Pilot:** the COM1 major, COM1 minor, COM2 minor and NAV1 minor encoders have A and B the other way round from the sketch (see the Radio controller chapter); the other four match. The volume pots and mux address pins are wired but not used by the sketch (the autopilot knobs, MAX7219 lines and fifth matrix column are now used).
- **Servo, 13 Stepper and Upper Controller:** a net `GPS_TX` joins Mega pin D18 (TX1) to pin 5 of an 8-pin header (J11 on the Servo and 13 Stepper, J43 on the Upper). No sketch uses Serial1 on those boards. The Dual Stepper uses D16 to D19 for the OLED multiplexer instead.
- **Upper Controller, 13 Stepper and OLED Dual Stepper:** pins otherwise match their sketches. The TCA9548A address pins (D17 to D19 on the Dual Stepper, D2 to D4 on the Radio Auto Pilot) are not driven; the module's pull-downs set address 0x70.

20.5 KiCad: changing the revision number

In the PCB Editor use *File, Page Settings*, set the *Title Block* revision field, then save the board. To show it on the board, place a text item on the `F.Silkscreen` layer (press T) containing `Rev ${REVISION}`; other variables such as `${TITLE}`, `${ISSUE_DATE}` and `${COMPANY}` work the same way. The schematic and the PCB each keep their own title block, so set both if they should match. Make a fresh Gerber zip in `MANUFACTURING` before ordering.

Appendix A: UDP token reference

From the current sketches. Gauge tokens take real units; lamp tokens take 0 (off) or 1 (on). `UDP_Interface_Values.xlsx` still lists the old 16-servo plan, so use these tables instead. Every gauge token also has a `RAW` twin (`PITCHRAW`, `IASRAW`) that takes a raw servo position or step count and skips the calibration table.

A.1 Gauge tokens

Token	Gauge	Board	Unit
`PITCH`	Attitude pitch	Servo .102	degrees, -30 to +30
`BANK`	Attitude bank	Servo .102	degrees, -90 to +90
`IAS`	Airspeed	Stepper .105	knots, one decimal allowed
`VSI`	Vertical speed	Stepper .105	ft/min
`AGL`	Radar altitude	Stepper .105	feet, 0 to 2500
`TQ`	Engine torque	Stepper .105	percent, 0 to 120
`N1`	Gas producer	Stepper .105	percent, 0 to 103
`ITT`	Turbine out temperature	Stepper .105 or Dual .106	degrees C
`FUEL`	Fuel quantity	Stepper .105 or Dual .106	US gallons
`OILT`	Engine oil temperature	Dual .106 (or Stepper)	degrees C
`OILP`	Engine oil pressure	Dual .106 (or Stepper)	psi
`XMSNT`	Transmission oil temperature	Dual .106 (or Stepper)	degrees C
`XMSNP`	Transmission oil pressure	Dual .106 (or Stepper)	psi
`RPME`	Engine speed (N2)	Dual .106	percent, one decimal
`RPMR`	Rotor speed	Dual .106	percent, one decimal
`ALT`	Altimeter	Dual .106	feet
`FUELLOAD`	Fuel load	Dual .106	psi
`ELECTRICALLOAD`	Electrical load	Dual .106	percent

A.2 Display and status tokens

Token	Board	Meaning
`ZULU`	Dual .106	Zulu time as HHMM, for example `1430`
`TEMP`	Dual .106	Temperature text up to 8 characters, for example `25C`
`PWR`	Dual .106	1 = all four OLEDs on, 0 = blank (main bus below 24 V). Sent first in the packet
`C1A`, `C1S`, `C2A`, `C2S`	Radio .101	COM1 and COM2 active and standby frequency, MHz
`MAINBUS`	Radio .101	Main bus voltage
`NAVCOM1`	Radio .101	0 = avionics bus below 24 V (COM and NAV displays blank, panel LEDs off), 1 = on
`LED_CRS`, `LED_HDG`, `LED_IAS`, `LED_ALT`, `LED_APPR`, `LED_FD`, `LED_BC`, `LED_APENG`	Radio .101	Auto Pilot panel LEDs (CRS, HDG, IAS, ALT, APPR, FD, BC, AP ENGAGE): 1 = on
`LED_COM1`, `LED_BOTH`, `LED_NAV2`, `LED_DME`, `LED_OUTER`, `LED_INNER`, `LED_COM2`, `LED_NAV1`, `LED_MKR`, `LED_ADF`, `LED_MID`	Radio .101	Audio panel LEDs: 1 = on
`BPWR`	Servo .102, Stepper .105, Dual .106	Backlight power: 1 = main bus 20 V or more, 0 = backlight off

A.3 Lamp tokens (Servo .102)

`EOUT`, `RLOW`, `TOPW`, `TOTW`, `BHOT`, `BTMP`, `EC`, `TRC`, `TC`, `BD`, `FFLTR`, `FPMP`, `GENF`, `LOWF`, `SCF` on pins A1 to A15 in that order (see the Servo controller chapter).

A.4 Control tokens (C,<COMMAND>:<value>)

Command	Meaning
SWING<CODE>	Run one gauge's start-up swing on demand, for example C,SWINGTQ:1; the value is ignored
OLEDTEST	1 fills the four Dual Stepper OLEDs solid white, 0 restores them
KEEPALIVE	Ignored command that still counts as received data and restarts the 30 second no-data timer
RESET	Reboot the board about 2 seconds later (port 13136; USB HID board on 4210)
MATRIX	Radio and ADF controllers: print the raw keypad matrix to the debug stream
LEDTEST	Radio controller: 1 lights all 19 Auto Pilot and Audio panel LEDs, 0 returns them to the simulator's state
POWER	ADF controller: send a power status (12 V and 5 V rails) now
B	Backlight brightness 0 to 15 (0 off, 15 full): Servo, Stepper and Dual Stepper; the Radio sets the intensity of its Auto Pilot and Audio panel LEDs from it (0 = LEDs off), the Upper ignores it. The ADF controller's pot sends it to the other panels

A.5 Messages from the ADF controller to the PC (UDP 27003)

ADF_FREQ:<kHz> (190 to 1799, sent 250 ms after the knobs stop), RADAR_FOLLOW_ID:<digits> (RADAR_FOLLOW_ID:0 = stop following) and POWER,V12:<volts>,V5:<volts>,STATUS:<OK, or 5V_LOW, 5V_HIGH, 12V_LOW, 12V_HIGH joined with +> every 30 seconds. No bridge reads this port yet.

A.6 Aircraft information to the GPS interface (UDP 27002)

D,LAT:<deg>,LON:<deg>,ALT:<feet>,IAS:<knots>,HDG:<degrees true>,UTC:<yyyyMMddHHmmss[.fff]>, every 500 ms from a bridge or the bench tester. Any subset of keys is accepted; a CSV form lat,lon,altitude_ft,airspeed_kt,heading_deg[,utc] also works. South and west are negative.

A.7 Messages from the Radio controller to the bridge (UDP 27001)

COM1_RADIO_SWAP, COM2_RADIO_SWAP, NAV1_RADIO_SWAP, NAV2_RADIO_SWAP, AVIONICS_MASTER_SET, MASTER_BATTERY_ON, MASTER_BATTERY_OFF, ALTERNATOR_ON, ALTERNATOR_OFF, COM2_STANDBY:<MHz>, NAV1_STANDBY:<MHz>, NAV2_STANDBY:<MHz> and a bare number for the COM1 standby frequency. The Radio controller also sends the Auto Pilot and Audio panel controls and the ADF controller the gear, flaps, speed brake and brake controls, each as the name of the simulator control with an optional :N repeat count: AP_MASTER, AP_HDG_HOLD, AP_NAV1_HOLD, AP_ALT_HOLD, AP_APR_HOLD, TOGGLE_FLIGHT_DIRECTOR, AP_BC_HOLD, AP_PANEL_SPEED_HOLD, VOR1_OBI_INC, VOR1_OBI_DEC, AP_ALT_VAR_INC, AP_ALT_VAR_DEC, AP_VS_VAR_INC, AP_VS_VAR_DEC, AP_SPD_VAR_INC, AP_SPD_VAR_DEC, HEADING_BUG_INC, HEADING_BUG_DEC, COM1_TRANSMIT_SELECT, COM2_TRANSMIT_SELECT, COM_RECEIVE_ALL_TOGGLE, RADIO_VOR1_IDENT_TOGGLE, RADIO_VOR2_IDENT_TOGGLE, RADIO_DME1_IDENT_TOGGLE, RADIO_ADF_IDENT_TOGGLE, MARKER_SOUND_TOGGLE, GEAR_UP, GEAR_DOWN, FLAPS_UP, FLAPS_DOWN, SPOILERS_ON, SPOILERS_OFF and PARKING_BRAKES.

Appendix B: Building and flashing

B.1 Libraries

Ethernet 2.0.2 with SPI, AccelStepper 1.64.0 (steppers), Servo and Ewma (Servo controller), U8g2 and Wire (OLEDs), DCS-BIOS 0.3.13 (sketches that still carry DCS-BIOS code), and the Joystick and Keyboard libraries for the HID board. Install them from the Arduino Library Manager.

B.2 Compile and upload

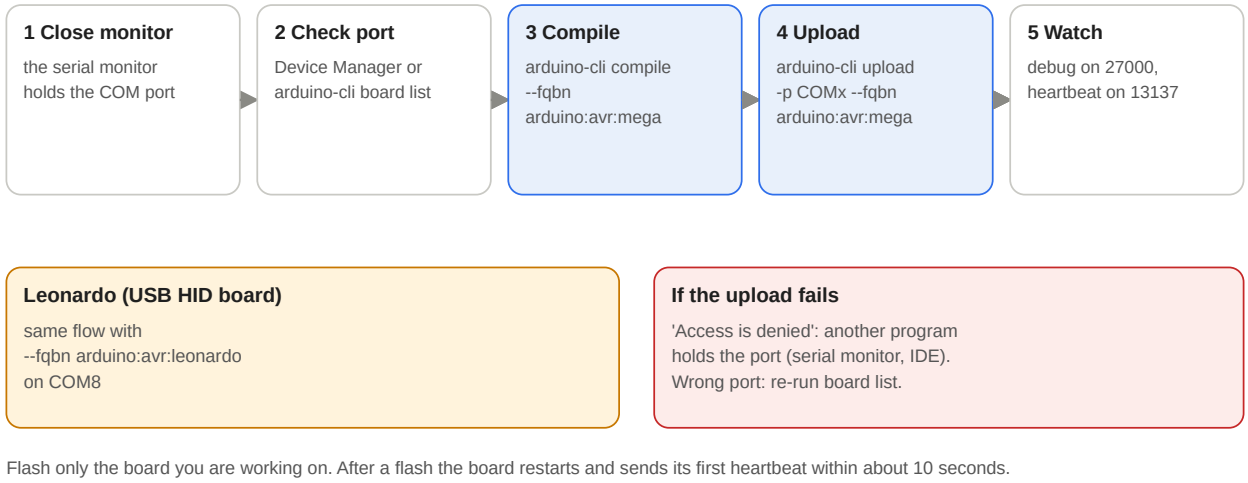
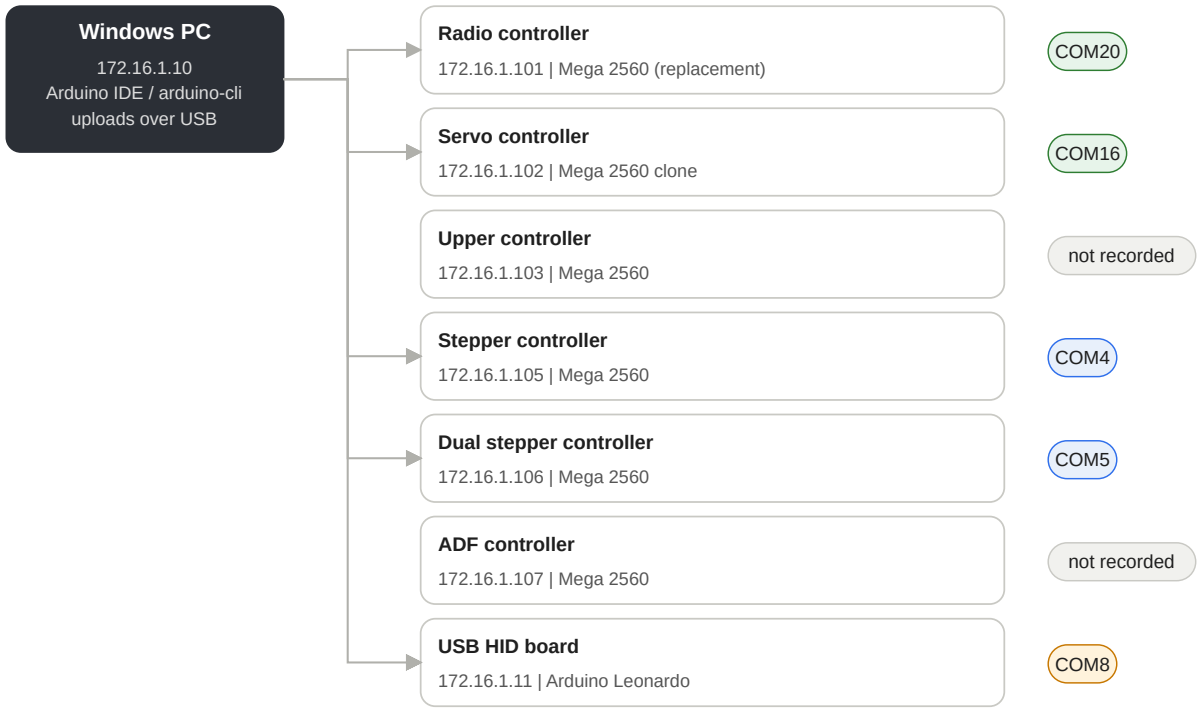


Figure 33 — Flashing a board

```
arduino-cli compile --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
arduino-cli upload -p COM4 --fqbn arduino:avr:mega "Jet Ranger Arduino
Sketches\JET_RANGER_STEPPER_CONTROLLER"
```

Use `--fqbn arduino:avr:leonardo` for the USB HID board. Close the Arduino IDE serial monitor first: it holds the port ("Access is denied"). Check the port with Device Manager or `arduino-cli board list`.

B.3 COM port to board mapping



COM numbers are the ones used at the last flash. They follow the USB socket and the board, so check them (see the guide) before an upload. Each board also has a network connection: USB is for programming only, the cockpit runs over UDP.

Figure 34 — Which board is on which COM port

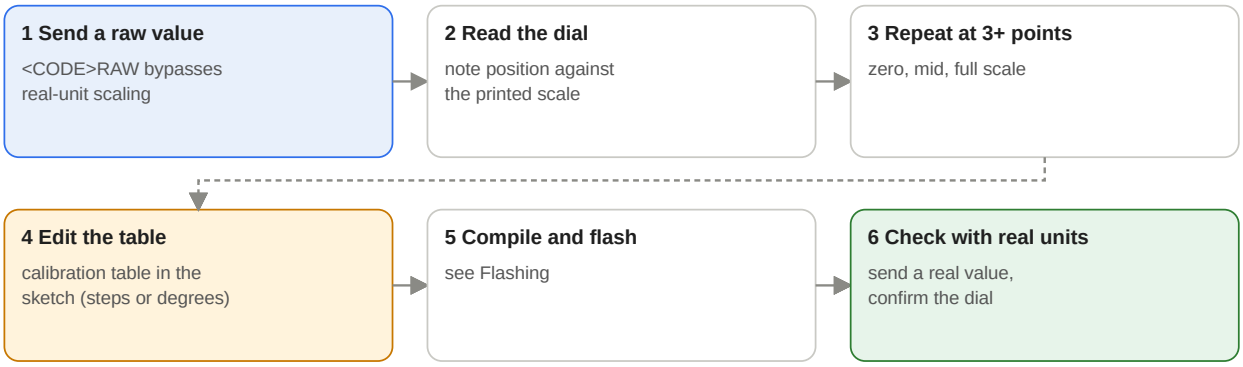
Board	IP	Hardware	COM port
Radio controller	.101	Mega 2560 (replacement board)	COM20
Servo controller	.102	Mega 2560 clone	COM16
Upper controller	.103	Mega 2560	not recorded
Stepper controller	.105	Mega 2560	COM4
Dual stepper controller	.106	Mega 2560	COM5
ADF controller	.107	Mega 2560	not recorded
USB HID board	.11	Arduino Leonardo	COM8

The Lowrance GPS cable is not one of these boards: the GPS interface asks for its serial port the first time it runs and remembers it. These are the ports used at the last flash of each board. They follow the USB socket and the board: the Servo clone has no USB serial number, so its COM number is tied to the socket it is plugged into, and a replaced Mega gets a new number. Check before every upload.

B.4 Sketch settings to know about

- Per-gauge `SwingXXX` gates and `SwingLoops` near the top of the Stepper and Dual Stepper sketches control which swings run at boot.
- Zero trims (`<CODE>ZeroOffset`, `TSoffset`, `RSoffset`) are set near the top of the same sketches.
- Every sketch has `reflectorIP` (172.16.1.10) for debug and heartbeat traffic.

Appendix C: Calibration



Typical tables: pitch -30 / 0 / +30 degrees, bank -90 / 0 / +90 degrees, one row per stepper or servo gauge.

Sending <CODE>RAW:n moves the motor to step n directly, so any dial can be calibrated without a simulator.

Figure 35 — The calibration loop

Between table points the boards interpolate in a straight line and hold the end value beyond the ends.

1. Send a **raw** value (`D,TQRAW:300`) to move the needle to a raw step or servo position.
2. Read the dial against its printed scale.
3. Repeat at three or more points (zero, mid scale, full scale).
4. Edit the calibration table in the sketch.
5. Compile and upload.
6. Send **real** values (`D,TQ:60`) and confirm the dial.

The StepperVSITester sends every raw and real token without a simulator. Do not drive a gauge hard against its stop: a stalled stepper or servo draws high current and gets hot. After any mechanical change to a gauge, recheck its table and zero trim.

Appendix D: How this document is built

This document and the User Guide are generated, not typeset by hand. The sources are Markdown files and vector diagrams in `Jet Ranger Documentation/DocBuild`:

Item	Purpose
`src/master/*.md`, `src/guide/*.md`	Text of this document and the User Guide
`diagrams_a.py` to `diagrams_d.py`, `svgdraw.py`, `make_diagrams.py`	The 34 diagrams, written as Python functions that draw SVG; `python make_diagrams.py` regenerates them
`build_docs.py`, `run_lo.ps1`	Builds the `.odt` and `.pdf` through LibreOffice's Python interface (`run_lo.ps1 build_docs.py master`)
`pdf2png.ps1`, `montage.ps1`, `svg2png.py`	Helpers for checking pages and diagrams visually

To change a diagram edit its function and rebuild; to change text edit the Markdown and rebuild. Keep `PROGRAM_SUMMARY.md` files and these sources in step after a code change.